

Threat Model - Juice Shop

Generated by appsec-advisor v0.6.0-beta.3 (analysis v5)

Project	Juice Shop v20.1.1
Description	Probably the most modern and sophisticated insecure web application
Author	Björn Kimminich <bjoern.kimminich@owasp.org> (https://kimminich.de)
License	MIT
Repository	https://github.com/juice-shop/juice-shop.git
Homepage	https://owasp-juice.shop
Runtime	Node.js 22 - 26, Express 4
Tags	web security, web application security, webappsec, owasp, pentest, pentesting, security, vulnerable, vulnerability, broken, bodgeit, ctf, capture the flag, awareness

Changelog

Append-only history of assessment runs. Most recent first.

Version	Date	Mode	Depth	Reasoning	Baseline → Current	Δ Threats	Code	Note
v1	2026-09-13 11:16 CEST	full	thorough	opus	(initial)	68 total	-	first full scan

Table of Contents

- Management Summary 4
- Critical Attack Tree 16
- 1. System Overview 17
 - Scope 17
 - Identified Actors 17
 - Trust Boundaries 17
- 2. Architecture Diagrams 19
 - 2.1 System Context. 19
 - 2.2 Container Architecture 20
 - 2.3 Components 21
 - 2.4 Technology Architecture 25
- 3. Attack Walkthroughs 26
 - 3.1 Insecure JWT Verification in JWT Authentication and Session Management 26
 - 3.2 SQL injection in login query 27
 - 3.3 Mass assignment of role on registration in Express js Backend API 28
 - 3.4 Server-side eval of stored username in User Profile 29
 - 3.5 Cross-Site Scripting in Search Result. 30
 - 3.6 Hard-coded JWT signing key in JWT Authentication and Session Management. . . 31
 - 3.7 Dependency lockfile disabled by config in Npmrc. 32
 - 3.8 Coupon discount bounded only by prompt text in Chat 33
- 4. Assets. 34
- 5. Attack Surface 39
 - 5.1 Unauthenticated Entry Points (55) 39
 - 5.2 Authenticated Entry Points (52) 42
- 6. Security Architecture 45
 - 6.1 Security Control Overview. 45
 - 6.2 Identity and Authentication Controls. 46
 - 6.3 Session and Token Controls. 53
 - 6.4 Authorization Controls. 55

◦ 6.5 Query Construction and Data Access Controls	58
◦ 6.6 Input Boundary Validation Controls	59
◦ 6.7 Output Encoding and Rendering Controls	60
◦ 6.8 Browser and Cross-Origin Controls	61
◦ 6.9 Cryptography Secrets and Data Protection	62
◦ 6.10 File Parser and Outbound Request Controls	64
◦ 6.11 Operations Runtime and Supply Chain Controls	65
◦ 6.12 Real-time and Not Applicable Controls	67
◦ 6.13 Defense-in-Depth Summary	68
7. Weakness Register	69
8. Findings Register	76
9. Abuse Cases	120
10. Mitigation Register	126
11. Out of Scope	178
• Not Covered by This Method	178
• Excluded from This Assessment.	179
• Appendix: Run Statistics	179
• Appendix A - Vektor Taxonomy.	180

Management Summary

Verdict

● Multiple critical defects allow unauthenticated attackers to seize any account, forge administrator credentials, extract the full customer database, and execute arbitrary server commands.

Risk distribution: ● Critical: 8 · ● High: 30 · ● Medium: 20 · ● Low: n/a · **Total: 58**

Reporting threshold: medium - Low and Informational excluded

Assessment evidence: 49 confirmed-exploitable finding(s) · 5 implementation weakness(es) · 7 design weakness(es)

Basis: a code-derived threat model at implementation level - built from repository evidence, not a planning document. Design intent, business processes, runtime behaviour and production-only configuration are outside what this analysis can see (see [§11 Out of Scope](#)).

What an attacker can do today, worst first:

- **Active login stolen** — Stored cross-site scripting (XSS) lets a page visitor steal session tokens from browser memory and replay them as the victim. (● [F-001](#) — *Cross-Site Scripting*, ● [F-002](#) — *JWT in localStorage (request.interceptor.ts:13)*) — ✓ verified attack path
- **Admin impersonation** — A hard-coded token signing key lets anyone forge a credential claiming full administrator rights without knowing any password. (● [F-006](#) — *Hard-coded JWT signing key (lib/insecurity.ts:21)*, ● [F-007](#) — *Insecure JWT Verification* → [W-004](#))
- **Customer data exfiltrated** — SQL injection on the login form allows unauthenticated read of all user records and order history. (● [F-008](#) — *SQL injection in login query*, ● [F-009](#) — *Insecure Direct Object Reference* → [W-001](#), [W-002](#))
- **Server takeover** — Server-side code execution via a stored display name lets any logged-in user run arbitrary commands on the host. (● [F-011](#) — *Server-side eval of stored username (routes/userProfile.ts:61)*) — ✓ verified attack path
- **Self-granted admin account** — Mass assignment on account registration lets any new user supply a `role` field and create their own administrator account. (● [F-012](#) — *Mass assignment of role on registration*) — ✓ verified attack path
- **Supply chain backdoor** — A disabled dependency version pin lets a poisoned third-party package silently `replace` application code at the next build. (● [F-010](#) — *Dependency lockfile disabled by config — .npmrc:1* → [W-003](#))

Fix all eight critical defects before any production deployment to prevent account compromise, data breaches, and server-level intrusion.

Top Weaknesses

The systemic root-cause problems behind the findings - the classes to fix, not just individual bugs. Each links to the full [Weakness Register](#) with its findings and remediation.

- ● **W-001 - Database access relies on concatenated queries** (Critical) - Database queries are assembled from application values instead of passing those values through an enforced parameterised data-access path. *Proven by [F-008](#).*
- ● **W-002 - Authorization is implemented route by route** (Critical) - Authorization depends on per-handler checks instead of a policy boundary that consistently enforces role, ownership, and tenant scope. *Proven by ● [F-009](#), ● [F-038](#), ● [F-039](#) (+2 more).*
- ● **W-003 - Build pipeline trusts mutable third-party references** (Critical) - CI/CD workflows resolve third-party actions and other build dependencies to mutable tags or branches instead of immutable commit digests, so a retagged or compromised upstream runs inside the pipel... *Proven by [F-010](#), [F-022](#).*
- ● **W-004 - Secrets are committed to source instead of a managed store** (Critical) - Cryptographic keys, credentials, and other high-entropy secrets are embedded as literals in source or `config` rather than resolved at runtime from a managed secret store, so anyone with repository r... *Proven by ● [F-006](#), ● [F-018](#), ● [F-031](#) (+2 more).*
- ● **W-005 - Endpoints are reachable without enforced authentication** (High) - Sensitive API routes and real-time channels are exposed without an enforced authentication check at the endpoint boundary. *Proven by [F-017](#).*
- ● **W-006 - Denial of Service is implemented inconsistently** (High) - Resource-consuming operations lack rate limiting or bounds, so a single actor can exhaust capacity. *Proven by [F-034](#), [F-061](#).*

Open Questions for the Team

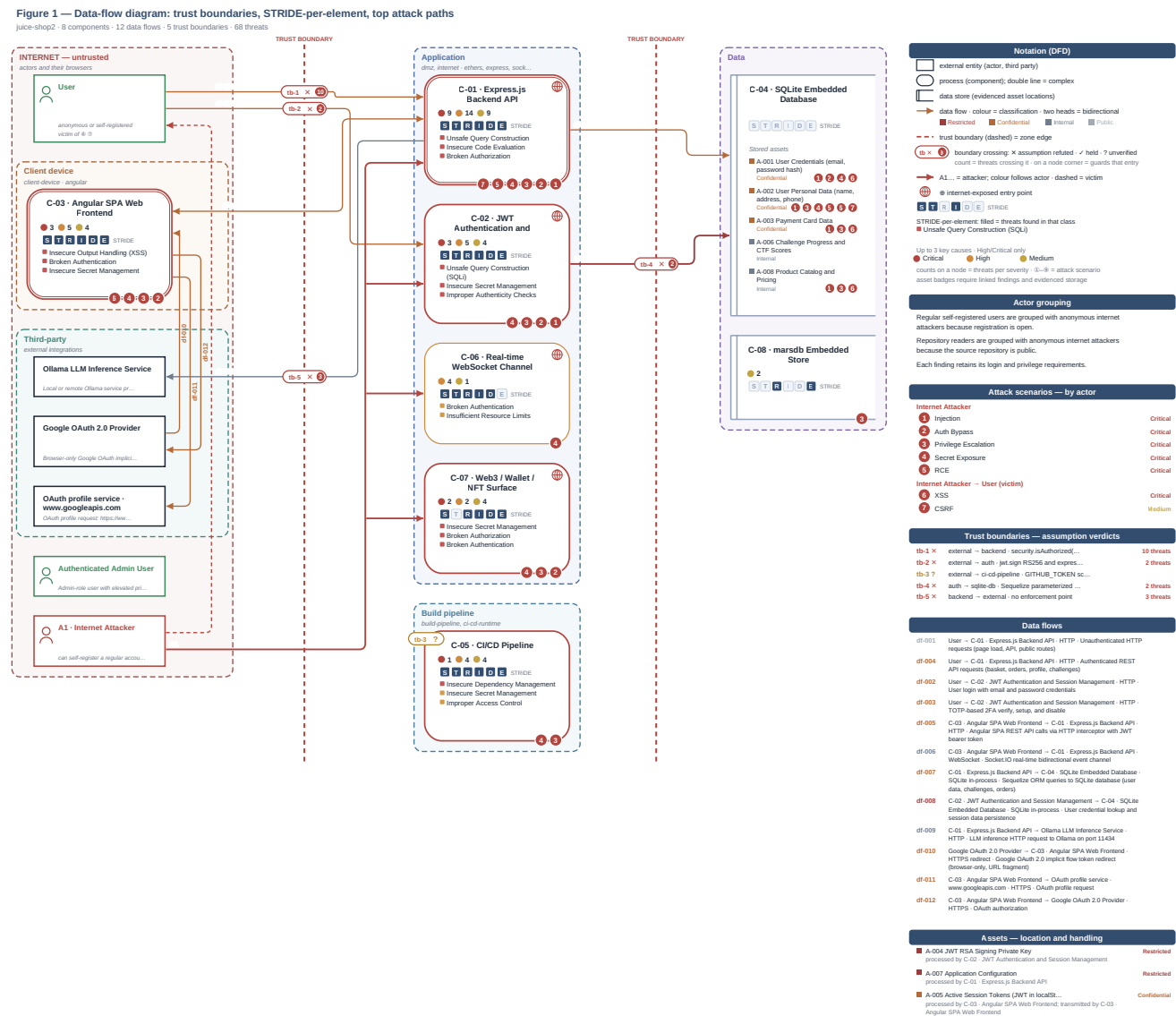
The code cannot settle these points. They depend on deployment or business decisions and are the input for a manual threat-modeling session.

- **W-002:** ● [F-009](#) (unproven), ● [F-038](#) — Coupon discount bounded only by prompt text (`routes/chat.ts:184`), ● [F-039](#) (+2 more) - Which cross-user or cross-tenant accesses are intended (support, admin bulk operations), and which single policy layer should enforce ownership and role for every route?
- **W-003:** [F-010](#), [F-022](#) — Who can change build inputs or publish artifacts, and which independent approval applies before a pipeline change takes effect?
- **W-004:** ● [F-006](#) — Hard-coded JWT signing key (`lib/insecurity.ts:21`), ● [F-018](#) (unproven), ● [F-031](#) (+1 more) - These secrets stay in git history: which deployments used them, who rotates them, and where do they live afterwards?
- [F-009](#), [F-018](#), [F-021](#), [F-040](#), [F-049](#) (+4 more) — Unverified evidence: confirm or rule out what the code alone could not establish before scheduling the fix.

Security Posture & Top Threats

Figure 1 - Architecture & Top Threats

Data-flow diagram: external entities, processes and data stores in their trust zones (Internet → Application → Data), the data flows between them, and the attack scenarios numbered as in the table below. Each crossing of a dashed trust-boundary line carries the `tb-N` id catalogued in §1 Trust Boundaries with the verdict on its enforcement assumption. The in-figure legend on the right explains the notation.

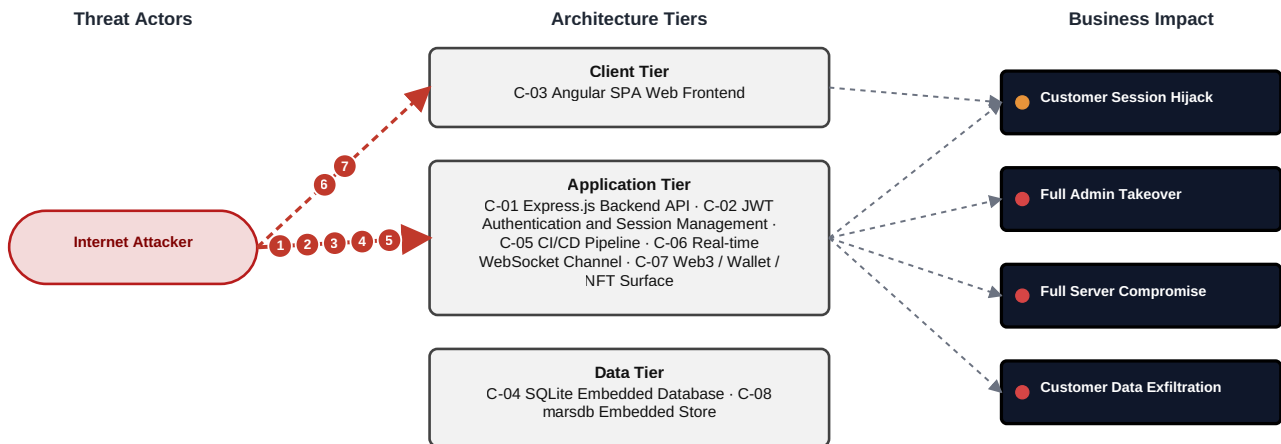


Anonymous and authenticated regular users share one card because self-registration is open. Individual flows may still require login.

Actor grouping. Regular self-registered users are grouped with anonymous internet attackers because registration is open. Repository readers are grouped with anonymous internet attackers because the source repository is public. Each finding retains its login and privilege requirements.

Figure 2 - Risk Flow: Actor → Tier → Impact

Heatmap: **actors** (left) → **architecture tiers** (middle, Client → Application → Data) → **impact** (right). Numbered red arrows ①–⑦ are the threats enumerated in the Top Threats table below.



Threat actors. The actors below drive the numbered attack paths in the figures above. The **Shop User** is the *victim* of client-side attacks (XSS / CSRF), not an attacker - in Figure 2 the compromise surfaces as the resulting business-impact node rather than as a separate actor box.

- **Shop User** — legitimate customer; target of client-side attacks; target of ⑥ Output Encoding / Cross-Site Scripting, ⑦ CSRF / Permissive CORS.
- **Internet Attacker** — can self-register a regular account; drives ① Insecure Query Construction & Data Access, ② Hardcoded Secrets & Weak Cryptography, ③ Broken Authorization & Access Control, ④ Sensitive File & Secret Exposure, ⑤ Remote Code Execution (unsafe eval).

7 structural threats, grouped by weakness class - each row is one threat, not one finding. *Threat Description* states the general architectural weakness (STRIDE in brackets); *Findings* lists the concrete instances, each linked to §8 Findings Register with its component; *Risk & Impact* combines severity with business consequence.

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
①	Insecure Query Construction & Data Access (T-I) Unsanitized query construction in authentication and order-tracking routes exposes the entire database to unauthenticated	● F-008 - SQL injection in login query (login.ts:34) → C-02 JWT Authentication and Session Management ● F-009 - Insecure Direct Object Reference (address.ts:11) → C-01 Express.js Backend API ● F-019 - NoSQL ` \$where JavaScript injection	● Critical Customer Data Exfiltration	● M-018 — Use parameterized database queries ● M-019 — Enforce object-level (ownership)

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
	extraction or manipulation.	(trackOrder.ts:18) → [C-01] (#c-01) Express\js Backend API → [F-021] (#f-021) - Input compiled as template source (userProfile.ts:87`) → C-01 Express.js Backend API		authorizatio n
②	Hardcoded Secrets & Weak Cryptography (S·E) Hard-coded and algorithm-confusion weaknesses in session token signing allow unauthenticated forging of valid administrator credentials, while unvalidated OAuth callbacks provide additional account-takeover vectors.	● F-006 - Hard-coded JWT signing key (insecurity.ts:21) → C-02 JWT Authentication and Session Management ● F-007 - Insecure JWT Verification (insecurity.ts:52) → C-02 JWT Authentication and Session Management ● F-015 - Unvalidated OAuth token (oauth.component.ts:28) → C-03 Angular SPA Web Frontend ● F-016 - Predictable derived credential (oauth.component.ts:30) → C-03 Angular SPA Web Frontend ● F-018 - Hard-coded seeded account credentials (datacreator.ts:200) → C-01 Express.js Backend API ● F-020 - Non-cryptographic RNG for a secret/token (insecurity.ts:53) → C-02 JWT Authentication and Session Management ● F-025 - Unsalted MD5 password hashing (insecurity.ts:41) → C-02 JWT Authentication and Session Management ● F-031 - Hard-coded wallet	● Critical Full Admin Takeover · Customer Session Hijack	● M-016 — Move cryptographic keys to a managed secret store ● M-017 — Enforce JWT signature and algorithm verification

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
		mnemonic (checkKeys.ts:10) → C-07 Web3 / Wallet / NFT Surface ● F-056 - Hard-coded test credential (login.component.ts:62) → C-03 Angular SPA Web Frontend ● F-066 - Security answers stored under embedded HMAC key (datacreator.ts:691) → C-01 Express.js Backend API ● F-073 - Token flow accepts a stolen bearer token without (login.component.ts:148) → C-03 Angular SPA Web Frontend		
③	Broken Authorization & Access Control (E·I) Unfiltered object binding on account registration and client-only authorization guards allow ordinary users to self-grant administrator roles or bypass access controls.	● F-003 - Client-side authorization decision (app.guard.ts:54) → C-03 Angular SPA Web Frontend ● F-005 - Unmediated data-store access control (mongodb.ts:10) → C-08 marsdb Embedded Store ● F-009 - Insecure Direct Object Reference (address.ts:11) → C-01 Express.js Backend API ● F-012 - Mass assignment of role on registration (server.ts:484) → C-01 Express.js Backend API ● F-036 - Password change without current password (changePassword.ts:39) → C-02 JWT Authentication and Session Management ● F-038 - Coupon discount bounded only by prompt text (chat.ts:184) → C-01 Express.js Backend API	● Critical Full Admin Takeover	● M-019 — Enforce object-level (ownership) authorization ● M-022 — Allowlist client-controlled fields

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
		● F-039 - Deluxe upgrade skips payment check (<code>deluxe.ts:43</code>) → C-01 Express.js Backend API ● F-040 - Sensitive Routes Registered Without Authentication Middleware (<code>server.ts:310</code>) → C-01 Express.js Backend API ● F-041 - Missing workflow permissions block (<code>ci.yml:190</code>) → C-05 CI/CD Pipeline ● F-065 - Basket item update without ownership check (<code>server.ts:426</code>) → C-01 Express.js Backend API		
④	Sensitive File & Secret Exposure (I) Credentials stored with weak hashing, session tokens held in unprotected browser memory, and application logs accessible without authentication collectively expose customer account and authentication data.	● F-002 - JWT in <code>localStorage</code> (<code>request.interceptor.ts:13</code>) → C-03 Angular SPA Web Frontend ● F-025 - Unsalted MD5 password hashing (<code>insecurity.ts:41</code>) → C-02 JWT Authentication and Session Management ● F-026 - Denylisted path traversal in erasure layout (<code>dataErasure.ts:104</code>) → C-01 Express.js Backend API ● F-027 - HTTP access logs browsable without authentication (<code>server.ts:281</code>) → C-01 Express.js Backend API ● F-028 - Unvalidated URL fetch in profile image (<code>profileImageUrlUpload.ts:24</code>) → C-01 Express.js Backend API ● F-029 - Secrets broadcast to job environment (<code>ci.yml:253</code>) → C-05 CI/CD Pipeline ● F-030 - Unauthenticated data exposure	● Critical Customer Data Exfiltration · Customer Session Hijack	● M-035 — Hash passwords with a strong, salted algorithm ● M-036 — Constrain file paths to a safe base directory

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
		(registerWebsocketEvents.ts:30) → C-06 Real-time WebSocket Channel ● F-031 - Hard-coded wallet mnemonic (checkKeys.ts:10) → C-07 Web3 / Wallet / NFT Surface ● F-044 - Open redirect (redirect.ts:19) → C-07 Web3 / Wallet / NFT Surface ● F-052 - Chat tool calls streamed to every caller (chat.ts:228) → C-01 Express.js Backend API ● F-053 - Admin configuration endpoints exposed unauthenticated (server.ts:607) → C-01 Express.js Backend API ● F-054 - Development error handler returns stack traces (server.ts:682) → C-01 Express.js Backend API ● F-055 - Client-gated tool-call disclosure (chat-conversation.component.ts:68) → C-03 Angular SPA Web Frontend ● F-058 - Ineffective author email masking in feedback (datacreator.ts:578) → C-01 Express.js Backend API ● F-059 - Plaintext security answer written to log (datacreator.ts:692) → C-01 Express.js Backend API ● F-060 - Raw provider error returned to caller (web3Wallet.ts:36) → C-07 Web3 / Wallet / NFT Surface		
⑤	Remote Code Execution (unsafe	● F-011 - Server-side eval of stored username		● M-021 — Remove

#	Threat Description	Findings (→ Component)	Risk & Impact	Fix
	eval) (E) Server-side dynamic code execution of attacker-controlled profile content allows any authenticated user to run arbitrary commands directly on the host.	(userProfile.ts:61) → C-01 Express.js Backend API	● Critical Full Server Compromise	server-side evaluation of untrusted input
⑥	Output Encoding / Cross-Site Scripting (T·I) Reflected and stored script injection through the search interface and product reviews executes attacker payloads in victim browsers, enabling session token theft from unprotected browser storage.	● F-001 - Cross-Site Scripting (search-result.component.ts:143) → C-03 Angular SPA Web Frontend ● F-002 - JWT in localStorage (request.interceptor.ts:13) → C-03 Angular SPA Web Frontend	● Critical Customer Session Hijack	● M-011 — Encode output instead of bypassing the framework sanitizer ● M-012 — Store session tokens in HttpOnly , Secure cookies
⑦	CSRF / Permissive CORS (S·T) a permissive CORS policy plus missing anti-CSRF tokens let any external page issue authenticated state-changing requests in the victim's session.	● F-045 - Cookie-only authentication on POST / profile (updateUserProfile.ts:17) → C-01 Express.js Backend API	● Medium Customer Session Hijack	● M-055 — Add anti-CSRF protection to state-changing requests

STRIDE: S spoofing · T tampering · R repudiation · I information disclosure · D denial of service · E elevation of privilege. Risk, findings, components, impact and Fix are derived deterministically; only the one-line weakness description is authored.

Verified attack chains. 3 fully viable ([AC-T-001](#), [AC-T-004](#), [AC-T-006](#)); 1 partially blocked ([AC-T-002](#)). These chains combine individual findings into end-to-end exploitation paths verified step-by-step against the code - see [§9 Abuse Cases](#) for the per-step breakdown and blocking mitigations.

Top Mitigations

Highest-impact P1/P2 mitigations - 13 of 38 qualifying (78 total). Full detail in [§10 Mitigation Register](#). All 13 mitigation(s) that fix a Critical finding are always listed here.

#	Component	Mitigation	Addresses	Effort
1	C-01 — Express.js Backend API	● M-021 — Remove server-side evaluation of untrusted input (<code>userProfile.ts:61</code>)	● F-011 — Server-side eval of stored username (<code>routes/userProfile.ts</code>)	Low
2	C-01 — Express.js Backend API	● M-022 — Allowlist client-controlled fields (<code>server.ts:484</code>)	● F-012 — Mass assignment of role on registration (<code>server.ts</code>)	Low
3	C-01 — Express.js Backend API	● M-019 — Enforce object-level (ownership) authorization (<code>address.ts:11</code>)	● F-009 — Insecure Direct Object Reference (<code>routes/address.ts</code>)	Medium
4	C-02 — JWT Authentication and Session Management	● M-017 — Enforce JWT signature and algorithm verification (<code>insecurity.ts:52</code>)	● F-007 — Insecure JWT Verification (<code>lib/insecurity.ts</code>)	Low
5	C-02 — JWT Authentication and Session Management	● M-018 — Use parameterized database queries (<code>login.ts:34</code>)	● F-008 — SQL injection in login query (<code>routes/login.ts</code>)	Low
6	C-02 — JWT Authentication and Session Management	● M-016 — Move cryptographic keys to a managed secret store (<code>insecurity.ts:21</code>)	● F-006 — Hard-coded JWT signing key (<code>lib/insecurity.ts</code>)	Medium
7	C-03 — Angular SPA Web Frontend	● M-011 — Encode output instead of bypassing the framework sanitizer (<code>search-result.component.ts:143</code>)	● F-001 — Cross-Site Scripting (<code>frontend/src/app/search-result/search-result.component.ts</code>)	Low
8	C-05 — CI/CD Pipeline	● M-020 — Pin third-party dependencies to immutable versions (<code>.npmrc:1</code>)	● F-010 — Dependency lockfile disabled by config (<code>.npmrc</code>)	Medium
9	C-01 — Express.js Backend API	● M-029 — Use parameterized database queries (<code>trackOrder.ts:18</code>)	● F-019 — NoSQL \$where JavaScript injection (<code>routes/trackOrder.ts</code>)	Low
10	C-01 — Express.js Backend API	● M-049 — Enforce correct server-side authorization (<code>deluxe.ts:43</code>)	● F-039 — Deluxe upgrade skips payment check (<code>routes/deluxe.ts</code>)	Low

#	Component	Mitigation	Addresses	Effort
11	C-01 — Express.js Backend API	● M-048 — Enforce correct server-side authorization (chat.ts:184)	● F-038 — Coupon discount bounded only by prompt text (routes/chat.ts)	Medium
12	C-03 — Angular SPA Web Frontend	● M-025 — Bind the OAuth callback in oauth.component.ts to a generated state value (oauth.component.ts:28)	● F-015 — Unvalidated OAuth token (oauth.component.ts)	Medium
13	C-07 — Web3 / Wallet / NFT Surface	● M-041 — Move secrets to a managed secret store (checkKeys.ts:10)	● F-031 — Hard-coded wallet mnemonic (routes/checkKeys.ts)	Medium

25 additional P1/P2 mitigations capped from the leader-board · 40 P3 backlog items in [§10 Mitigation Register](#). Sorted by priority (P1 first), then component, then leverage (most findings first), severity (Critical first), and effort (Low first).

AI / LLM Exposure

The application embeds a tool-calling chatbot backed by an Ollama LLM that can apply discounts and access internal state, creating prompt injection, excessive agency, and information disclosure risks. See [§6 Security Architecture](#) for the per-control detail.

- **LLM01 / ASI01 Prompt Injection** — User chat input is passed directly to the model without sanitization, allowing an attacker to override system instructions and trigger tool calls that apply unauthorized coupon discounts or access restricted backend actions. (C-01 — Express.js Backend API)
 - ↳ ● F-038 — Coupon discount bounded only by prompt text (chat.ts:184)
- **LLM06 / ASI02 Excessive Agency** — The model has tool-calling access to privileged backend operations without authorization checks on individual tool invocations, so a manipulated model can execute actions beyond the caller's permitted scope. (C-01 — Express.js Backend API)
 - ↳ ● F-038 — Coupon discount bounded only by prompt text (chat.ts:184)
 - ↳ ● F-052 — Chat tool calls streamed to every caller (chat.ts:228)
- **LLM02 Sensitive Information Disclosure** — Internal tool-call results and model reasoning traces are streamed to every connected chat client regardless of their privilege level, exposing application internals to ordinary or unauthenticated users. (C-01 — Express.js Backend API, C-03 — Angular SPA Web Frontend)
 - ↳ ● F-052 — Chat tool calls streamed to every caller (chat.ts:228)
 - ↳ ● F-055 — Client-gated tool-call disclosure (chat-conversation.component.ts:68)

- **LLM10 / ASI08 Unbounded Consumption** — The chat endpoint applies no per-user token budget, request rate limit, or conversation-history cap, allowing any caller to drive unbounded inference cost and degrade service availability. ([C-01](#) — *Express.js Backend API*, [C-03](#) — *Angular SPA Web Frontend*)

- ↳ ● [F-034](#) — Unbounded LLM consumption on chat endpoint (`server.ts:638`)
- ↳ ● [F-063](#) — Unbounded chat history resend (`chat-conversation.component.ts:131`)

Operational Strengths

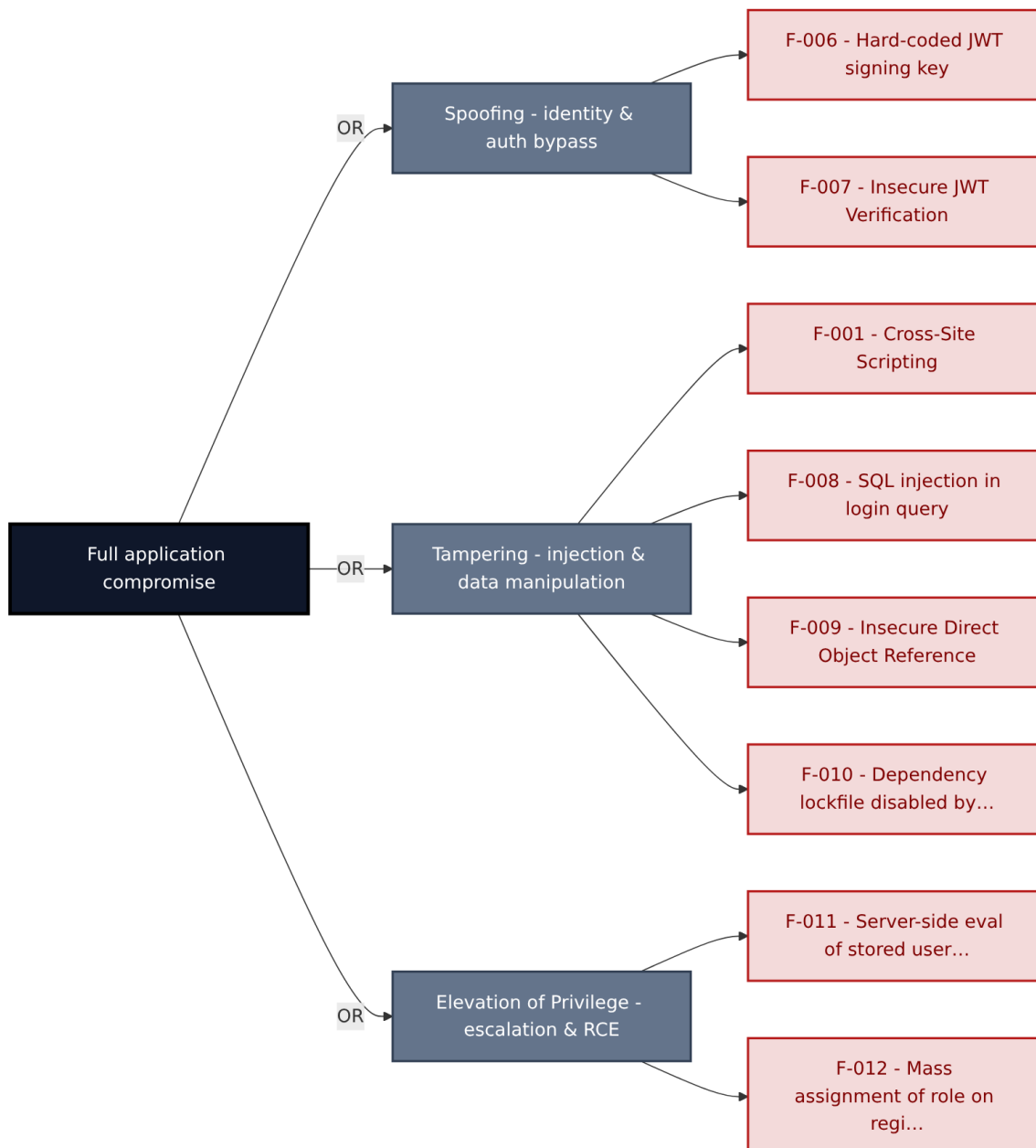
Operational controls rated Adequate or Partial - grouped into broad clusters (full per-control breakdown in §6). Clusters demoted to Weak by open Critical/High findings appear in §6 instead, not here.

Strength	What's in Place	Effectiveness	Mitigates
Hardened HTTP Stack	<i>Browser-facing HTTP hardening — security headers, cookie flags, cross-origin policy, and abuse-protection limits.</i> CORS and WebSocket <code>Origin</code> Enforcement	▲🕒 Partial — Bypassed by 1 High finding(s) of the kind this cluster is supposed to prevent — e.g. ● F-028 — Unvalidated URL fetch in profile image (<code>routes/profileImageUrlUpload.ts:24</code>).	● F-045 — Cookie-only authentication on POST <code>/profile</code> (<code>routes/updateUserProfile.ts:17</code>)
Observability & Audit	<i>Runtime visibility - access logging, audit trails, and operational telemetry for post-incident review.</i> Security Event Logging	▲🕒 Partial - Coverage incomplete - see §7 control assessment.	-
Container & Supply-Chain Hardening	<i>Build-time and runtime hardening - minimal base image, non-root execution, dependency inventory.</i> Third-Party CI/CD Action Pinning	▲🕒 Partial - Coverage incomplete - see §7 control assessment.	-
Cryptographic Hygiene	<i>Transport security, key lifecycle, and password-hash strength.</i> Transport Layer Encryption (TLS)	▲🕒 Partial - Bypassed by 2 High finding(s) of the kind this cluster is supposed to prevent - e.g. ● F-025 — Unsalted MD5 password hashing ● F-020 — Non-cryptographic RNG for a secret/token (<code>lib/insecurity.ts:53</code>).	-

Bottom line: These controls narrow specific attack surfaces but none eliminates a Critical finding on its own.

Critical Attack Tree

The root is the worst-case attacker goal; below it, each capability branch groups the Critical findings that achieve it. Branches feed the goal by OR - any single path suffices.



Findings (full detail in [§8 Findings Register](#)): [F-006](#) · [F-007](#) · [F-001](#) · [F-008](#) · [F-009](#) · [F-010](#) · [F-011](#) · [F-012](#)

1. System Overview

Repository: <https://github.com/juice-shop/juice-shop.git>

Scope

This threat model covers 8 components of juice-shop2: **Express.js Backend API, JWT Authentication and Session Management, Angular SPA Web Frontend, SQLite Embedded Database, CI/CD Pipeline, Real-time WebSocket Channel, Web3 / Wallet / NFT Surface, marsdb Embedded Store.**

All 8 modeled components received full STRIDE threat analysis.

Out of scope: third-party hosted dependencies, browser runtime, operating-system kernel, and the underlying network infrastructure.

Basis: a code-derived threat model at implementation level, built from source, configuration and git history. It describes the system as built, not as designed.

Identified Actors

The consolidated threat actors that drive this model - the same set named in the Management Summary. Each row aggregates the findings reachable from that actor's position; the **Shop User** appears as the *victim* of client-side attacks, not an attacker.

Actor grouping. Regular self-registered users are grouped with anonymous internet attackers because registration is open. Repository readers are grouped with anonymous internet attackers because the source repository is public. Each finding retains its login and privilege requirements.

Actor	Role	Reach	Findings	Components
Shop User	victim	legitimate customer; target of client-side attacks	3	backend, frontend, web3-nft
Internet Attacker	attacker	can self-register a regular account	65	auth, backend, ci-cd-pipeline, frontend, marsdb-store, realtime-channel, web3-nft

Trust Boundaries

Canonical boundary crossings. **Assumption & verdict** names the condition that makes each crossing safe and whether this report still supports it, judged one condition at a time - which conditions apply follows from the direction of the crossing.

ID	Boundary / crossing	Exposure	Kind	Assumption & verdict	Linked findings
tb-1	external → backend Control: security.isAuthenticated() expressJwt · Also covers: realtime-channel, web3-nft	 External	network	Every state-changing route is registered behind security.isAuthenticated() before reaching backend business logic. <i>confirmed</i> Validation: broken · 3 related · §6.6 Authentication: broken · 5 related · §6.2 Authorization: broken · 4 related · §6.4	<i>Validation</i> ● F-012 ● F-023 ● F-043 ● F-044 <i>Authentication</i> ● F-007 ● F-017 ● F-027 ● F-053 <i>Authorization</i> ● F-065
tb-2	external → auth Control: jwt.sign RS256 and expressJwt verification	 External	network · identity	A JWT is accepted only if it is signed with the server-held RSA private key. <i>confirmed</i> Validation: in doubt · 1 related · §6.6 Authentication: broken · 2 related · §6.2 Authorization: <i>not examined</i> · §6.4	<i>Authentication</i> ● F-006 ● F-007
tb-3	external → ci-cd-pipeline Control: GITHUB_TOKEN scopes and branch protection	 Unverified	build pipeline · operator	Only authorized contributors can trigger production pipeline runs and access build secrets. <i>inferred</i> Validation: in doubt · 1 related · §6.6 Authentication: <i>not examined</i> · §6.2 Authorization: in doubt · 1 related · §6.4	-
tb-4	auth → sqlite-db Control: Sequelize parameterized query binding · Also covers: backend	 Internal	in-process - enforcement interface, no trust transition	Every credential lookup reaches SQLite through Sequelize parameter binding. <i>confirmed</i> Query construction:	<i>Query construction</i> ● F-008

ID	Boundary / crossing	Exposure	Kind	Assumption & verdict	Linked findings
				broken · 2 related · §6.5	
tb-5	backend → external Control: none identified	↗ Egress	network · operator	Nothing attacker-controlled reaches the Ollama LLM provider unfiltered. <i>confirmed</i> Egress content: broken Egress destination: <i>not examined</i> · §6.10 Response trust: broken	<i>Egress content</i> ● F-034 ● F-038 <i>Response trust</i> ● F-052

Exposure, most exposed first: ▲ Review (endpoints unresolved) · 🌐 External (reached from outside the model) · 🔒 Unverified (crossing not confirmed) · 🔑 Internal (both ends modelled) · ↗ Egress (reaches outside the model).

Kind - the crossing mechanism (build pipeline, in-process, network), then after what changes across it: data-origin, identity, operator, privilege, tenant. Shown alone, nothing changes.

Conditions - **inbound**: validation, authentication, authorization · **outbound**: egress content, egress destination, response trust (replies are untrusted input) · **internal**: query construction (data never becomes program text), authentication, authorization.

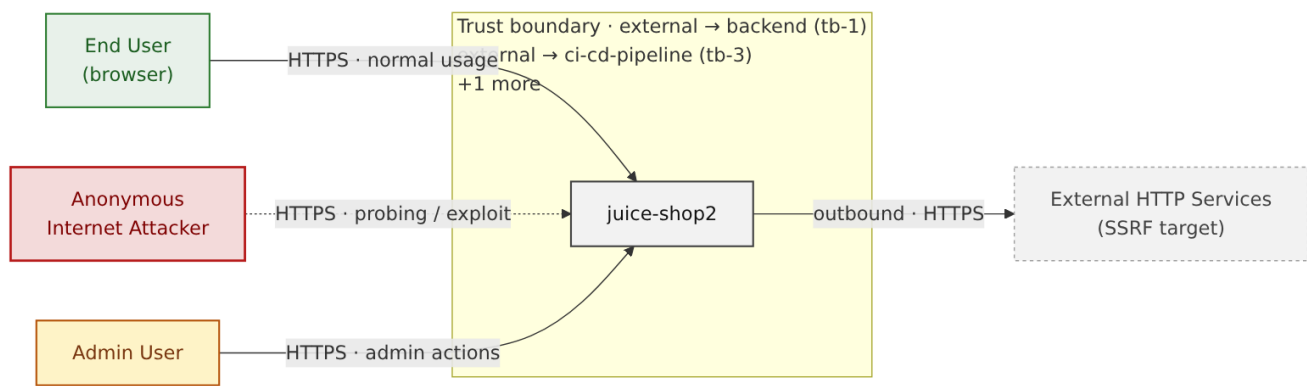
Verdict per condition - **broken**: a linked finding proves the gap · in doubt: related findings, none linked here · not examined: nothing bears on it. N related counts further findings on the same condition. Linked findings sit under the condition they break, or under Unattributed. A link raises effective severity only at a confirmed internet ingress; raw risk never changes.

Identical on every row, so stated once here instead of in a column: source detected (derived from inspected repository evidence); status resolved. Any row that deviates is shown in the table.

2. Architecture Diagrams

2.1 System Context

Who interacts with juice-shop2 from the outside, and through which channels. Solid arrows show normal usage; dashed red arrows mark unauthenticated probing or exploit paths (C4 Level 1).

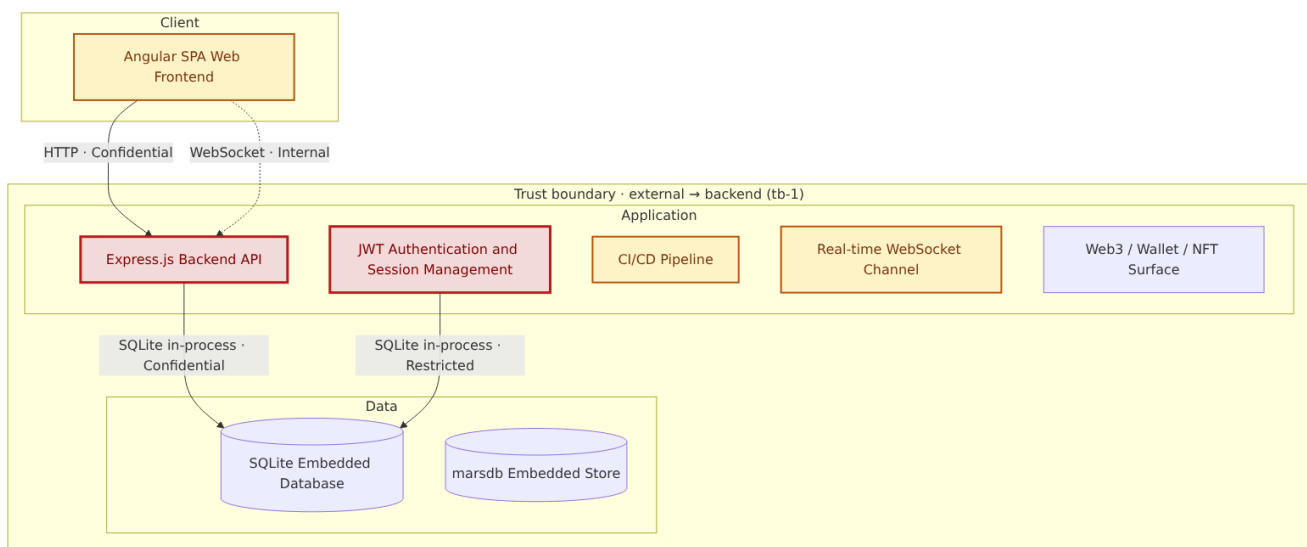


Trust boundaries not named above: external → auth (tb-2), auth → sqlite-db (tb-4), backend → external (tb-5) - every boundary is listed in [§1 Trust Boundaries](#).

Key takeaway: Every actor in the context interacts with juice-shop2 through its external interface, so authentication and input validation at that edge govern the entire attack surface.

2.2 Container Architecture

How the system decomposes into deployable units. Each box is a separate runtime process or service container; arrows show synchronous request paths between them. Components with ≥ 3 Critical findings carry a red border, ≥ 2 High amber (C4 Level 2).

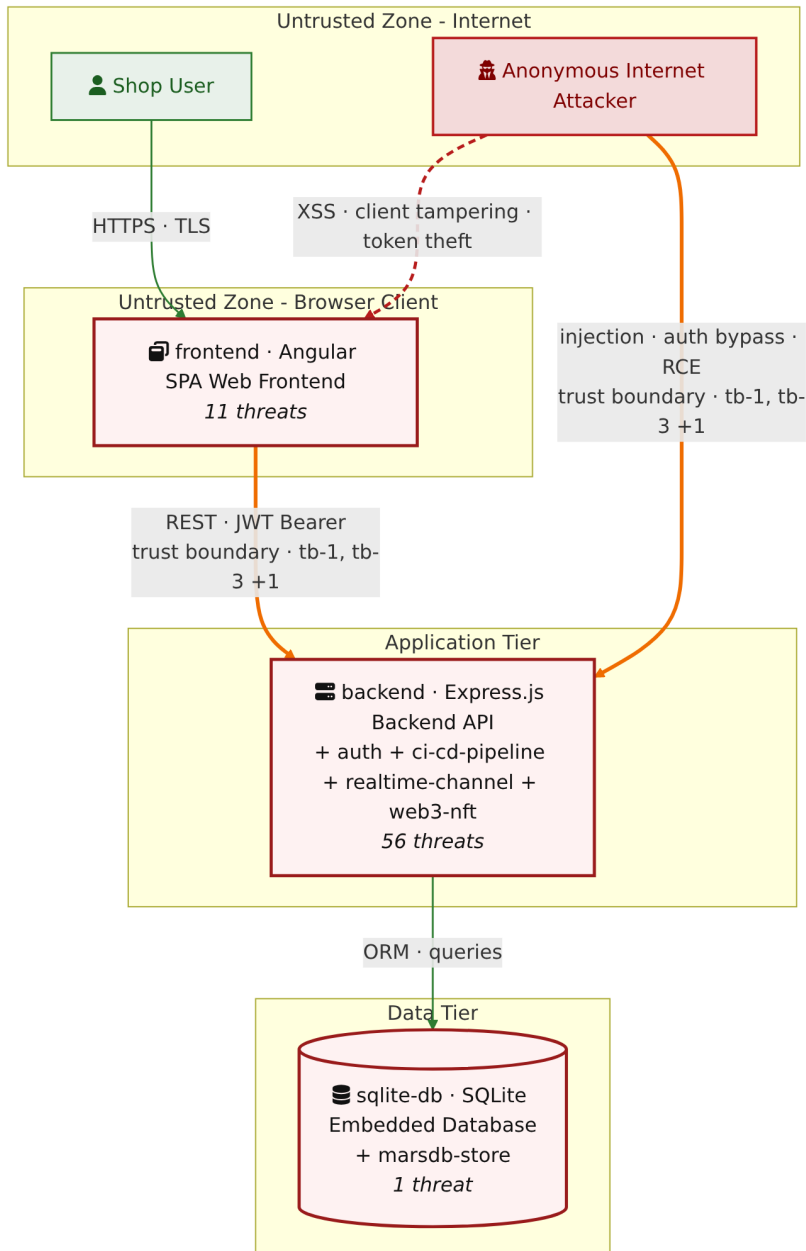


Trust boundaries not drawn above: auth → sqlite-db (tb-4), backend → external (tb-5) - every boundary is listed in [§1 Trust Boundaries](#).

Key takeaway: The system decomposes into 1 client, 5 application and 2 data unit(s); JWT Authentication and Session Management carries the most Critical findings (3) and bounds the worst-case blast radius.

2.3 Components

Who reaches each component, and through which trust zone. Browser code runs on the user's device, so the client column is part of the untrusted zone, not a zone of its own - trust changes only where traffic enters the Application tier. Solid green arrows show legitimate data flow, dashed red arrows mark intrusion vectors. The component table directly below holds source paths and linked threats per c-NN ; per-finding evidence is in [§8 Findings Register](#).



Trust boundaries crossed by the ==> edges above: external → backend (tb-1), external → ci-cd-pipeline (tb-3), external → auth (tb-2) - every boundary is listed in [§1 Trust Boundaries](#).

Key takeaway: Express.js Backend API concentrates the most findings (26 of 68 across all components); the table below maps each component to its source paths and linked threats.

ID	Name	Type	Key Paths	Linked Threats
C-01	Express.js Backend API	application	<pre>server.ts app.ts routes/*.ts lib/**/*.ts models/**/*.ts</pre>	● F-009 — Insecure Direct Object Reference (<code>address.ts:11</code>) ● F-011 — Server-side eval of stored username (<code>userProfile.ts:61</code>) ● F-012 — Mass assignment of role on registration (<code>server.ts:484</code>) ● F-019 — NoSQL \$where JavaScript injection (<code>trackOrder.ts:18</code>) ● F-038 — Coupon discount bounded only by prompt text (<code>chat.ts:184</code>) ● F-039 — Deluxe upgrade skips payment check (<code>deluxe.ts:43</code>) ● F-018 — Hard-coded seeded account credentials (<code>datacreator.ts:200</code>) ● F-021 — Input compiled as template source (<code>userProfile.ts:87</code>) ● F-026 — Denylisted path traversal in erasure layout (<code>dataErasure.ts:104</code>) ● F-027 — HTTP access logs browsable without authentication (<code>server.ts:281</code>) ● F-028 — Unvalidated URL fetch in profile image (<code>profileImageUrlUpload.ts:24</code>) ● F-032 — Rate limiter keyed on client-supplied header (<code>server.ts:346</code>) ● F-033 — No rate limit or lockout on login (<code>server.ts:596</code>) ● F-034 — Unbounded LLM consumption on <code>/rest/chat</code> (<code>server.ts:638</code>) ● F-037 — Client-controlled wallet credit amount (<code>wallet.ts:27</code>) ● F-040 — Sensitive Routes Registered Without Authentication Middleware (<code>server.ts:310</code>) ● F-053 — Admin configuration endpoints exposed unauthenticated (<code>server.ts:607</code>) ● F-065 — Basket item update without ownership check (<code>server.ts:426</code>) ● F-045 — Cookie-only authentication on POST <code>/profile</code> (<code>updateUserProfile.ts:17</code>) ● F-049 — Unescaped anchor persisted into product description (<code>datacreator.ts:406</code>) ● F-052 — Chat tool calls streamed to every caller (<code>chat.ts:228</code>) ● F-054 — Development error handler

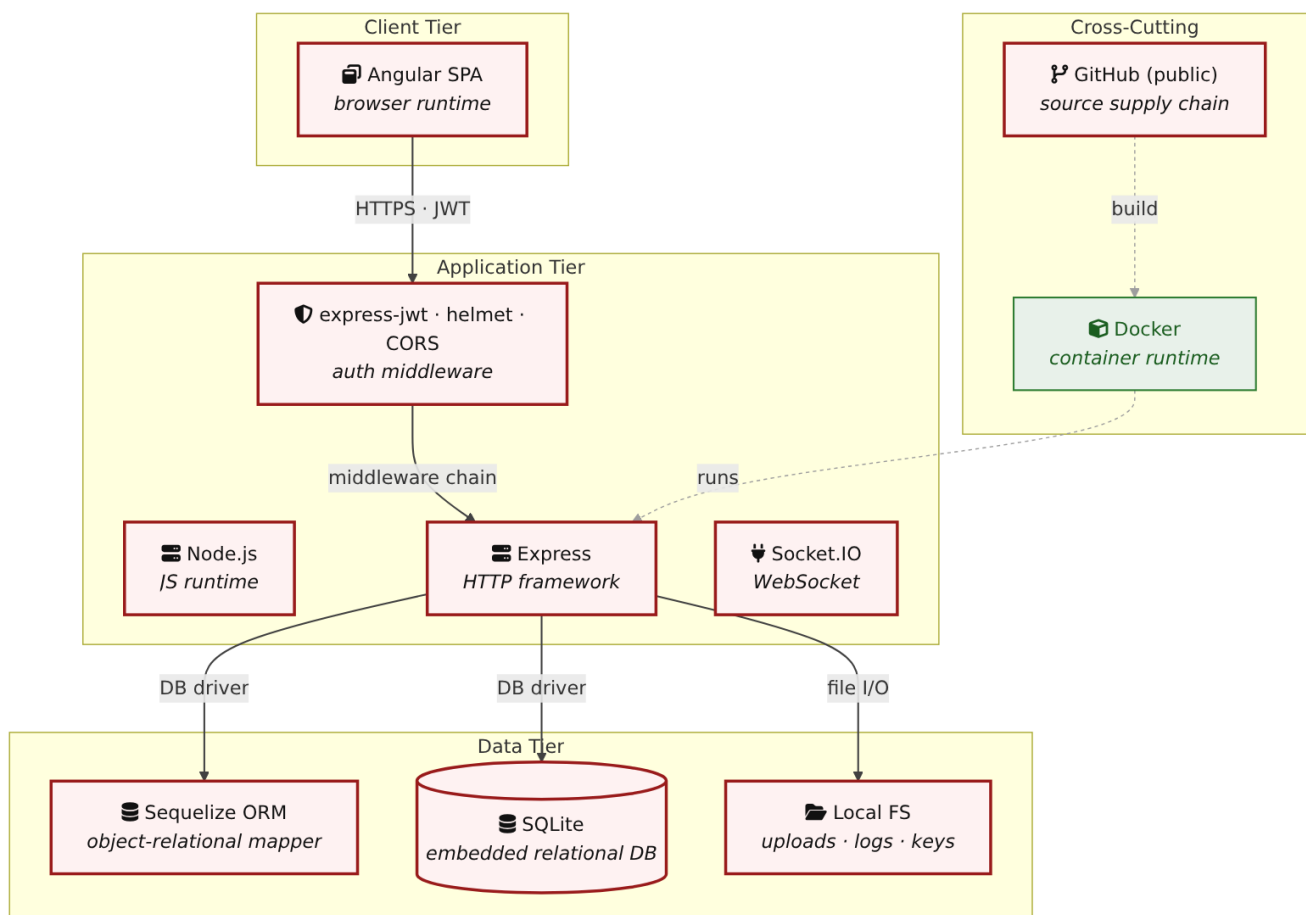
ID	Name	Type	Key Paths	Linked Threats
				returns stack traces (<code>server.ts:682</code>) ● F-058 — Ineffective author email masking in feedback (<code>datacreator.ts:578</code>) ● F-059 — Plaintext security answer written to log (<code>datacreator.ts:692</code>) ● F-066 — Security answers stored under embedded HMAC key (<code>datacreator.ts:691</code>) ● F-067 — Entitlement token derived from account email (<code>datacreator.ts:198</code>)
C-02	JWT Authentication and Session Management	application	<code>routes/login.ts</code> <code>routes/2fa.ts</code> <code>routes/changePassword.ts</code> <code>routes/currentUser.ts</code> <code>routes/authenticatedUsers.ts</code>	● F-006 — Hard-coded JWT signing key (<code>insecurity.ts:21</code>) ● F-007 — Insecure JWT Verification (<code>insecurity.ts:52</code>) ● F-008 — SQL injection in login query (<code>login.ts:34</code>) ● F-013 — Weak password recovery mechanism (<code>resetPassword.ts:41</code>) ● F-020 — Non-cryptographic RNG for a secret/token (<code>insecurity.ts:53</code>) ● F-024 — Passwords passed in URL query string (<code>changePassword.ts:14</code>) ● F-025 — Unsalted MD5 password hashing (<code>insecurity.ts:41</code>) ● F-036 — Password change without current password (<code>changePassword.ts:39</code>) ● F-004 — Missing server-side security audit logging (<code>login.ts:50</code>) ● F-042 — Missing session token revocation (<code>insecurity.ts:54</code>) ● F-051 — Credential validity oracle before 2FA (<code>login.ts:38</code>) ● F-061 — Unbounded in-memory session map (<code>insecurity.ts:74</code>)
C-03	Angular SPA Web Frontend	client	<code>frontend/src/**/*.ts</code> <code>frontend/src/**/*.html</code> <code>frontend/src/**/*.scss</code>	● F-001 — Cross-Site Scripting (<code>search-result.component.ts:143</code>) ● F-015 — Unvalidated OAuth token (<code>oauth.component.ts:28</code>) ● F-056 — Hard-coded test credential (<code>login.component.ts:62</code>) ● F-002 — JWT in localStorage (<code>request.interceptor.ts:13</code>) ● F-003 — Client-side authorization decision (<code>app.guard.ts:54</code>) ● F-016 — Predictable derived credential

ID	Name	Type	Key Paths	Linked Threats
				(<code>oauth.component.ts:30</code>) ● F-055 — Client-gated tool-call disclosure (<code>chat-conversation.component.ts:68</code>) ● F-073 — Token flow accepts a stolen bearer token without (<code>login.component.ts:148</code>) ● F-048 — Missing Content-Security-Policy ● F-057 — Session cookie without <code>HttpOnly</code> (<code>login.component.ts:104</code>) ● F-063 — Unbounded chat history resend (<code>chat-conversation.component.ts:131</code>)
C-04	SQLite Embedded Database	data	<code>data/</code> <code>datacreator.ts</code> <code>data/</code> <code>datacache.ts</code>	-
C-05	CI/CD Pipeline	application	<code>.github/</code> <code>workflows/**</code> <code>.gitlab-ci.yml</code> <code>Dockerfile</code> <code>**/Dockerfile</code> <code>docker-</code> <code>compose*.yml</code>	● F-010 — Dependency lockfile disabled by config (<code>.npmrc:1</code>) ● F-014 — Long-lived registry publish credential (<code>ci.yml:327</code>) ● F-022 — Unpinned third-party action (<code>image_actions.yml:33</code>) ● F-029 — Secrets broadcast to job environment (<code>ci.yml:253</code>) ● F-041 — Missing workflow permissions block (<code>ci.yml:190</code>) ● F-046 — Remote script piped to shell (<code>ci.yml:358</code>) ● F-047 — Unpinned container base image — <code>Dockerfile:1</code> (<code>Dockerfile:1</code>) ● F-050 — Mutable image tag without build attestation (<code>ci.yml:345</code>) ● F-062 — Unbounded CI matrix fan-out (<code>ci.yml:58</code>)
C-06	Real-time WebSocket Channel	application	<code>lib/</code> <code>challengeUtils.ts</code> <code>lib/startup/</code> <code>registerWebsocketEvents.ts</code>	● F-017 — Unauthenticated WebSocket Channel (<code>registerWebsocketEvents.ts:23</code>) ● F-023 — Client-trusted security decision (<code>registerWebsocketEvents.ts:50</code>) ● F-030 — Unauthenticated data exposure (<code>registerWebsocketEvents.ts:30</code>) ● F-035 — Inefficient regex complexity (<code>registerWebsocketEvents.ts:46</code>)

ID	Name	Type	Key Paths	Linked Threats
C-07	Web3 / Wallet / NFT Surface	application	routes/ checkKeys.ts routes/ nftMint.ts routes/ redirect.ts routes/ web3Wallet.ts	● F-031 — Hard-coded wallet mnemonic (<code>checkKeys.ts:10</code>) ● F-043 — Wallet identity accepted without proof of possession (<code>web3Wallet.ts:16</code>) ● F-044 — Open redirect (<code>redirect.ts:19</code>) ● F-060 — Raw provider error returned to caller (<code>web3Wallet.ts:36</code>) ● F-064 — Listener guard race opens unbounded providers (<code>nftMint.ts:16</code>)
C-08	<code>marsdb</code> Embedded Store	data	<code>data/mongodb.ts</code>	● F-005 — Unmediated data-store access control (<code>mongodb.ts:10</code>)

2.4 Technology Architecture

The technology stack the system is built on. Each box names the framework or runtime that fills that role; per-component findings live in the [§2.3](#) component table above, and the full per-finding catalogue is in [§8 Findings Register](#).



Key takeaway: The stack spans 2 data-tier store(s) behind the application tier; injection and data-at-rest exposure track the data tier, detailed per finding in [§8 Findings Register](#).

Legend: `-->` synchronous request/response (REST, HTTPS, gRPC) · `-.->` asynchronous / event-driven (WebSocket, queue, pub-sub) · `==>` crosses an untrusted trust boundary (security-critical) · **red border** ≥ 3 Critical threats on the component · **amber border** ≥ 2 High threats

3. Attack Walkthroughs

This section walks through how the **8 highest-priority of 15 Critical findings** are exploited - selected by severity, chain relevance, and threat-category diversity, with Access Control and LLM Abuse represented when present. Each walkthrough has attack steps, a focused sequence diagram, and the primary mitigation. How weaknesses combine toward the worst-case goal is in the [Critical Attack Tree](#); every other Critical, plus full per-finding context (severity rationale, assets, detection signals), is in the [§8 Findings Register](#).

3.1 Insecure JWT Verification in JWT Authentication and Session Management

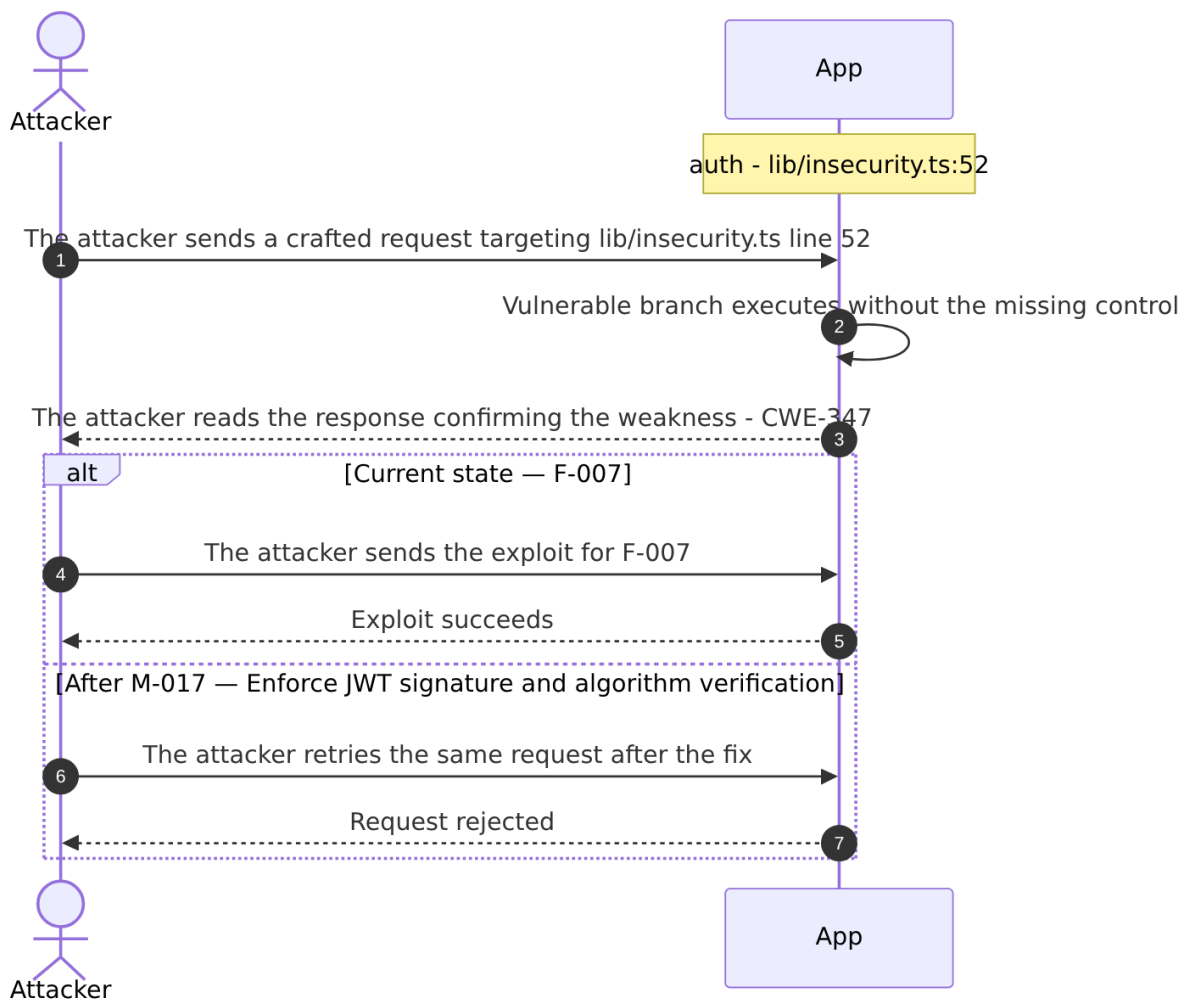
Source: ● [F-007](#) — `lib/insecurity.ts:52`

Severity **Critical** ([CWE-347](#)). STRIDE: Spoofing. See [§8 F-007](#) for the full register row.

Attack Steps

1. I fetch the RSA public key that `lib/insecurity.ts:20` loads from `encryptionkeys/jwt.pub`, which is distributed with the application.
2. I build a JWT whose header is alg `HS256` and whose data claim names the administrator account.
3. I compute the HMAC signature using the public key PEM text as the shared secret and send the token to a protected route.
4. `express-jwt` verifies it with the same PEM as an HMAC secret and grants me the administrator session.

Sequence Diagram



Key takeaway: Until ● [M-017](#) (Enforce JWT signature and algorithm verification) lands, ● [F-007](#) — Insecure JWT Verification is exploitable at `lib/insecurity.ts:52` (Critical-severity, [CWE-347](#)).

Defense in Depth

- Primary mitigation: ● [M-017](#) (Enforce JWT signature and algorithm verification)

3.2 SQL injection in login query

Source: ● [F-008](#) — `routes/login.ts:34`

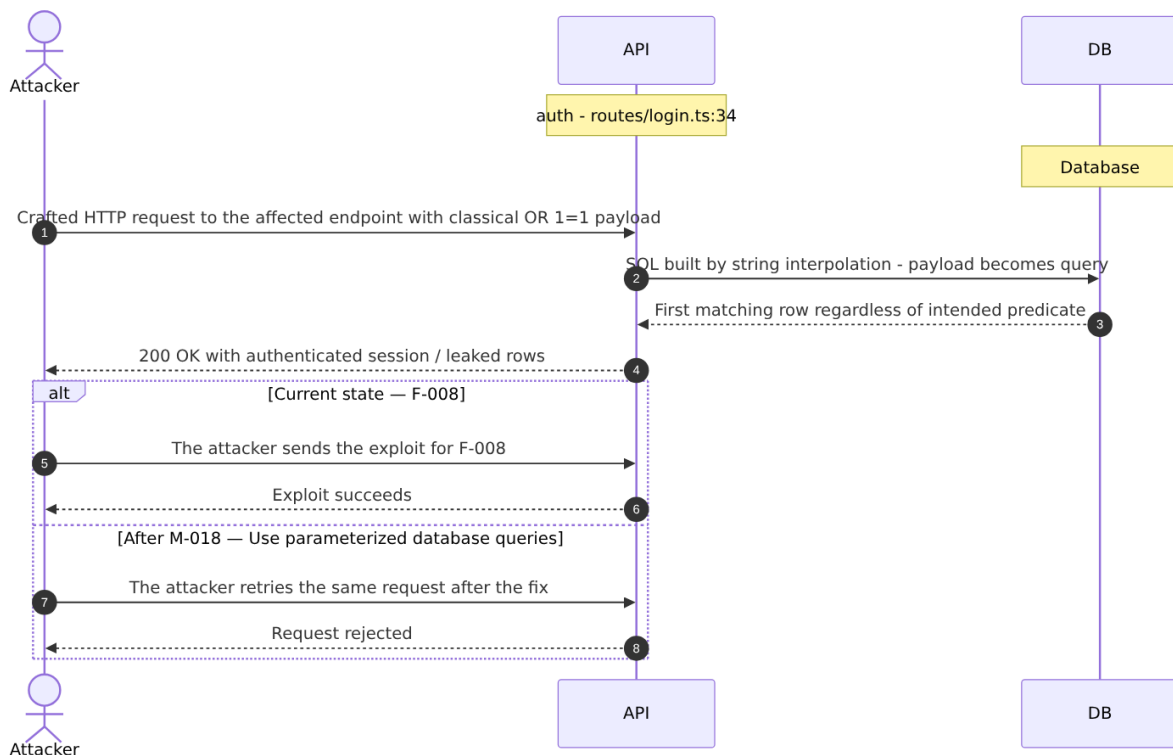
Severity **Critical** ([CWE-89](#)). STRIDE: Tampering. See §8 [F-008](#) for the full register row.

Attack Steps

1. I POST to `/rest/user/login` with the email field set to `' OR 1=1--` and any password.
2. The handler splices my value into the raw SQL at `routes/login.ts:34`, so the WHERE clause matches the first Users row.
3. The response at `routes/login.ts:26` hands me a signed session token for that account, which is the administrator.

4. I extend the same injection with UNION SELECT to read password hashes and security answers out of the database.

Sequence Diagram



Key takeaway: Until ● [M-018](#) (Use parameterized database queries) lands, ● [F-008](#) — SQL injection in login query is exploitable at `routes/login.ts:34` (Critical-severity, [CWE-89](#)).

Defense in Depth

- Primary mitigation: ● [M-018](#) (Use parameterized database queries)

3.3 Mass assignment of role on registration in Express js Backend API

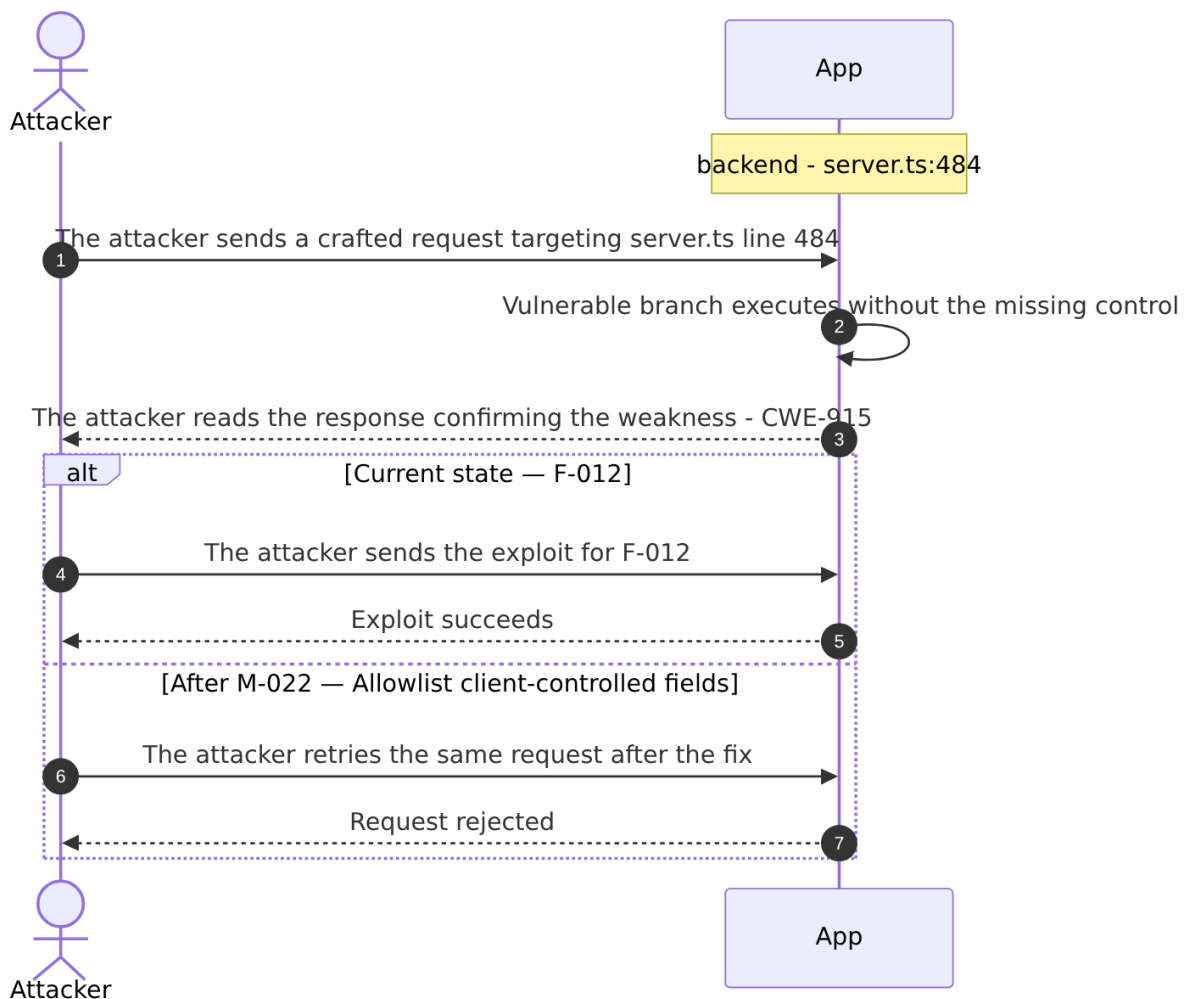
Source: ● [F-012](#) — `server.ts:484`

Severity **Critical** ([CWE-915](#)). STRIDE: Elevation of Privilege. See [§8 F-012](#) for the full register row.

Attack Steps

1. I post `{"email":"x@x.tld", "password":"p", "passwordRepeat":"p", "role":"admin"}` to `/api/users` with no session.
2. The finale User resource binds the role attribute from my body because the exclude list at `server.ts:484` omits it.
3. I log in with the new account and receive a token whose `data.role` claim is admin, unlocking every role-gated route.

Sequence Diagram



Key takeaway: Until ● [M-022](#) (Allowlist client-controlled fields) lands, ● [F-012](#) — Mass assignment of role on registration is exploitable at `server.ts:484` (Critical-severity, [CWE-915](#)).

Defense in Depth

- Primary mitigation: ● [M-022](#) (Allowlist client-controlled fields)

3.4 Server-side eval of stored username in User Profile

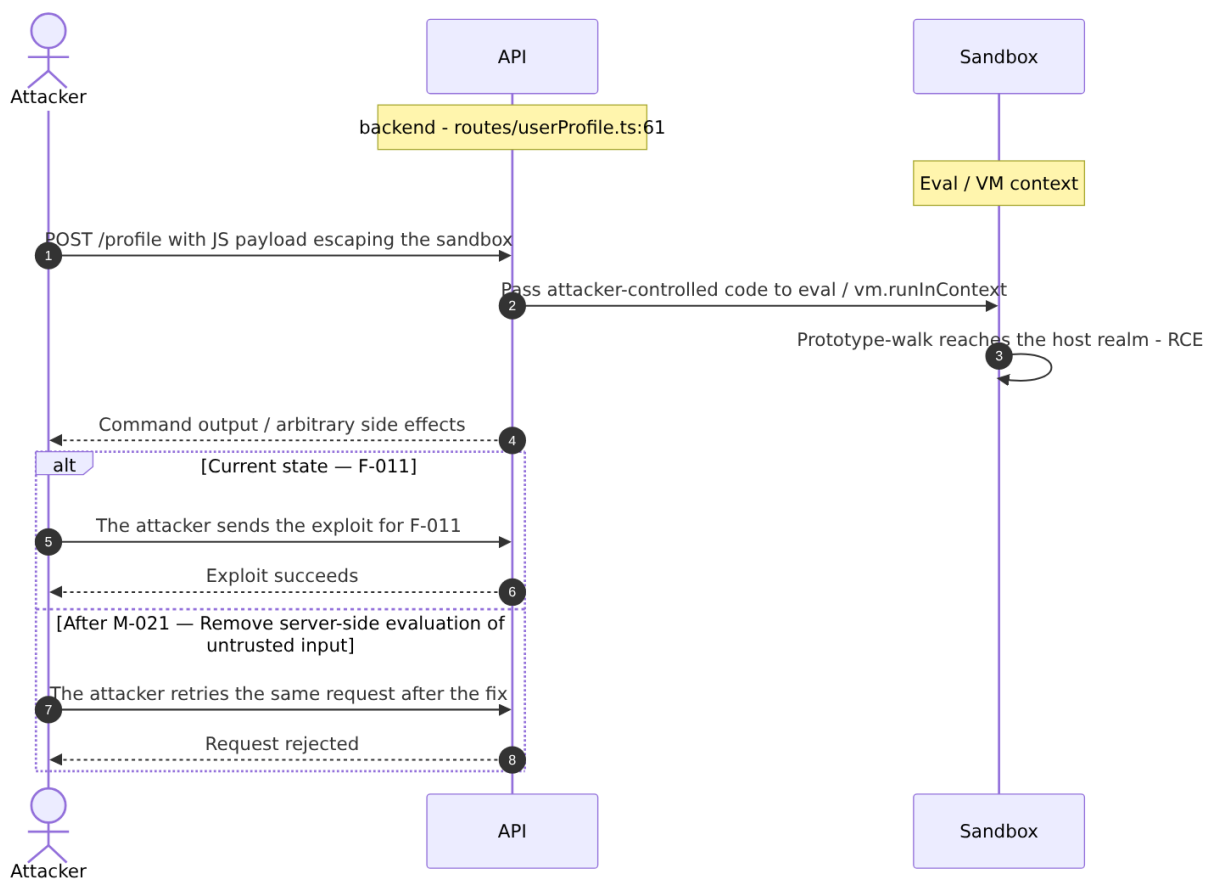
Source: ● [F-011](#) — `routes/userProfile.ts:61`

Severity **Critical** ([CWE-94](#)). STRIDE: Elevation of Privilege. See [§8 F-011](#) for the full register row.

Attack Steps

1. I register an account and post a username of `#{ process.mainModule.require('child_process').execSync('id') }` to `/profile`.
2. The update handler stores my username unchanged in the Users table.
3. I request GET `/profile`, and the profile renderer extracts my payload and passes it to `eval`, executing my command in the server process.

Sequence Diagram



Key takeaway: Until ● [M-021](#) (Remove server-side evaluation of untrusted input) lands, ● [F-011](#) — Server-side eval of stored username (`routes/userProfile.ts:61`) is exploitable at `routes/userProfile.ts:61` (Critical-severity, [CWE-94](#)).

Defense in Depth

- Primary mitigation: ● [M-021](#) (Remove server-side evaluation of untrusted input)

3.5 Cross-Site Scripting in Search Result

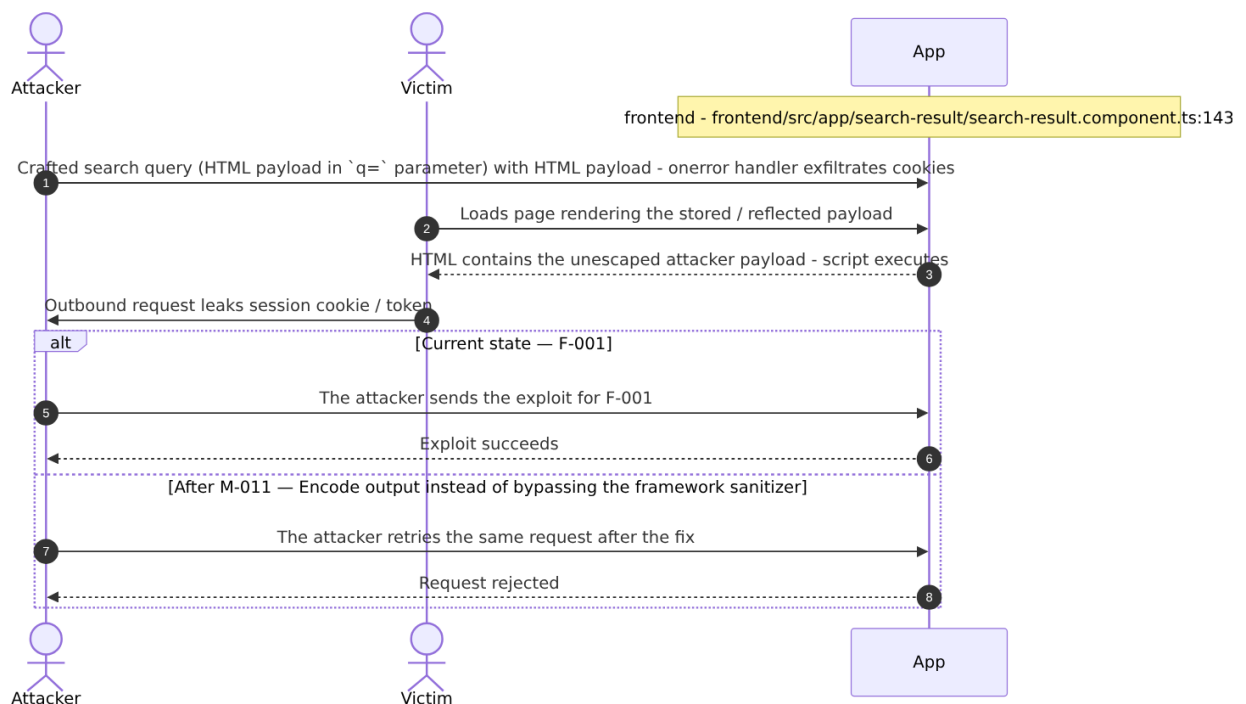
Source: ● [F-001](#) — `frontend/src/app/search-result/search-result.component.ts:143`

Severity **Critical** ([CWE-79](#)). STRIDE: Tampering. See [§8 F-001](#) for the full register row.

Attack Steps

1. I craft a URL whose `q` query parameter contains an HTML event-handler payload that posts `localStorage` to my server.
2. I send the link to a signed-in customer, who opens it in the shop they already trust.
3. `filterTable` passes my `q` value through `bypassSecurityTrustHtml`, so Angular renders it as live markup instead of text.
4. My payload reads the JWT from `localStorage` and posts it to my server, letting me replay the victim's session against the REST API.

Sequence Diagram



Key takeaway: Until ● [M-011](#) (Encode output instead of bypassing the framework sanitizer) lands, ● [F-001](#) — Cross-Site Scripting is exploitable at

`frontend/src/app/search-result/search-result.component.ts:143` (Critical-severity, [CWE-79](#)).

Defense in Depth

- Primary mitigation: ● [M-011](#) (Encode output instead of bypassing the framework sanitizer)

3.6 Hard-coded JWT signing key in JWT Authentication and Session Management

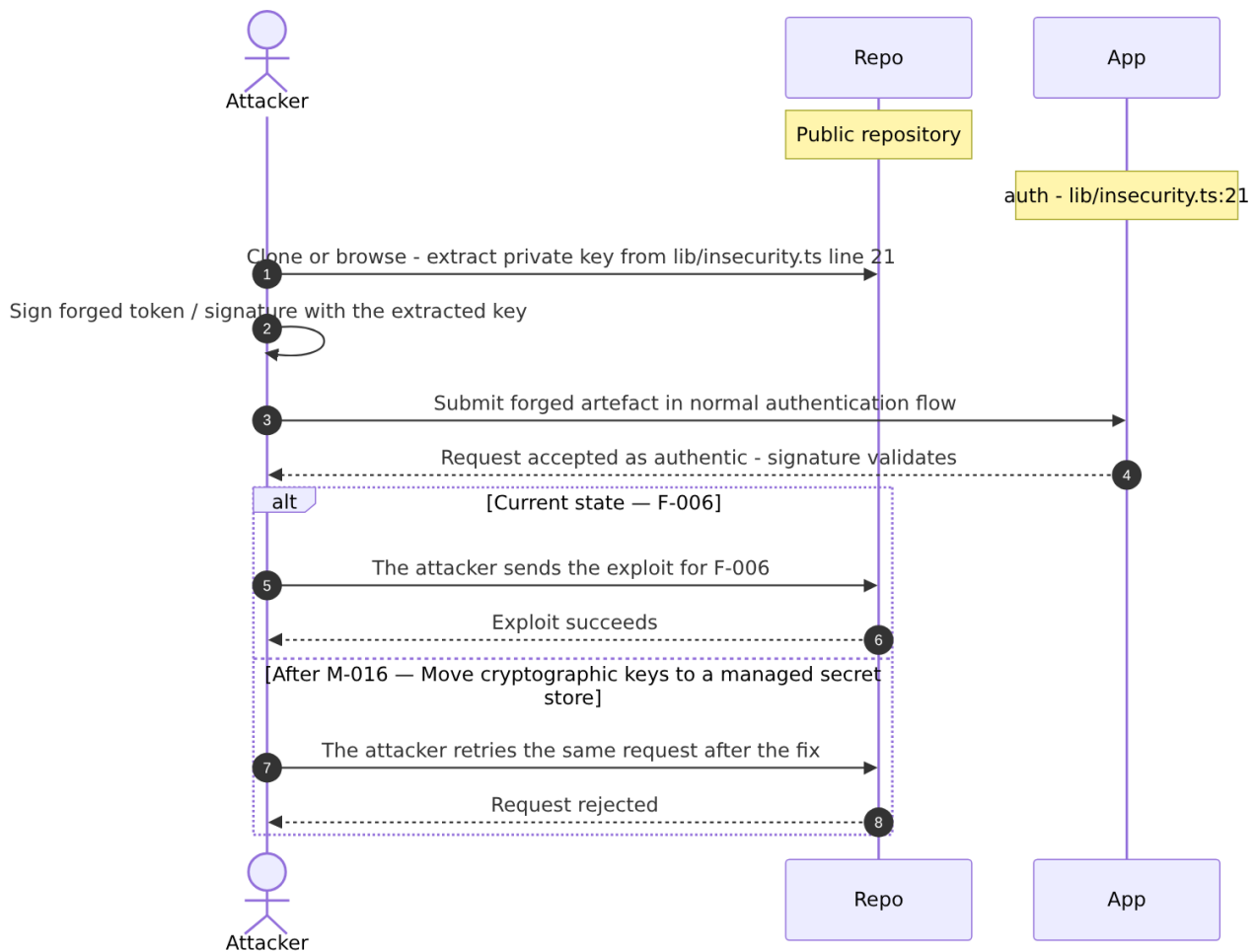
Source: ● [F-006](#) — `lib/insecurity.ts:21`

Severity **Critical** ([CWE-321](#)). STRIDE: Spoofing. See [§8 F-006](#) for the full register row.

Attack Steps

1. I copy the RSA private key literal embedded at `lib/insecurity.ts:21` out of the published source tree.
2. I sign my own JWT payload with that key using `RS256`, setting `data.id` and `data.role` to the administrator account.
3. I send the forged token as the `Authorization` bearer header to any route registered behind `isAuthorized()`.
4. I operate with administrator authority for the full six-hour token lifetime because no revocation list exists.

Sequence Diagram



Key takeaway: Until ● [M-016](#) (Move cryptographic keys to a managed secret store) lands, ● [F-006](#) — Hard-coded JWT signing key (`lib/insecurity.ts:21`) is exploitable at `lib/insecurity.ts:21` (Critical-severity, [CWE-321](#)).

Defense in Depth

- Primary mitigation: ● [M-016](#) (Move cryptographic keys to a managed secret store)

3.7 Dependency lockfile disabled by config in Npmrc

Source: ● [F-010](#) — `.npmrc:1`

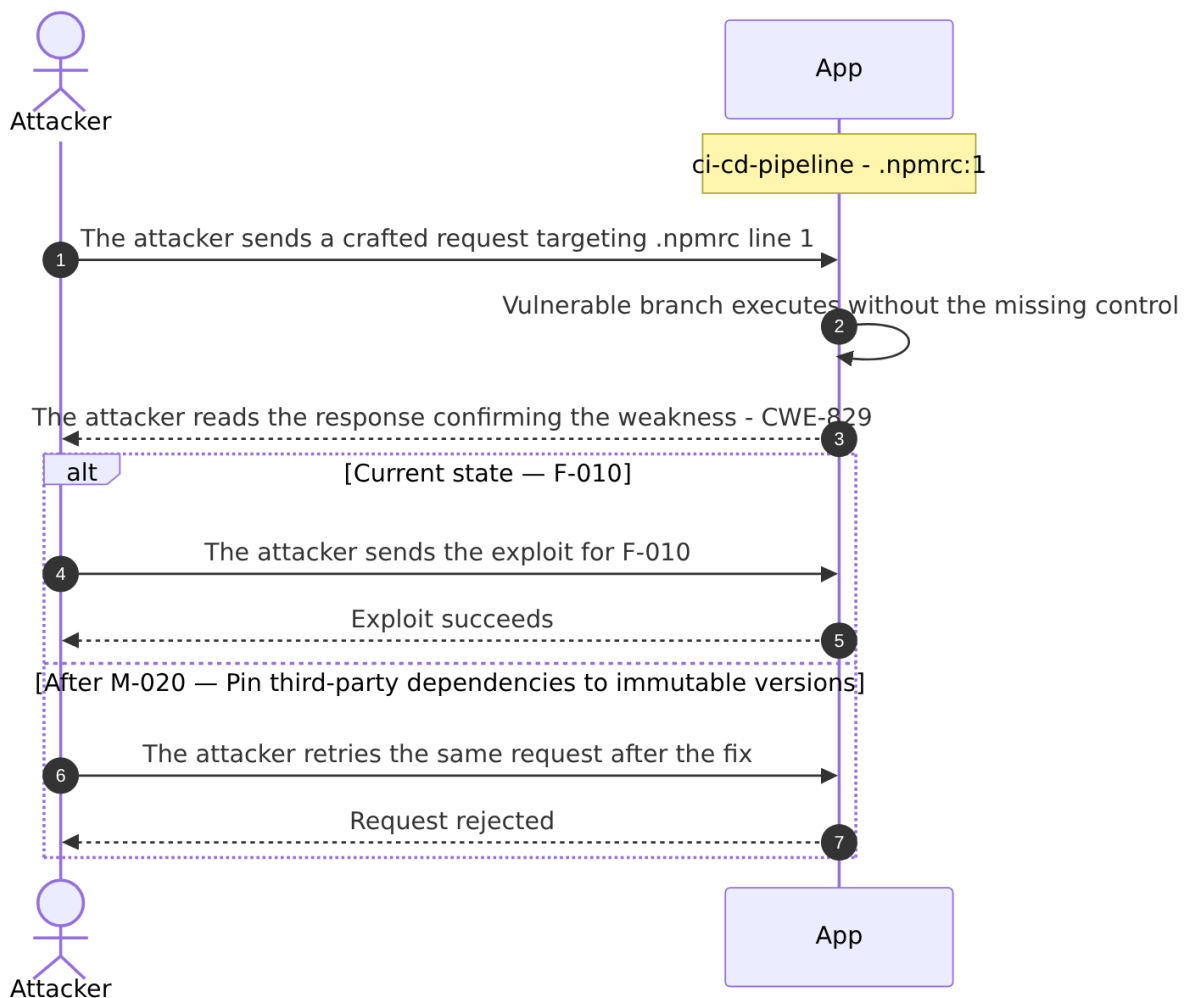
Severity **Critical** ([CWE-829](#)). STRIDE: Tampering. See §8 [F-010](#) for the full register row.

Attack Steps

1. I take over a maintainer account for a small transitive dependency of juice-shop and publish a patch version that satisfies the existing caret range.
2. The next CI or Docker build runs `npm install` with no lockfile, so `npm` resolves my freshly published version instead of the one the last build used.
3. My `postinstall` script runs on the runner and reads the job environment, capturing `DOCKERHUB_TOKEN` , `HEROKU_API_KEY` and `GITHUB_TOKEN` .

4. In the `Dockerfile` installer stage I also patch the compiled output, so the published `bkimminich/juice-shop` image ships my backdoor.

Sequence Diagram



Key takeaway: Until ● [M-020](#) (Pin third-party dependencies to immutable versions) lands, ● [F-010](#) — Dependency lockfile disabled by config — `.npmrc:1` is exploitable at `.npmrc:1` (Critical-severity, [CWE-829](#)).

Defense in Depth

- Primary mitigation: ● [M-020](#) (Pin third-party dependencies to immutable versions)

3.8 Coupon discount bounded only by prompt text in Chat

Source: ● [F-038](#) — `routes/chat.ts:184`

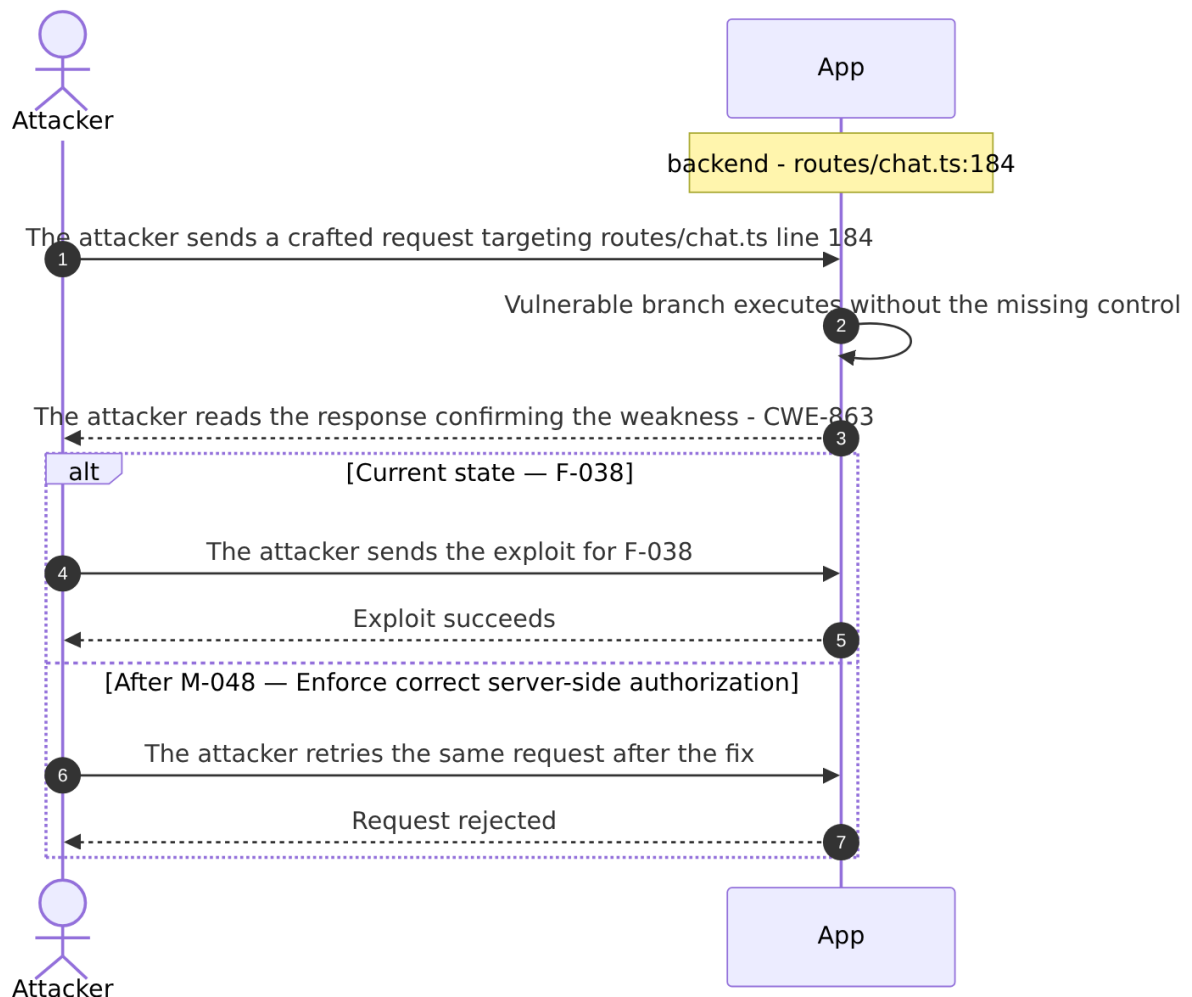
Severity **High** ([CWE-863](#)). STRIDE: Elevation of Privilege. See [§8 F-038](#) for the full register row.

Attack Steps

1. LLM06 - Excessive Agency: an attacker injects instructions into the chat messages array so the model calls `generateCoupon` with a discount far above the stated maximum.

2. The coupon policy, including the 10 percent ceiling and the damaged-order precondition, exists only as system-prompt text at `routes/chat.ts:98-105`.
3. The tool's execute body at `routes/chat.ts:184` passes the model-chosen discount straight to `security.generateCoupon` with no numeric bound, no order verification, and no caller check, and the resulting code is redeemable through `PUT /rest/basket/:id/coupon/:coupon` at.

Sequence Diagram



Key takeaway: Until 🟡 **M-048** (Enforce correct server-side authorization) lands, 🔴 **F-038** — Coupon discount bounded only by prompt text (`routes/chat.ts:184`) is exploitable at `routes/chat.ts:184` (High-severity, **CWE-863**).

Defense in Depth

- Primary mitigation: 🟡 **M-048** (Enforce correct server-side authorization)

4. Assets

Information assets and the classification level that drives the Confidentiality / Integrity / Availability targets used in §8 Findings Register risk scoring.

Asset	Classification	Description	Linked Threats
JWT RSA Signing Private Key	Restricted	2048-bit RSA private key used to sign all JWT session tokens. Hardcoded as a string literal in <code>lib/insecurity.ts</code> . Compromise of this key enables forging valid tokens for any user including administrators.	● F-006 — Hard-coded JWT signing key (<code>insecurity.ts:21</code>) ● F-007 — Insecure JWT Verification (<code>insecurity.ts:52</code>) ● F-018 — Hard-coded seeded account credentials (<code>datacreator.ts:200</code>) ● F-020 — Non-cryptographic RNG for a secret/token (<code>insecurity.ts:53</code>) ● F-026 — Denylisted path traversal in erasure layout (<code>dataErasure.ts:104</code>) ● F-029 — Secrets broadcast to job environment (<code>ci.yml:253</code>) ● F-030 — Unauthenticated data exposure (<code>registerWebsocketEvents.ts:30</code>) ● F-031 — Hard-coded wallet mnemonic (<code>checkKeys.ts:10</code>) ● F-042 — Missing session token revocation (<code>insecurity.ts:54</code>) ● F-052 — Chat tool calls streamed to every caller (<code>chat.ts:228</code>) ● F-053 — Admin configuration endpoints exposed unauthenticated (<code>server.ts:607</code>) ● F-055 — Client-gated tool-call disclosure (<code>chat-conversation.component.ts:68</code>) ● F-056 — Hard-coded test credential (<code>login.component.ts:62</code>) ● F-061 — Unbounded in-memory session map (<code>insecurity.ts:74</code>) ● F-066 — Security answers stored under embedded HMAC key (<code>datacreator.ts:691</code>)
Application Configuration	Restricted	Runtime application configuration including	-

Asset	Classification	Description	Linked Threats
		feature flags, customization settings, and integration parameters returned by <code>/rest/admin/application-configuration</code> . Exposed without authentication, enabling information disclosure about enabled features and integration endpoints.	
User Credentials (email, password hash)	Confidential	Email addresses and bcrypt password hashes for all registered users. Stored in the SQLite users table, transmitted during login, and used by the auth component for verification. Compromise enables account takeover across all users.	● F-001 — Cross-Site Scripting (<code>search-result.component.ts:143</code>) ● F-008 — SQL injection in login query (<code>login.ts:34</code>) ● F-014 — Long-lived registry publish credential (<code>ci.yml:327</code>) ● F-018 — Hard-coded seeded account credentials (<code>datacreator.ts:200</code>) ● F-025 — Unsalted MD5 password hashing (<code>insecurity.ts:41</code>) ● F-032 — Rate limiter keyed on client-supplied header (<code>server.ts:346</code>) ● F-033 — No rate limit or lockout on login (<code>server.ts:596</code>) ● F-067 — Entitlement token derived from account email (<code>datacreator.ts:198</code>)
User Personal Data (name, address, phone)	Confidential	Personal identifiable information including delivery addresses, phone numbers, and profile details stored per user. Subject to GDPR data erasure requests handled by <code>routes/dataErasure.ts</code> . Exposed via data export and admin panel.	● F-001 — Cross-Site Scripting (<code>search-result.component.ts:143</code>) ● F-008 — SQL injection in login query (<code>login.ts:34</code>) ● F-009 — Insecure Direct Object Reference (<code>address.ts:11</code>) ● F-011 — Server-side eval of stored username (<code>userProfile.ts:61</code>) ● F-012 — Mass assignment of role on registration (<code>server.ts:484</code>) ● F-021 — Input compiled as

Asset	Classification	Description	Linked Threats
			template source (userProfile.ts:87) ● F-026 — Denylisted path traversal in erasure layout (dataErasure.ts:104) ● F-028 — Unvalidated URL fetch in profile image (profileImageUrlUpload.ts:24) ● F-029 — Secrets broadcast to job environment (ci.yml:253) ● F-030 — Unauthenticated data exposure (registerWebsocketEvents.ts:30) ● F-040 — Sensitive Routes Registered Without Authentication Middleware (server.ts:310) ● F-045 — Cookie-only authentication on POST /profile (updateUserProfile.ts:17) ● F-052 — Chat tool calls streamed to every caller (chat.ts:228) ● F-053 — Admin configuration endpoints exposed unauthenticated (server.ts:607) ● F-055 — Client-gated tool-call disclosure (chat-conversation.component.ts:68) ● F-058 — Ineffective author email masking in feedback (datacreator.ts:578) ● F-065 — Basket item update without ownership check (server.ts:426)
Payment Card Data	Confidential	Saved payment card details including card number and expiry stored in the SQLite cards table. Processed during basket checkout. No evidence of PCI-compliant tokenization was found.	● F-001 — Cross-Site Scripting (search-result.component.ts:143) ● F-008 — SQL injection in login query (login.ts:34) ● F-009 — Insecure Direct Object Reference (address.ts:11) ● F-040 — Sensitive Routes

Asset	Classification	Description	Linked Threats
			Registered Without Authentication Middleware (server.ts:310) ● F-065 — Basket item update without ownership check (server.ts:426)
Active Session Tokens (JWT in localStorage)	Confidential	JWT bearer tokens stored in browser localStorage by the Angular frontend. Accessible to any JavaScript running on the page, making them vulnerable to XSS-based exfiltration. Used by the request interceptor to authenticate all API calls.	● F-001 — Cross-Site Scripting (search-result.component.ts:143) ● F-002 — JWT in localStorage (request.interceptor.ts:13) ● F-007 — Insecure JWT Verification (insecurity.ts:52) ● F-042 — Missing session token revocation (insecurity.ts:54) ● F-045 — Cookie-only authentication on POST /profile (updateUserProfile.ts:17) ● F-046 — Remote script piped to shell (ci.yml:358) ● F-073 — Token flow accepts a stolen bearer token without (login.component.ts:148)
Challenge Progress and CTF Scores	Internal	Per-user challenge completion state, scores, and code-fix verdicts stored in the SQLite challenges and related tables. Integrity of challenge state determines the validity of training outcomes and CTF competition results.	-
Product Catalog and Pricing	Internal	Product definitions, pricing, reviews, and quantity data stored in SQLite and served via the CRUD API. Modification without authorization (via missing authz on PUT/DELETE /api/Products/:id) constitutes business-logic tampering.	● F-001 — Cross-Site Scripting (search-result.component.ts:143) ● F-008 — SQL injection in login query (login.ts:34) ● F-012 — Mass assignment of role on registration (server.ts:484)

5. Attack Surface

Network-reachable entry points classified by authentication requirement. Each row links to the threat(s) referenced in its **Notes** column. The **Risk** column reflects the highest-severity linked finding. Entry points with no linked finding are still listed when they sit on a sensitive surface (authentication, registration, management) or look like a missing-auth/authz suspect - marked **Review** in Notes.

5.1 Unauthenticated Entry Points (55)

Method	Route	Risk	Notes
POST	/profile	Critical	- F-011 — Server-side eval of stored username (userProfile.ts:61) - F-028 — Unvalidated URL fetch in profile image (profileImageUrlUpload.ts:24) - F-045 — Cookie-only authentication on POST /profile (updateUserProfile.ts:17) Missing auth: POST /profile — profile update without authentication
POST	/rest/user/login	Critical	- F-008 — SQL injection in login query (login.ts:34) - F-033 — No rate limit or lockout on login (server.ts:596) - F-004 — Missing server-side security audit logging (login.ts:50) handler: server.ts:596
POST	/rest/web3/walletExploitAddress	Critical	- F-043 — Wallet identity accepted without proof of possession (web3Wallet.ts:16) - F-060 — Raw provider error returned to caller (web3Wallet.ts:36) Missing auth: POST /rest/web3/walletExploitAddress — exploit address submission without auth
GET	/profile	Critical	- F-011 — Server-side eval of stored username (userProfile.ts:61) - F-028 — Unvalidated URL fetch in profile image (profileImageUrlUpload.ts:24) - F-045 — Cookie-only authentication on POST /profile (updateUserProfile.ts:17) handler: server.ts:666
GET	/rest/track-order/:id	Critical	F-019 — NoSQL \$where JavaScript injection (trackOrder.ts:18) handler: server.ts:617
POST	/profile/image/file	High	F-028 — Unvalidated URL fetch in profile image (profileImageUrlUpload.ts:24)

Method	Route	Risk	Notes
			Missing auth: POST <code>/profile/image/file</code> — profile image file upload without auth
POST	<code>/profile/image/url</code>	● High	● F-028 — Unvalidated URL fetch in profile image (<code>profileImageUrlUpload.ts:24</code>) Missing auth: POST <code>/profile/image/url</code> — SSRF-candidate URL-based image upload without auth
GET	<code>/rest/admin/application-configuration</code>	● High	● F-053 — Admin configuration endpoints exposed unauthenticated (<code>server.ts:607</code>) Management: GET <code>/rest/admin/application-configuration</code> — no auth signal; exposes feature <code>config</code> without authentication
GET	<code>/rest/admin/application-version</code>	● High	● F-053 — Admin configuration endpoints exposed unauthenticated (<code>server.ts:607</code>) Management: GET <code>/rest/admin/application-version</code> — no auth signal; exposes version info
POST	<code>/rest/user/reset-password</code>	● High	● F-013 — Weak password recovery mechanism (<code>resetPassword.ts:41</code>) ● F-033 — No rate limit or lockout on login (<code>server.ts:596</code>) ● F-004 — Missing server-side security audit logging (<code>login.ts:50</code>) handler: <code>server.ts:598</code>
GET	<code>/redirect</code>	● High	● F-035 — Inefficient regex complexity (<code>registerWebsocketEvents.ts:46</code>) ● F-044 — Open redirect (<code>redirect.ts:19</code>) handler: <code>server.ts:659</code>
GET	<code>/rest/user/change-password</code>	● High	● F-024 — Passwords passed in URL query string (<code>changePassword.ts:14</code>) ● F-036 — Password change without current password (<code>changePassword.ts:39</code>) ● F-004 — Missing server-side security audit logging (<code>login.ts:50</code>) handler: <code>server.ts:597</code>
GET	<code>/this/page/is/hidden/behind/an/incredibly/high/paywall/that/could/only/be/unlocked/by/sending/1btc/to/us</code>	● High	● F-032 — Rate limiter keyed on client-supplied header (<code>server.ts:346</code>) ● F-044 — Open redirect (<code>redirect.ts:19</code>) ● F-055 — Client-gated tool-call disclosure (<code>chat-conversation.component.ts:68</code>) handler: <code>server.ts:652</code>
GET	<code>/rest/web3/nftMintListen</code>	● Medium	● F-064 — Listener guard race opens unbounded providers (<code>nftMint.ts:16</code>) handler: <code>server.ts:643</code>

Method	Route	Risk	Notes
POST	/	-	Missing auth: POST /rest/data-erasure - no auth signal on GDPR erasure endpoint Review: no auth guard detected
POST	/api/Feedbacks	-	handler: server.ts:402 Review: no auth guard detected
POST	/file-upload	-	Missing auth: POST /file-upload - unrestricted file upload without authentication Review: no auth guard detected
PUT	/rest/continue-code-findIt/apply/:continueCode	-	Missing auth: PUT /rest/continue-code-findIt/apply/:continueCode - challenge progress manipulation Review: no auth guard detected
PUT	/rest/continue-code-fixIt/apply/:continueCode	-	Missing auth: PUT /rest/continue-code-fixIt/apply/:continueCode - challenge progress manipulation Review: no auth guard detected
PUT	/rest/continue-code/apply/:continueCode	-	Missing auth: PUT /rest/continue-code/apply/:continueCode - challenge progress manipulation Review: no auth guard detected
POST	/rest/memories	-	Missing auth: POST /rest/memories - memory creation without authentication Review: no auth guard detected
PUT	/rest/order-history/:id/delivery-status	-	Missing auth: PUT /rest/order-history/:id/delivery-status - delivery status change without auth Review: no auth guard detected
POST	/rest/user/data-export	-	Missing auth: POST /rest/user/data-export - personal data export without authentication Review: no auth guard detected
POST	/rest/web3/submitKey	-	Missing auth: POST /rest/web3/submitKey - Web3 key submission without auth Review: no auth guard detected
POST	/rest/web3/walletNFTVerify	-	Missing auth: POST /rest/web3/walletNFTVerify - NFT wallet verification without auth Review: no auth guard detected
POST	/snippets/fixes	-	Missing auth: POST /snippets/fixes - code fix submission without auth Review: no auth guard detected
POST	/snippets/verdict	-	

Method	Route	Risk	Notes
			Missing auth: POST <code>/snippets/verdict</code> - code challenge verdict submission without auth Review: no auth guard detected

28 further entry point(s) in this category carry no linked finding and no elevated review signal, and are not listed individually (55 total). The complete route inventory is available in `route-inventory.json` and, when exported, `pentest-tasks.yaml`.

5.2 Authenticated Entry Points (52)

Method	Route	Risk	Notes
GET	<code>/rest/basket/:id</code>	● Critical	● F-038 — Coupon discount bounded only by prompt text (<code>chat.ts:184</code>) Missing authz: GET <code>/rest/basket/:id</code> — basket read without owner verification; IDOR
PUT	<code>/rest/basket/:id/coupon/:coupon</code>	● Critical	● F-038 — Coupon discount bounded only by prompt text (<code>chat.ts:184</code>) Missing authz: PUT <code>/rest/basket/:id/coupon/:coupon</code> — coupon application without ownership
POST	<code>/rest/chat</code>	● Critical	● F-038 — Coupon discount bounded only by prompt text (<code>chat.ts:184</code>) ● F-034 — Unbounded LLM consumption on <code>/rest/chat</code> (<code>server.ts:638</code>) ● F-052 — Chat tool calls streamed to every caller (<code>chat.ts:228</code>) LLM: POST <code>/rest/chat</code> — Ollama-backed chat endpoint with <code>middleware_present</code> ; prompt injection surface
GET	<code>/api/Users</code>	● Critical	● F-012 — Mass assignment of role on registration (<code>server.ts:484</code>) handler: <code>server.ts:363</code>
POST	<code>/api/Users</code>	● Critical	● F-012 — Mass assignment of role on registration (<code>server.ts:484</code>) handler: <code>server.ts:408</code>
GET	<code>/rest/deluxe-membership</code>	● Critical	● F-039 — Deluxe upgrade skips payment check (<code>deluxe.ts:43</code>) handler: <code>server.ts:628</code>
POST	<code>/rest/deluxe-membership</code>	● Critical	● F-039 — Deluxe upgrade skips payment check (<code>deluxe.ts:43</code>) handler: <code>server.ts:629</code>
PUT	<code>/api/BasketItems/:id</code>	● High	● F-065 — Basket item update without ownership check (<code>server.ts:426</code>)

Method	Route	Risk	Notes
			Missing authz: PUT /api/BasketItems/:id — IDOR; cross-user basket item modification
POST	/api/BasketItems	● High	● F-065 — Basket item update without ownership check (server.ts:426) handler: server.ts:427
GET	/rest/wallet/balance	● High	● F-037 — Client-controlled wallet credit amount (wallet.ts:27) handler: server.ts:626
PUT	/rest/wallet/balance	● High	● F-037 — Client-controlled wallet credit amount (wallet.ts:27) handler: server.ts:627
PUT	/api/Addressss/:id	-	Missing authz: PUT /api/Addressss/:id - address update without ownership check ▀ Review: no authz guard detected
DELETE	/api/Addressss/:id	-	Missing authz: DELETE /api/Addressss/:id - address deletion without ownership check ▀ Review: no authz guard detected
PUT	/api/Cards/:id	-	Missing authz: PUT /api/Cards/:id - payment card update without ownership check ▀ Review: no authz guard detected
DELETE	/api/Cards/:id	-	Missing authz: DELETE /api/Cards/:id - payment card deletion without ownership check ▀ Review: no authz guard detected
GET	/api/Cards/:id	-	Missing authz: GET /api/Cards/:id - payment card read without ownership check ▀ Review: no authz guard detected
PUT	/api/Feedbacks/:id	-	Missing authz: PUT /api/Feedbacks/:id - feedback update without ownership check ▀ Review: no authz guard detected
PUT	/api/Products/:id	-	Missing authz: PUT /api/Products/:id - authenticated but no authorization check; any user can modify products ▀ Review: no authz guard detected
DELETE	/api/Products/:id	-	Missing authz: DELETE /api/Products/:id - authenticated but no authorization check ▀ Review: no authz guard detected
DELETE	/api/Quantitys/:id	-	Missing authz: DELETE /api/Quantitys/:id - quantity deletion without ownership check ▀ Review: no authz guard detected

Method	Route	Risk	Notes
GET	/api/Recycles/:id	-	Missing authz: GET /api/Recycles/:id - IDOR on recycle record retrieval ▀ Review: no authz guard detected
PUT	/api/Recycles/:id	-	Missing authz: PUT /api/Recycles/:id - IDOR on recycle record update ▀ Review: no authz guard detected
DELETE	/api/Recycles/:id	-	Missing authz: DELETE /api/Recycles/:id - IDOR on recycle record deletion ▀ Review: no authz guard detected
GET	/metrics	-	Management: GET /metrics - middleware_present but no authz; Prometheus metrics exposed
POST	/rest/2fa/disable	-	Auth: POST /rest/2fa/disable - 2FA removal ▀ Review: auth/token endpoint
POST	/rest/2fa/setup	-	Auth: POST /rest/2fa/setup - 2FA enrollment; TOTP secret generation ▀ Review: auth/token endpoint
GET	/rest/2fa/status	-	handler: server.ts:463 ▀ Review: auth/token endpoint
POST	/rest/2fa/verify	-	Auth: POST /rest/2fa/verify - TOTP second-factor verification ▀ Review: auth/token endpoint
POST	/rest/basket/:id/checkout	-	Missing authz: POST /rest/basket/:id/checkout - checkout without basket ownership check ▀ Review: no authz guard detected
GET	/rest/products/:id/reviews	-	Missing authz: GET /rest/products/:id/reviews - product review read ▀ Review: no authz guard detected
PUT	/rest/products/:id/reviews	-	Missing authz: PUT /rest/products/:id/reviews - review update without authorship check ▀ Review: no authz guard detected

21 further entry point(s) in this category carry no linked finding and no elevated review signal, and are not listed individually (52 total). The complete route inventory is available in `route-inventory.json` and, when exported, `pentest-tasks.yaml`.

6. Security Architecture

This chapter is organized by security-control category. The architecture section avoids artificial control IDs and finding-ID columns in overview tables. Findings are listed only where the affected control is described.

§6 schema v2 (13-section control-category layout). Cataloged controls: 24 total - 0 adequate, 10 partial, 3 weak, 7 unsafe, 4 missing. Linked threats: 68.

How to read the verdicts. Every control category (and every sub-control below it) carries exactly one status. The two red verdicts do **not** mean the same thing - the distinction determines the required response:

Status	Meaning	What it asks of you
● Adequate	Control is present and sound	Nothing - keep it
● Partial	Present, but with meaningful gaps	Close the gap
● Weak	Present, but has exploitable gaps	Strengthen it
● Unsafe	Present and relied upon, but defeated / trivially bypassable	Fix the existing control
● Missing	Control was never built	Add the control
-	Not applicable to this codebase	-

So "● Unsafe" on a control category does *not* mean the control is absent - it means the control exists but does not hold (e.g. an MD5 password hash, a raw-SQL query path, a hardcoded signing key). "● Missing" is reserved for controls that were never built (e.g. no Content-Security-Policy header).

6.1 Security Control Overview

Control category	Verdict	Main reason
6.2 Identity and Authentication Controls	● Unsafe	11 routed findings; catalogued controls are present but defeated (e.g. Route Authentication Middleware, TOTP Two-Factor Authentication).
6.3 Session and Token Controls	● Unsafe	3 routed findings; catalogued controls are present but defeated (e.g. Session Token Signing (JWT Based), Session Token Storage and Cookie Hardening).
6.4 Authorization Controls	● Weak	10 routed findings; catalogued controls are weak (e.g. Centralized Authorization Policy, Object-Level Ownership Check (BOLA/IDOR)).
	● Weak	

Control category	Verdict	Main reason
6.5 Query Construction and Data Access Controls		2 routed findings; catalogued controls are weak (e.g. SQL Injection Prevention via Parameterized Queries).
6.6 Input Boundary Validation Controls	● Weak	5 routed findings; catalogued controls are weak (e.g. Schema and Allowlist Input Validation).
6.7 Output Encoding and Rendering Controls	● Unsafe	2 routed findings; catalogued controls are present but defeated (e.g. Angular Template Sanitization and XSS Prevention).
6.8 Browser and Cross-Origin Controls	● Weak	2 routed findings; catalogued controls are weak (e.g. CORS and WebSocket origin Enforcement).
6.9 Cryptography Secrets and Data Protection	● Unsafe	6 routed findings; catalogued controls are present but defeated (e.g. Managed Secret Storage, Transport Layer Encryption (TLS)).
6.10 File Parser and Outbound Request Controls	● Weak	11 routed findings; no compensating controls catalogued.
6.11 Operations Runtime and Supply Chain Controls	● Missing	8 routed findings; required controls not in place (e.g. Automated SCA scanning, Automated dependency updates).
6.12 Real-time and Not Applicable Controls	● Partial	0 routed findings; 1 partial control (e.g. LLM Prompt Injection Prevention) leave gaps.
6.13 Defense-in-Depth Summary	-	No controls or findings routed to this category.

6.2 Identity and Authentication Controls

Dependent crossings: [tb-1](#) refuted - ● [F-007](#) — Insecure JWT Verification, ● [F-017](#) — Unauthenticated WebSocket Channel, ● [F-027](#) — HTTP access logs browsable without authentication (`server.ts:281`), +1 · [tb-2](#) refuted - ● [F-006](#) — Hard-coded JWT signing key (`lib/insecurity.ts:21`), ● [F-007](#) — Insecure JWT Verification

Systemic weaknesses: [W-005](#), [W-008](#) **Verdict:** ● Unsafe

Controls covered:

- [6.2.1 TOTP Two-Factor Authentication](#)
- [6.2.2 Password-Based Login](#)
- [6.2.3 User Registration](#)
- [6.2.4 Password Reset](#)
- [6.2.5 Social Login](#)

Implemented controls: `lib/insecurity.ts:52` ; `server.ts:604` ; `routes/2fa.ts` ; Detected in scope: POST `/rest/user/login` ; Detected in scope: POST `/api/Users` ; Detected in scope: POST `/rest/user/reset-password` .

Assessment: Six authentication mechanisms are in place, but every one carries a critical defect that nullifies the protection. The login path is injectable, registration accepts mass-assigned roles, the reset flow relies on guessable security answers, the OAuth adapter derives credentials from public email addresses, and TOTP leaks credential validity before the second factor is checked. Each successful flow above terminates in the server issuing a session token; the signing, validation, propagation, storage, and lifecycle of that token are described in [§6.3 Session and Token Controls](#).

Authentication mechanisms (at a glance). Every authentication mechanism detected on the application, its effective status, where it is assessed, and its linked findings. Controls are catalogued by domain, so JWT/session handling is assessed under [§6.3 Session and Token Controls](#) and password hashing under [§6.9 Cryptography Secrets and Data Protection](#).

Mechanism	Status	Assessed in	Findings
User registration	● Unsafe	§6.2	● F-012 — Mass assignment of role on registration ● F-023 — Client-trusted security decision — <code>registerWebsocketEvents.ts:50</code> ● F-030 — Unauthenticated data exposure — <code>registerWebsocketEvents.ts:30</code> ● F-035 — Inefficient regex complexity — <code>registerWebsocketEvents.ts:46</code> W-002 — <code>Authorization</code> is implemented route by route
Password login	● Unsafe	§6.2	-
Password reset / change	● Unsafe	§6.2	● F-036 — Password change without current password — <code>routes/changePassword.ts:39</code>
Password storage (hashing)	● High	§6.9	W-008 — Security-sensitive data uses weak cryptographic primitives
JWT / bearer-token session	● Unsafe	§6.3	W-004 — Secrets are committed to source instead of a managed store ● F-007 — Insecure JWT Verification ● F-002 — JWT in <code>localStorage</code> — <code>request.interceptor.ts:13</code> ● F-073 — Token flow accepts a stolen bearer token without — <code>login.component.ts:148</code>
Session-token storage	● Partial	§6.3	● F-002 — JWT in <code>localStorage</code> — <code>request.interceptor.ts:13</code> W-011 — Cross-Site Scripting is implemented inconsistently

Mechanism	Status	Assessed in	Findings
Multi-factor authentication (TOTP / 2FA)	● Partial	§6.2	● F-051 — Credential validity oracle before 2FA — <code>routes/login.ts:38</code>
OAuth / OIDC federated login	● Unsafe	§6.2	● F-015 — Unvalidated OAuth token — <code>oauth.component.ts:28</code> ● F-016 — Predictable derived credential — <code>oauth.component.ts:30</code>

Route Authentication Middleware.

Status: ● Weak - most API routes require a valid Bearer token via `lib/insecurity.ts:52`, but several management-level routes are registered in `server.ts` without the middleware and are reachable without credentials.

`lib/insecurity.ts:52`; `server.ts:604`

`express-jwt` middleware is configured at `lib/insecurity.ts:52` and applied at `server.ts:604` to protect the `/api/` and `/rest/` route namespaces. Any request without a valid Bearer token is rejected with a 401 before the route handler runs.

Security assessment

Two independent gaps undermine the middleware coverage:

- [F-040](#f-040) – Sensitive Routes Registered Without Authentication Middleware establishes that several sensitive routes are registered outside the protected namespace - they respond to unauthenticated callers.
- The JWT verification at `lib/insecurity.ts:52` does not pin the algorithm (`algorithms` option absent on `express-jwt@0.1.3`), so a token carrying `alg:none` in its header bypasses the signature check even where the middleware runs.

Relevant findings

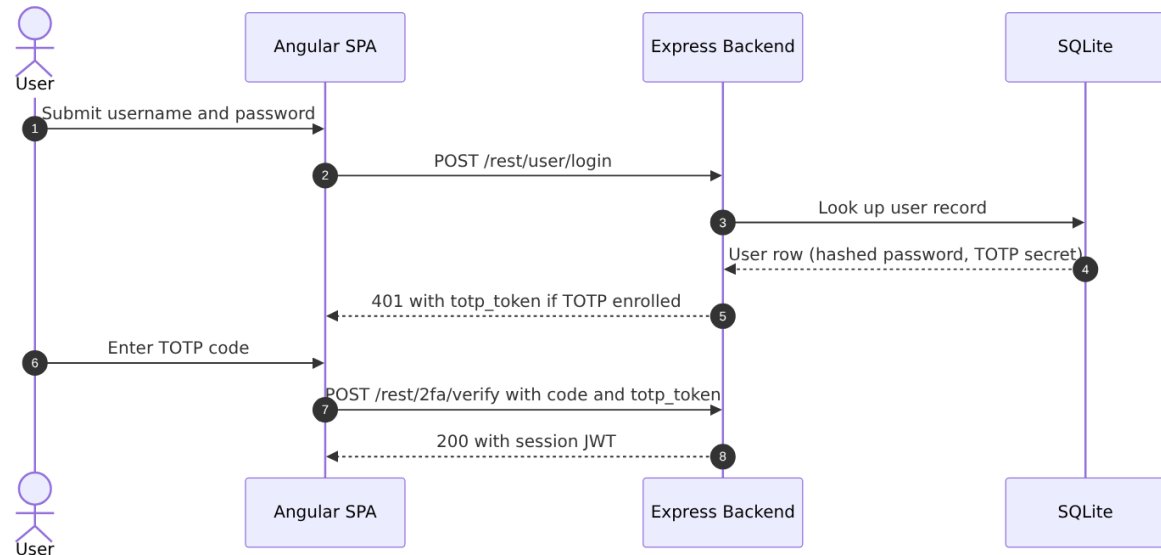
- ● F-007 — Insecure JWT Verification — JWT verification does not restrict the accepted algorithm, making the middleware bypassable with an unsigned token.
- ● F-013 — Weak password recovery mechanism — Security question reset issues compound authentication coverage gaps.
- ● F-015 — Unvalidated OAuth token — The OAuth token is accepted without server-side verification, bypassing the credential check that the middleware is meant to enforce.

6.2.1 TOTP Two-Factor Authentication

Status: ● Partial - TOTP enrollment and verification are implemented in `routes/2fa.ts`, but the second factor is optional and the login endpoint leaks credential validity before the code is checked.

TOTP-based second-factor authentication is available as an opt-in step for any registered user. After a successful password login, users who have enrolled a TOTP device must supply a valid time-based one-time code before the session token is issued.

The diagram shows the intended two-factor login flow:



Security assessment

Two weaknesses reduce the 2FA control to partial:

- `routes/login.ts:38` returns a distinct response when the password is correct but TOTP is pending - an attacker can use this to confirm that a credential pair is valid before ever supplying a second factor (credential oracle, ● F-051 — Credential validity oracle before 2FA).
- TOTP is opt-in and is not enforced for admin accounts, so the highest-privilege sessions operate on a single factor.

Relevant findings

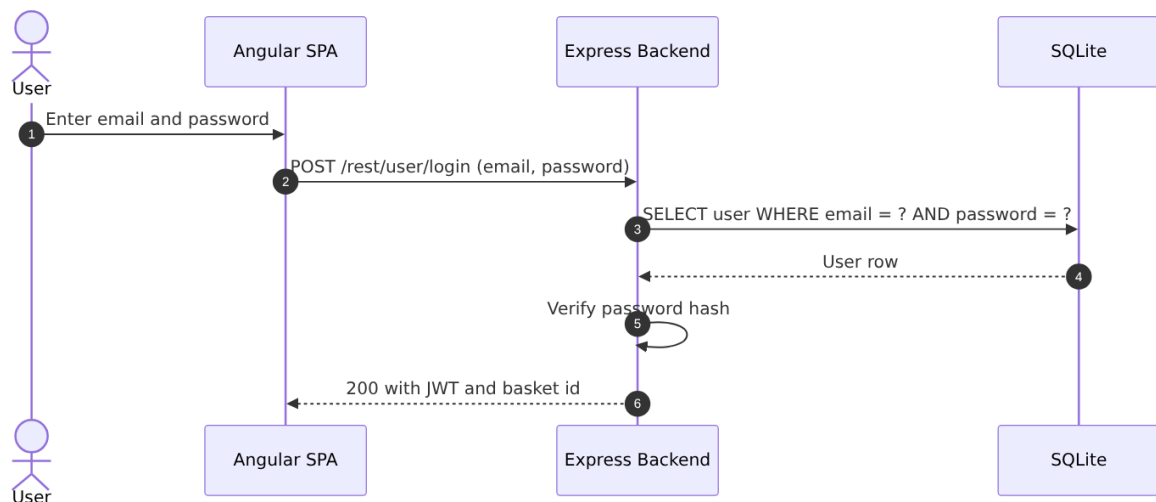
- ● F-007 — Insecure JWT Verification — Algorithm-pinning gap means the session JWT issued after 2FA can be forged without knowing the 2FA secret.
- ● F-013 — Weak password recovery mechanism — The account-recovery path (security questions) bypasses 2FA entirely.
- ● F-015 — Unvalidated OAuth token — The OAuth path enters the session flow without a 2FA step.

6.2.2 Password-Based Login

Status: ● Unsafe - `routes/login.ts` builds the login SQL query by interpolating `req.body.email` directly into a raw string, bypassing Sequelize parameter binding and allowing authentication bypass via SQL injection.

Detected in scope: POST `/rest/user/login`

The diagram shows the intended login flow from credential submission to session issuance:



Security assessment

Two independent weaknesses sit on the login path:

- `routes/login.ts` interpolates `req.body.email` into a raw `models.sequelize.query()` call - submitting `' OR 1=1--` as the email returns the first database row (the seeded admin account) and authenticates without a password.
- No rate limit protects `POST /rest/user/login`, enabling unlimited credential-stuffing or brute-force attempts.

Relevant findings

- **F-008** — SQL injection in login query — SQL injection in the login query allows authentication bypass without knowing a valid password.
- **F-013** — Weak password recovery mechanism — The password-reset flow shares the same weak credential-identity model.
- **F-014** — Long-lived registry publish credential — Missing brute-force protection on the login endpoint enables offline password guessing at network speed.
- **F-016** — Predictable derived credential — Predictable credential derivation in the OAuth adapter maps into the same login endpoint.
- **F-018** — Hard-coded seeded account credentials — Hardcoded credentials in `login.component.ts` bypass the login check for demo accounts.

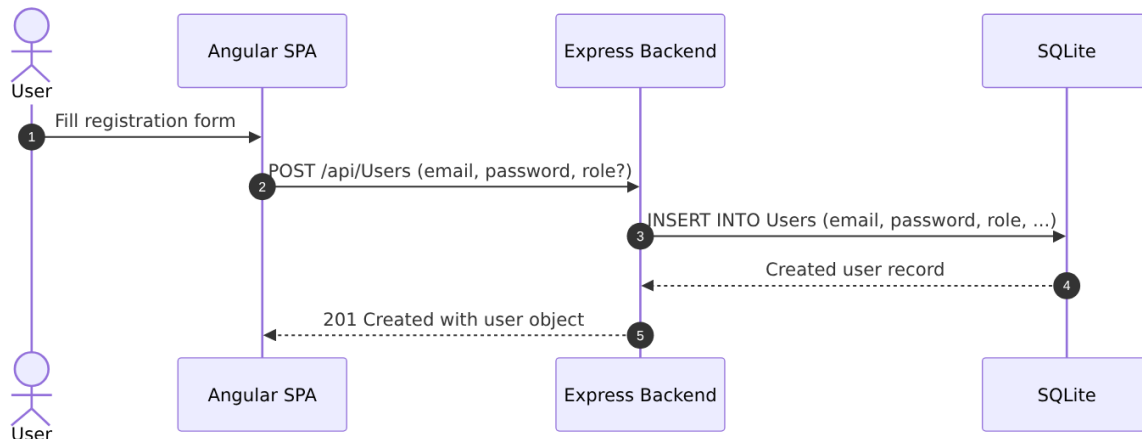
6.2.3 User Registration

Status: **Unsafe** - `POST /api/Users` accepts a `role` field in the request body and persists it without sanitization, allowing any caller to self-assign the `admin` role.

Detected in scope: `POST /api/Users`

`finale-rest` exposes Sequelize models as REST endpoints. `POST /api/Users` creates a new user record from the JSON body. No field allowlist is applied before the record is written to SQLite.

The diagram shows the registration flow:



Security assessment

Three weaknesses span the registration boundary:

- `finale-rest` writes every field in the request body to the `Users` table - including `role: admin` - because no attribute allowlist strips privileged columns before the INSERT.
- `registerWebSocketEvents.ts:50` makes a server-side security decision based on a client-supplied value without server-side re-validation.
- `registerWebSocketEvents.ts:46` compiles a user-controlled regex on each connection, enabling ReDoS with a crafted pattern.

Relevant findings

- **F-012** — Mass assignment of role on registration — Mass assignment of `role` on the `Users` endpoint grants admin privileges to any registrant.
- **F-014** — Long-lived registry publish credential — No rate limit on registration enables account enumeration and bulk account creation.
- **F-023** — Client-trusted security decision — The WebSocket handler trusts a client-supplied flag for a server-side access decision.
- **F-030** — Unauthenticated data exposure — Unauthenticated callers receive data from the WebSocket channel before authentication completes.
- **F-035** — Inefficient regex complexity — User-controlled input reaches a regex engine without length or complexity constraints, enabling ReDoS.

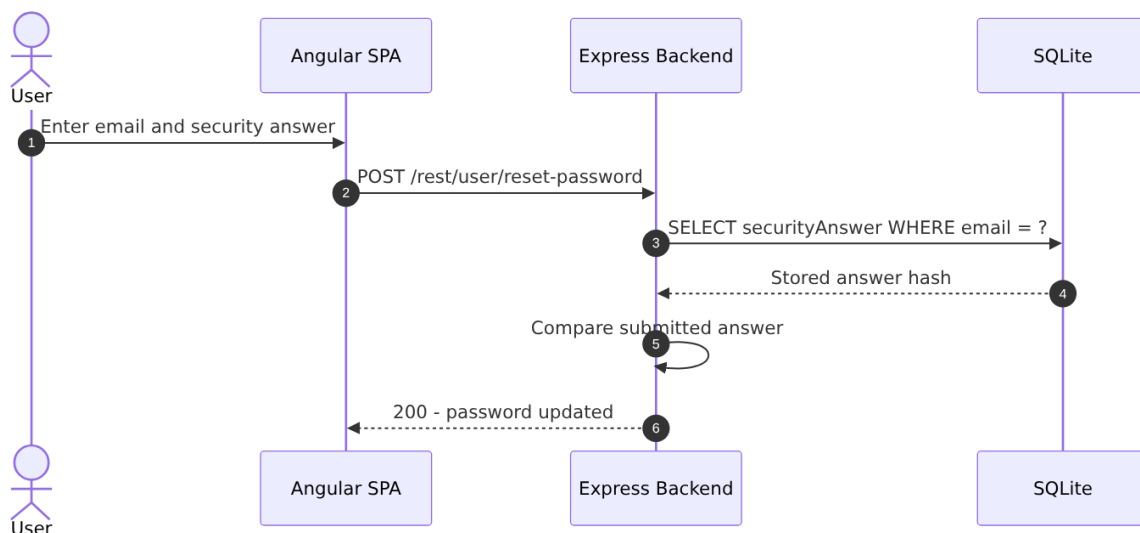
6.2.4 Password Reset

Status: ● Unsafe - the reset flow authenticates identity through security questions whose answers are publicly guessable from user profile data, with no rate limit on the reset endpoint.

Detected in scope: POST `/rest/user/reset-password`

POST `/rest/user/reset-password` accepts an email and a security question answer; if the answer matches the stored value, it sets a new password. No token or out-of-band channel (email link, SMS) is involved - the reset is complete in a single unauthenticated HTTP request.

The diagram shows the intended reset flow:



Security assessment

The reset relies entirely on security questions. Answers to questions like "Mother's maiden name?" or pet names are derivable from public social-media profiles. An attacker who knows the target's email can exhaust guesses without restriction - there is no lockout or CAPTCHA on the endpoint.

Relevant findings

- **F-013** — Weak password recovery mechanism — Security-question answers are guessable from public profile data, making password reset a credential-bypass vector for any user whose profile is semi-public.

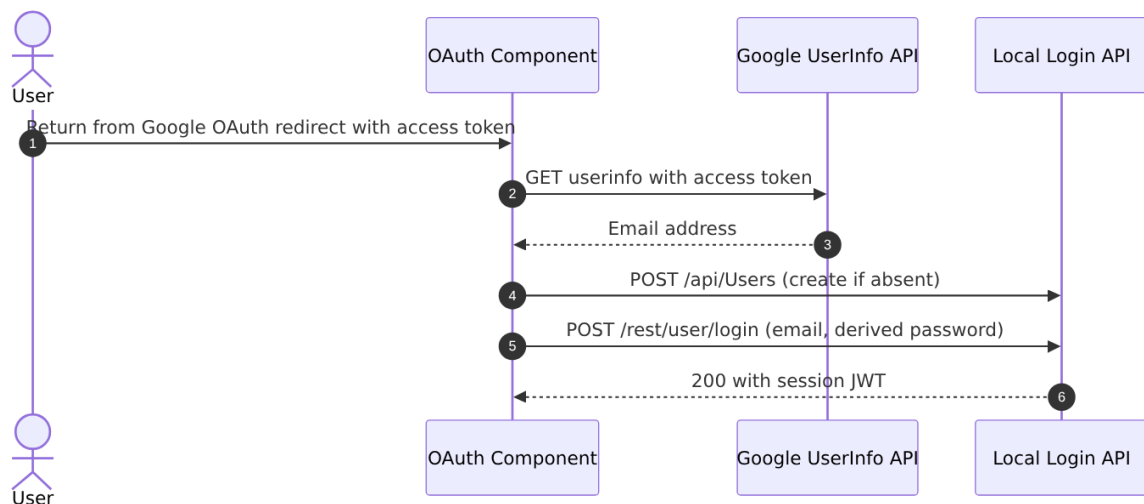
6.2.5 Social Login

Status: ● Unsafe - `oauth.component.ts` reads the access token from the URL redirect without server-side verification, then derives a deterministic local password from the user's email - an attacker who knows the email can compute the credential and log in without ever touching Google.

Detected in scope: `frontend/src/app/oauth`

The OAuth flow is implemented as a frontend adapter, not a server-side authorization-code exchange. `oauth.component.ts` reads the Google access token from the redirect URL and calls Google's `userinfo` endpoint to retrieve the email. It then derives a local password using a predictable hash function and calls `POST /rest/user/login` with those derived credentials.

The diagram shows how the frontend adapter enters the local login flow:



Security assessment

Two independent failures in the adapter:

- `oauth.component.ts:28` passes the Google access token directly to `UserService.oauthLogin()` with no state parameter, nonce, or CSRF protection - an attacker who intercepts or forges the redirect can replay the token.
- Line 30 derives the local password deterministically from the email address using a function documented in `lib/insecurity.ts` - knowing the target's email is sufficient to compute the credential and authenticate without Google involvement.

Relevant findings

- **F-015** — Unvalidated OAuth token — The access token from the OAuth redirect is used without server-side verification, making it trivially replaceable.
- **F-016** — Predictable derived credential — The deterministic local credential derived from the email allows account takeover for any OAuth user without interacting with the OAuth provider.

6.3 Session and Token Controls

Dependent crossings: **tb-1** refuted - **F-007** — Insecure JWT Verification, **F-017** — Unauthenticated WebSocket Channel, **F-027** — HTTP access logs browsable without authentication (`server.ts:281`), +1 · **tb-2** refuted - **F-006** — Hard-coded JWT signing key (`lib/insecurity.ts:21`), **F-007** — Insecure JWT Verification

Verdict: **Unsafe**

Controls covered:

- [6.3.1 Session Token Signing](#)
- [6.3.2 Session Token Storage and Cookie Hardening](#)

Implemented controls: `lib/insecurity.ts:21,54`; `routes/login.ts:18`; `frontend/src/app/Services/request.interceptor.ts:13`.

Assessment: A single locally-signed JWT covers every authenticated session regardless of which login flow in §6.2 established it. Both catalogued controls are defeated: the signing key is hardcoded in source and the token is stored in `localStorage` where any XSS payload can read it.

6.3.1 Session Token Signing

Status: ● Unsafe - the `RS256` private key is committed to the public repository as a string literal at `lib/insecurity.ts:21`, and the verification middleware does not restrict the accepted algorithm, so tokens with `alg:none` are accepted.

`lib/insecurity.ts:21,54`; `routes/login.ts:18`

`RS256`-signed JWTs are issued at `lib/insecurity.ts:54` after successful login and returned in the login response. The signing key is read from `lib/insecurity.ts:21` at module load time and reused for every token issued.

Security assessment

Two independent failures break the signing boundary:

- `lib/insecurity.ts:21` stores the RSA private key as a multi-line string literal committed to the public GitHub repository - anyone with repository read access can sign arbitrary tokens accepted by the server.
- `express-jwt@0.1.3` at `lib/insecurity.ts:52` is called without the `algorithms` option, so a token whose header declares `alg:none` bypasses signature verification entirely.

Relevant findings

- ● **F-002** — JWT in `localStorage` — JWT stored in `localStorage` means a forged token retrieved via XSS is immediately usable against the API.
- ● **F-042** — Missing session token revocation — Algorithm confusion allows a forged unsigned token to pass the verification middleware.
- ● **F-057** — Session cookie without `HttpOnly` — The session cookie set alongside the JWT lacks `HttpOnly`, widening the XSS-to-session-theft surface.

6.3.2 Session Token Storage and Cookie Hardening

Status: ● Partial - `request.interceptor.ts:13` stores the JWT in `localStorage` (readable by same-origin scripts) and the session cookie at `login.component.ts:104` is missing the `HttpOnly` flag.

After login, `request.interceptor.ts:13` stores the session JWT in browser `localStorage` and attaches it as an `Authorization: Bearer` header on every subsequent API request. A cookie-based session is also set in some flows for server-side access control.

Security assessment

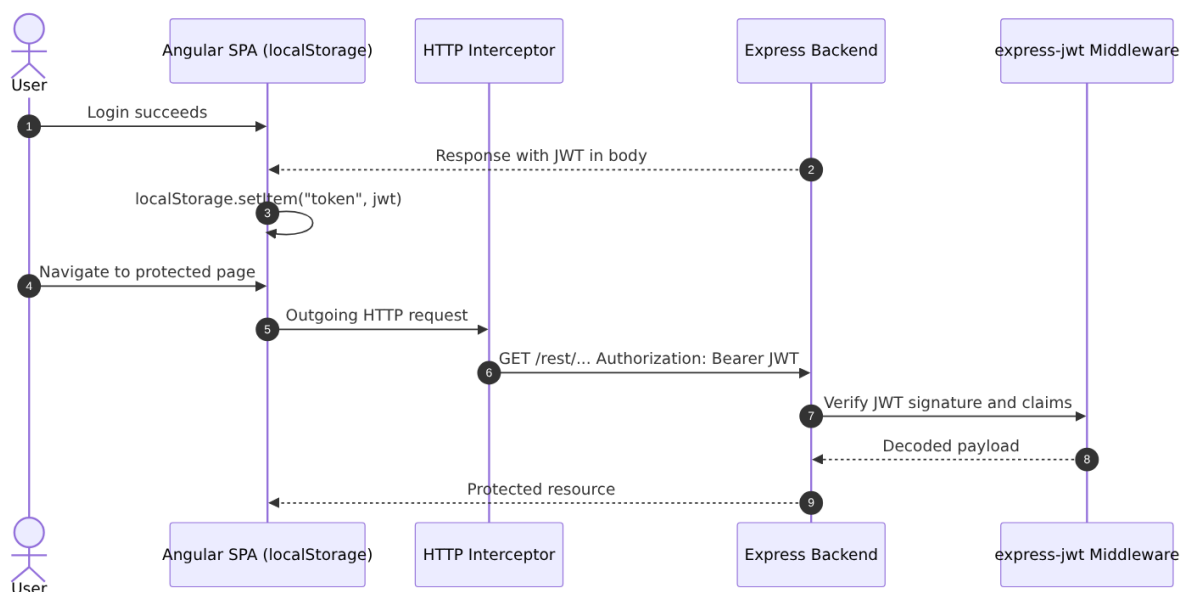
Two weaknesses on the storage boundary:

- `localStorage` is accessible to any JavaScript running in the same origin - any XSS payload on the Angular SPA reads the token and replays it against the REST API for the full lifetime of the session.
- The cookie set at `login.component.ts:104` lacks the `HttpOnly` attribute, making it readable by the same XSS payload and providing no fallback isolation.

Relevant findings

- **F-002** — JWT in `localStorage` — JWT in `localStorage` allows any XSS exploit to exfiltrate the session token without browser restriction.
- **F-042** — Missing session token revocation — A forged token passed via `localStorage` injection is accepted by the verification middleware due to the algorithm gap.
- **F-057** — Session cookie without `HttpOnly` — Session cookie without `HttpOnly` compounds `localStorage` exposure — both token stores are readable by a single XSS payload.

The diagram shows the token lifecycle from issuance through browser storage and bearer presentation to server-side validation:



6.4 Authorization Controls

Dependent crossings: `tb-1` refuted - **F-065** — Basket item update without ownership check · `tb-3` unconfirmed - **F-041** — Missing workflow permissions block

Systemic weaknesses: **W-002** **Verdict:** **Weak**

Controls covered:

- [6.4.1 Centralized Authorization Policy](#)
- [6.4.2 Object-Level Ownership Check](#)
- [6.4.3 Management Endpoint Protection](#)

Implemented controls: `server.ts` (route registrations); `server.ts`; `server.ts:606-607`.

Assessment: `Authorization` is route-level and inconsistent. `server.ts` registers each route individually with or without `express-jwt` middleware; there is no central policy object or framework-level authorization layer. Angular route guards protect the admin UI but have no server-side counterpart. Object-level ownership is not checked on basket, order, or profile endpoints - any authenticated user can read or modify any other user's data by guessing the resource ID.

6.4.1 Centralized Authorization Policy

Status: ● Weak - `server.ts` attaches `express-jwt` per-route at registration time, but no shared policy object governs which roles can access which operations, and several routes skip the middleware.

`server.ts` (route registrations)

`server.ts` is the single point where routes are registered and middleware is optionally attached. The `express-jwt` middleware at `lib/insecurity.ts:52` is applied to most `/api/` and `/rest/` routes; role checks beyond "authenticated vs. not" are performed inline in individual route handlers.

Security assessment

No shared authorization policy exists. Role decisions are scattered across handlers with no auditable mapping between operation and required role. `PUT /api/Products` is registered without authentication middleware, allowing unauthenticated callers to modify product records. Angular route guards at the frontend enforce role-based navigation but are bypassed by direct API calls.

Relevant findings

- ● **F-003** — Client-side authorization decision — Missing server-side authorization on the product update route allows unauthenticated modification.
- ● **F-005** — Unmediated data-store access control — Client-side-only route guards leave the underlying API endpoints unprotected.
- ● **F-009** — Insecure Direct Object Reference — Inconsistent middleware attachment across route registrations leaves privilege escalation paths open.

6.4.2 Object-Level Ownership Check

Status: ● Weak - basket, order, and user-data endpoints accept any resource ID from the request path without verifying that the authenticated user owns that object.

Object-level authorization ensures that a user can only access or modify their own records - orders, baskets, saved addresses - even when they have a valid session token. The basket endpoint at `/rest/basket/:id` returns cart contents for any numeric basket ID.

Security assessment

`/rest/basket/:id` returns the full basket contents for any ID passed in the URL. No check compares `req.user.bid` from the JWT against the `:id` parameter - a signed-in customer increments the ID to read other customers' baskets and order histories. The same pattern applies to several `/api/Users/:id` sub-routes.

Relevant findings

- **F-003** — Client-side authorization decision — Authorization bypass on the basket endpoint exposes other customers' order data without ownership verification.
- **F-005** — Unmediated data-store access control — Missing ownership check on user-profile endpoints allows reading and modifying any account by guessing the numeric user ID.
- **F-009** — Insecure Direct Object Reference — The pattern of ID-based access with no ownership comparison recurs across multiple endpoints, indicating a structural gap rather than an isolated bug.

6.4.3 Management Endpoint Protection

Status: ● Partial - the `/administration` Angular route is guarded on the frontend, but the underlying admin API endpoints at `/rest/admin/` lack server-side role enforcement beyond authentication.

Administrative functions - user management, coupon operations, challenge status - appear in the `/administration` Angular route. `server.ts:606-607` applies `express-jwt` to the `/rest/admin/` prefix, requiring a valid token.

Security assessment

A valid session token is required to reach `/rest/admin/` endpoints, but the backend does not check that the token's `role` claim equals `admin`. Any authenticated user who calls those endpoints directly - bypassing the Angular route guard - receives the admin responses. Admin-role tokens are obtainable via the mass-assignment registration gap (● **F-012** — Mass assignment of role on registration).

Relevant findings

- **F-003** — Client-side authorization decision — Unauthenticated product-modification paths signal that the role enforcement gap extends beyond the admin prefix.

- ● **F-005** — Unmediated data-store access control — Frontend-only role guards provide no server-side protection against direct API access by a normal-role session.
- ● **F-009** — Insecure Direct Object Reference — A user who self-assigns `role: admin` during registration passes the token check and accesses all admin endpoints without restriction.

6.5 Query Construction and Data Access Controls

Dependent crossings: [tb-4](#) refuted - ● **F-008** — SQL injection in login query

Systemic weaknesses: [W-001](#) **Verdict:** ● Weak

Controls covered:

- [6.5.1 SQL Injection Prevention via Parameterized Queries](#)

Implemented controls: `models/index.ts` ; `routes/login.ts:11` .

Assessment: Sequelize ORM backs the majority of data reads with parameterized queries; the login and search routes are deliberate exceptions that call raw `models.sequelize.query()` with user-controlled input concatenated into the SQL string. As the highest-traffic unauthenticated entry points, these two routes are the highest-priority injection targets.

6.5.1 SQL Injection Prevention via Parameterized Queries

Status: ● Partial - Sequelize model-layer queries use bound parameters throughout, but `routes/login.ts` and the product-search route bypass the ORM and build SQL strings by concatenating user-supplied values.

Sequelize models at `models/index.ts` define typed associations and handle most SELECT, INSERT, and UPDATE operations through auto-generated parameterized queries. Most API routes read data exclusively through these model methods.

Security assessment

Two routes call `models.sequelize.query()` directly with user-controlled input interpolated into the SQL string, bypassing `Sequelize` parameterization:

- `routes/login.ts:11` concatenates `req.body.email` - submitting `' OR 1=1--` as the email returns the seeded admin row without a password.
- The product search route concatenates the search term in the same pattern, enabling data exfiltration from the `Products` table.

Relevant findings

- ● **F-008** — SQL injection in login query — SQL injection at the login query enables authentication bypass and full `users` table read access.
- ● **F-019** — NoSQL \$where JavaScript injection — SQL injection in the product-search query allows reading arbitrary rows from the SQLite database.

6.6 Input Boundary Validation Controls

Dependent crossings: [tb-1](#) refuted - ● [F-012](#) — Mass assignment of role on registration, ● [F-023](#) — Client-trusted security decision (`registerWebsocketEvents.ts:50`), ● [F-043](#) — Wallet identity accepted without proof of possession (`routes/web3Wallet.ts:16`), +1 · [tb-2](#) unconfirmed - ● [F-008](#) — SQL injection in login query · [tb-3](#) unconfirmed - ● [F-010](#) — Dependency lockfile disabled by config — `.npmrc:1`

Systemic weaknesses: [W-009](#) **Verdict:** ● Weak

Controls covered:

- [6.6.1 Validation Approach](#)
- [6.6.2 Schema and Allowlist Input Validation](#)

Implemented controls: `server.ts:204,246,411-413`.

Assessment: A JSON body parser and a request-size cap are applied globally at `server.ts:204,246`. Upload routes at lines 411-413 add MIME-type and file-size constraints via `multer`. Beyond those, validation is ad-hoc per route - no framework-level schema enforces allowed fields or value shapes across the API surface, leaving mass-assignment and injection vectors open.

6.6.1 Validation Approach

Status: ● Weak - input validation is performed inline in individual route handlers with no shared schema library; the approach produces inconsistent coverage and leaves most endpoints without field-level constraints.

Input validation limits which values and structures a request body or path parameter may carry. `server.ts` applies a global JSON body parser and a request-size cap before routing. Some routes add per-field regex or length checks inline in the handler function.

Security assessment

No shared validation framework (`Joi`, `express-validator`, `Zod`) is applied at the router level. Per-route inline checks are inconsistent - most endpoints accept arbitrary JSON bodies. `registerWebsocketEvents.ts:46` compiles a user-supplied string as a regex on every WebSocket connection, enabling ReDoS with a crafted pattern that causes exponential backtracking and blocks the event loop.

Relevant findings

- ● [F-034](#) — Unbounded LLM consumption on `/rest/chat` — Missing input length constraints on user-controlled fields enable oversized payloads to reach downstream handlers.
- ● [F-061](#) — Unbounded in-memory session map — Absence of schema validation on the order and feedback endpoints allows unexpected fields that exploit mass-assignment.

- **F-062** — Unbounded CI matrix fan-out — Regex ReDoS at `registerWebsocketEvents.ts:46` is a direct consequence of compiling user-controlled patterns without a length or complexity guard.

6.6.2 Schema and Allowlist Input Validation

Status: ● Partial - `server.ts:204,246,411-413` applies content-type and size limits, and `multer` constrains file uploads, but field-level allowlists are absent from the majority of API endpoints.

`server.ts:204` limits JSON body size and `server.ts:246` applies a raw-body cap. Upload routes at lines 411-413 use `multer` to restrict accepted MIME types to images and limit file size before the route handler runs.

Security assessment

`multer` constraints are the strongest validation in the codebase, and they apply only to the upload routes. All other endpoints accept arbitrary JSON bodies via `finale-rest` model exposure - no attribute allowlist strips privileged columns (e.g. `role`, `isAdmin`) before the record is written. The gap enables mass-assignment across every model-backed endpoint.

Relevant findings

- **F-034** — Unbounded LLM consumption on `/rest/chat` — Absence of field-level allowlisting on model-backed endpoints allows privileged attribute injection.
- **F-061** — Unbounded in-memory session map — Unvalidated body fields on the order endpoint accept attacker-controlled data that reaches downstream business logic.
- **F-062** — Unbounded CI matrix fan-out — Schema-level enforcement is absent from the feedback and coupon endpoints, allowing parameter injection that the route handler does not anticipate.

6.7 Output Encoding and Rendering Controls

Systemic weaknesses: **W-011** **Verdict:** ● Unsafe

Controls covered:

- [6.7.1 Angular Template Sanitization and XSS Prevention](#)

Implemented controls: `frontend/src/app/search-result/search-result.component.ts:110,143`; `frontend/src/app/about/about.component.ts:119`; `frontend/src/app/administration/administration.component.ts:73,91`.

Assessment: Angular's default template escaping is present and active for most bindings throughout the SPA. Four explicit `bypassSecurityTrustHtml` calls in three components deliberately disable that protection for user-controlled values, creating reflected and stored XSS sinks that any attacker-controlled input can reach.

6.7.1 Angular Template Sanitization and XSS Prevention

Status: ● Unsafe - `bypassSecurityTrustHtml()` is called on URL query parameters and backend-supplied content in three components, overriding Angular's default escaping and rendering attacker-controlled markup as live HTML.

Angular's `DomSanitizer` escapes all template-bound values by default. Most of the SPA relies on standard template bindings (`{{ value }}` and `[textContent]`) that the compiler sanitizes automatically.

Security assessment

Four `bypassSecurityTrustHtml` calls across three components override the default escaping:

- `search-result.component.ts:143` marks the `q` URL query parameter as trusted HTML - a crafted search URL executes arbitrary JavaScript in the victim's session and can exfiltrate the JWT from `localStorage`.
- `search-result.component.ts:110` marks every backend-supplied product description as trusted - a stored XSS payload in any product description executes for every customer who browses the catalogue.
- `about.component.ts:119` trusts the about-page content fetched from the API.
- `administration.component.ts:73, 91` trusts admin-displayed user-generated content in two separate bindings.

Relevant findings

- ● **F-001** — Cross-Site Scripting — Reflected XSS via the `q` URL parameter allows session theft for any signed-in user who follows a crafted link.
- ● **F-049** — Unescaped anchor persisted into product description — Stored XSS via product descriptions executes for every shopper who views the search results or catalogue page.

6.8 Browser and Cross-Origin Controls

Verdict: ● Weak

Controls covered:

- [6.8.1 CORS and WebSocket Origin Enforcement](#)

Implemented controls: `lib/startup/registerWebsocketEvents.ts:20`.

Assessment: An origin check at `registerWebsocketEvents.ts:20` gates WebSocket connections. Two gaps remain:

- The Express HTTP API has no explicit CORS configuration; browser same-origin policy applies by default - adequate for a same-origin SPA but leaving no auditable header.
- No `Content-Security-Policy` header is set on any response, leaving XSS payloads free to load external scripts and exfiltrate data.

6.8.1 CORS and WebSocket Origin Enforcement

Status: ● Partial - WebSocket origin enforcement is in place at `registerWebsocketEvents.ts:20`, but HTTP CORS headers are absent and no `Content-Security-Policy` restricts external script loading.

Cross-origin policy controls which third-party origins may read API responses and which scripts a browser will execute. `lib/startup/registerWebsocketEvents.ts:20` checks the `Origin` header on WebSocket upgrade requests and closes connections from unexpected origins before the event handler runs.

Security assessment

Two gaps in the browser policy surface:

- The Express HTTP API sets no `Access-Control-Allow-Origin` header. The browser enforces same-origin policy by default, but the absence of an explicit CORS policy means any future API key or credential-in-URL pattern would be silently cross-origin readable.
- No `Content-Security-Policy` header is served on any HTTP response. Any XSS payload in the SPA may load external scripts, beacon data to attacker-controlled servers, or read the `localStorage` JWT without browser restriction.

Relevant findings

- ● [F-045](#) — Cookie-only authentication on POST `/profile` — Missing `Content-Security-Policy` allows XSS payloads to load and execute arbitrary external scripts in the victim's browser.
- ● [F-048](#) — Missing `Content-Security-Policy` — The absence of explicit CORS configuration leaves the origin policy implicit and unauditabile.

6.9 Cryptography Secrets and Data Protection

Systemic weaknesses: [W-004](#), [W-008](#) **Verdict:** ● Unsafe

Controls covered:

- [6.9.1 Managed Secret Storage](#)
- [6.9.2 Transport Layer Encryption](#)

Implemented controls: `lib/insecurity.ts:21`; `routes/checkKeys.ts:10`; `server.ts` (plain HTTP binding).

Assessment: Three secrets used by every authenticated session - the RSA signing key, the HMAC basket secret, and the cookie secret - are committed as string literals to the public repository. TLS is absent from the application's own binding; the server listens on plain HTTP and relies on an external proxy that is not evidenced in the codebase. Both controls exist in form but are defeated in practice.

6.9.1 Managed Secret Storage

Status: ● Unsafe - the RSA private key, HMAC secret, and cookie secret are hardcoded at `lib/insecurity.ts:21` and committed to the public repository; no runtime injection or rotation mechanism exists.

▲ **Anti-pattern:** Secrets hardcoded in source

`lib/insecurity.ts` centralizes three secrets consumed at startup: the `RS256` private key used to sign every session JWT, an HMAC key used to sign basket IDs, and a cookie-session secret. `routes/checkKeys.ts:10` reads these values at boot time. All three are plain string literals in the source file.

Security assessment

Two compounding failures make the secret-storage control fully defeated:

- `lib/insecurity.ts:21` stores the RSA private key as a committed string literal in a public repository - the key has been public since the repo was created, making every JWT ever issued by this application forgeable by anyone with repository access.
- No environment variable injection, vault integration, or Secrets Manager reference is present; secrets are identical across every deployment and cannot be rotated without a code change and re-deployment.

Relevant findings

- ● **F-006** — Hard-coded JWT signing key — The hardcoded RSA private key allows offline JWT forgery for any user or role without server interaction.
- ● **F-018** — Hard-coded seeded account credentials — Hardcoded credentials in login-related source files extend the secret-exposure surface beyond the signing key.
- ● **F-020** — Non-cryptographic RNG for a secret/token — Additional hardcoded values discovered in the codebase confirm the pattern is systemic rather than isolated.

6.9.2 Transport Layer Encryption

Status: ● Partial - `server.ts` binds only to plain HTTP via `http.createServer()`; TLS termination is absent from the application and relies on an external proxy not evidenced in the repository.

`server.ts` creates and starts the HTTP server, binding the Express application to a TCP port.

Security assessment

`server.ts` calls `http.createServer()` - no `https.createServer()` call, no TLS certificate loading, no `http2` configuration. Every credential, JWT, and customer record sent between the browser and the API travels over plaintext HTTP unless a TLS-terminating reverse proxy sits in front of the application. No such proxy configuration (nginx, Caddy, AWS ALB, etc.) is committed to the repository.

Relevant findings

- **F-006** — Hard-coded JWT signing key — Credentials and hardcoded signing material transmitted over plain HTTP can be intercepted by a network observer.
- **F-018** — Hard-coded seeded account credentials — Hardcoded secrets sent in-band are captured in full by any traffic intercept against the plain-HTTP binding.
- **F-020** — Non-cryptographic RNG for a secret/token — Sensitive application values exposed over an unencrypted channel compound the key-management exposure.

6.10 File Parser and Outbound Request Controls

No trust boundary in this model depends on this control class.

Systemic weaknesses: **W-007** **Verdict:** ● Weak

Controls covered:

- [6.10.1 File Parser and Outbound Request Handling](#)

Implemented controls: `multer` file-size and MIME-type limits on upload routes; `unzipper` archive extraction; `libxmljs2` XML parser; static `ftp/` directory served by Express.

Assessment: Upload routes apply `multer` constraints that limit file size and restrict MIME types. `unzipper` handles archive extraction and `libxmljs2` handles XML parsing, but neither is configured with path-traversal or external-entity restrictions. Eleven findings are routed here; the dominant weaknesses are Zip Slip, XXE, and unauthenticated static-file exposure of the `ftp/` directory.

6.10.1 File Parser and Outbound Request Handling

Status: ● Weak - `multer` size and MIME limits protect the upload entry point, but `unzipper` extracts archive entries without path-traversal checks and `libxmljs2` resolves external entities, enabling Zip Slip and XXE attacks.

File upload handling restricts which files users may submit to the application. `multer` is configured on the upload routes to enforce a file-size cap and an accepted MIME-type list before the uploaded data reaches the route handler. ZIP archives are extracted with `unzipper` and XML documents are parsed with `libxmljs2`.

Security assessment

Three distinct parser weaknesses sit behind the `multer` gate:

- `unzipper` extracts archive entries using the path embedded in each ZIP entry header without canonicalizing or bounding the destination - a ZIP file containing `../../server.js` as an entry name overwrites application files during extraction.

- `libxmljs2` is invoked with external-entity resolution enabled; a crafted XXE payload in an uploaded XML document reads arbitrary server-side files and returns their contents in the parsed document.
- The `ftp/` directory is mounted as an authenticated static tree that is accessible to any caller who can construct the URL, exposing previously uploaded files and generated reports.

Relevant findings

- **F-011** — Server-side eval of stored username — Archive extraction without path-traversal containment enables Zip Slip file overwrite.
- **F-021** — Input compiled as template source — XXE in the `libxmljs2` XML parser enables server-side file reads via external entity injection.
- **F-026** — Denylisted path traversal in erasure layout — Insecure direct file-path construction exposes server-side files via the download endpoint.
- **F-028** — Unvalidated URL fetch in profile image — The `ftp/` static directory is accessible to unauthenticated callers, exposing generated and uploaded files.
- **F-029** — Secrets broadcast to job environment — Additional outbound-request or parser weaknesses extend the attack surface beyond archive and XML inputs.

6.11 Operations Runtime and Supply Chain Controls

Systemic weaknesses: **W-003** **Verdict:** ● Missing

Controls covered:

- [6.11.1 Automated SCA scanning](#)
- [6.11.2 Automated dependency updates](#)
- [6.11.3 Lockfile hygiene](#)

Implemented controls: GitHub Actions CI pipeline runs test and build jobs; `package.json` declares npm dependencies; Docker image is used for containerized deployments.

Assessment: The CI pipeline builds and tests the application but runs no software composition analysis step. Neither Dependabot nor Renovate is configured.

`package-lock.json` is absent from the repository, making installs non-deterministic. Known-vulnerable packages including `express-jwt@0.1.3` and `libxmljs2@0.30` have shipped in this state for an extended period with no automated detection mechanism.

6.11.1 Automated SCA scanning

Status: ● Missing - no `npm audit`, Snyk scan, or equivalent SCA step runs in the GitHub Actions CI pipeline; known-vulnerable packages ship without any automated gate.

The GitHub Actions CI pipeline runs lint, test, and build jobs. Software composition analysis checks installed packages against public vulnerability databases and blocks builds or raises alerts when known-vulnerable versions are detected.

Security assessment

None of the CI workflow files include an `npm audit`, `snyk test`, or OWASP Dependency-Check step. `express-jwt@0.1.3` carries a publicly documented algorithm-confusion vulnerability that would be flagged by any SCA tool, yet it has shipped without detection. The same applies to `libxmljs2@0.30`, which has known XXE-related advisories.

Relevant findings

- **F-004** — Missing server-side security audit logging — A known-vulnerable dependency version is present in `package.json` with no CI gate to detect or block it.
- **F-010** — Dependency lockfile disabled by config — Additional outdated packages with public CVEs are installed without any automated scanning coverage.
- **F-027** — HTTP access logs browsable without authentication — Supply-chain risk from unscanned transitive dependencies compounds the direct-dependency exposure.

6.11.2 Automated dependency updates

Status: **Missing** - no Dependabot or Renovate configuration is present in the repository; dependency versions are pinned manually and have not been updated to address known vulnerabilities.

Automated update tooling raises pull requests when newer versions of direct or transitive dependencies become available. GitHub Dependabot and Renovate are the two dominant options for npm projects.

Security assessment

Neither `.github/dependabot.yml` nor `renovate.json` exists. `express-jwt@0.1.3` is superseded by versions that pin the accepted algorithm by default; no automated tool prompts an upgrade. `libxmljs2` has patched releases available that are never pulled in.

Relevant findings

- **F-004** — Missing server-side security audit logging — `express-jwt@0.1.3` remains in use because no automated mechanism has raised an update PR since its vulnerability was disclosed.
- **F-010** — Dependency lockfile disabled by config — Additional stale packages persist for the same reason — manual update discipline alone is insufficient for a project of this dependency count.
- **F-027** — HTTP access logs browsable without authentication — Without update automation, transitive dependencies drift silently into vulnerable ranges over time.

6.11.3 Lockfile hygiene

Status: **Missing** - `package-lock.json` is absent from the repository; `npm install` resolves dependency versions non-deterministically from floating `semver` ranges on every build.

A committed lockfile pins the exact resolved version of every direct and transitive dependency, making builds reproducible and preventing a compromised package registry from serving a different version than the one audited. npm writes `package-lock.json` on first install when it does not exist.

Security assessment

`package-lock.json` is not committed and is absent from `.gitignore` as a deliberate inclusion. Each CI run and each developer install resolves `semver` ranges independently - a malicious package registry could serve a version within the declared range that differs from what was last tested. The absence of a lockfile also means `npm audit` output would vary between runs, making SCA results unreliable even if a step were added.

Relevant findings

- **F-004** — Missing server-side security audit logging — Non-deterministic installs mean the vulnerable `express-jwt@0.1.3` pin in `package.json` may drift further as patch versions are released without audit coverage.
- **F-010** — Dependency lockfile disabled by config — Floating transitive version ranges create a supply-chain substitution surface on every install.
- **F-027** — HTTP access logs browsable without authentication — Without a lockfile, verifying that a specific audited dependency tree was actually built is not possible from CI artifacts alone.

6.12 Real-time and Not Applicable Controls

Verdict: ● Partial

Controls covered:

- [6.12.1 LLM Prompt Injection Prevention](#)

Implemented controls: `routes/chat.ts:108`.

Assessment: `routes/chat.ts` serves a chat endpoint backed by an AI model; a keyword filter at line 108 attempts to reject prompt-injection payloads before they reach the model. The real-time channel runs on `Socket.IO`, wired in `lib/startup/registerWebsocketEvents.ts`; its security findings are routed to [§6.2](#) and [§6.4](#). No finding was derived specifically for WebSocket message-injection or channel authorization beyond those sections.

6.12.1 LLM Prompt Injection Prevention

Status: ● Partial - `routes/chat.ts:108` applies a keyword filter before forwarding user messages to the AI model, but the filter is bypassable through encoding and character-substitution obfuscation.

The chatbot feature accepts user-supplied messages at `routes/chat.ts` and forwards them to an AI model. A keyword-based filter at line 108 inspects each message before dispatch, rejecting inputs that contain known prompt-injection trigger phrases.

Security assessment

Keyword filters are a weak defense against prompt injection because they match literal strings rather than semantic intent. Common obfuscation techniques - Unicode character substitution, zero-width character insertion, Base64 encoding, multilingual phrasing - bypass the filter and deliver the instruction to the model intact. No system-prompt isolation or instruction-hierarchy mechanism prevents user messages from overriding the model's operating context or extracting the system prompt.

Relevant findings

- No dedicated finding routed in this assessment.

6.13 Defense-in-Depth Summary

Verdict: -

The codebase carries a small set of positive controls that provide genuine layered protection where they apply:

- Angular's default template escaping is active for the majority of component bindings.
- `multer` enforces file-size and MIME-type limits on upload routes before data reaches the parser.
- `express-jwt` middleware provides a consistent token-verification boundary on most API routes.
- The WebSocket upgrade handler checks the `origin` header before admitting connections.
- The `RS256` algorithm choice for JWT signing is structurally sound even though the private key is compromised.

Four structural gaps remain. First, move all secrets out of source into runtime-injected values - environment variables or a secrets manager - and pin the accepted JWT algorithm in `express-jwt`. Second, replace raw SQL concatenation at the login and search routes with `Sequelize` parameterized queries. Third, remove `bypassSecurityTrustHtml` calls on user-controlled values and restrict them to genuinely trusted static content. Fourth, add a committed `package-lock.json`, a Dependabot or Renovate configuration, and an `npm audit` or Snyk step to CI - these three together close the entire supply-chain gap.

7. Weakness Register

Systemic control gaps behind the findings, ordered by severity (W-001 = most severe). Each weakness names the missing, home-grown, or misused control, the findings that evidence it, the components it spans, and its remediation. A weakness may also rest on observed unsafe practice or an absent architectural control with no confirmed exploit - only confirmed findings carry a CVSS score.

- **Critical** · [W-001](#) - Database access relies on concatenated queries · confirmed · 1 finding · 3 components
- **Critical** · [W-002](#) - `Authorization` is implemented route by route · confirmed · 5 findings · 3 components
- **Critical** · [W-003](#) - Build pipeline trusts mutable third-party references · confirmed · 2 findings · 1 component
- **Critical** · [W-004](#) - Secrets are committed to source instead of a managed store · confirmed · 5 findings · 4 components
- **High** · [W-005](#) - Endpoints are reachable without enforced authentication · confirmed · 1 finding · 1 component
- **High** · [W-006](#) - Denial of Service is implemented inconsistently · confirmed · 2 findings · 3 components
- **High** · [W-007](#) - Input handling lacks enforced boundary validation · confirmed · 1 finding · 2 components
- **High** · [W-008](#) - Security-sensitive data uses weak cryptographic primitives · confirmed · 2 findings · 3 components
- **Medium** · [W-009](#) - Denial of Service is implemented inconsistently · confirmed · 3 findings · 5 components
- **Medium** · [W-010](#) - Injection is implemented inconsistently · observed-practice · 1 finding · 1 component
- **Medium** · [W-011](#) - Cross-Site Scripting is implemented inconsistently · observed-practice · 1 finding · 1 component
- **Medium** · [W-012](#) - Frontend rendering lacks enforced output encoding · design-risk · 0 findings · 0 components

W-001 — Database access relies on concatenated queries

● **Critical** · design weakness · confirmed · 1 finding

Database queries are assembled from application values instead of passing those values through an enforced parameterised data-access path. This leaves every call site responsible for preserving query structure.

Architectural anti-pattern - Raw SQL string interpolation. Login and search routes build database queries by concatenating untrusted input directly into query strings rather than using parameterized bindings, making the full database readable and writable without authentication.

Confirmed findings:

- **F-008** — SQL injection in login query

Architecture evidence: Parameterized Queries, ORM / Repository Layer

Affected components: C-02, C-01, backend-api

Remediation:

- **Structural** — provide one parameterised repository or query-builder path and prohibit application-value interpolation in database queries
- **Tactical** — **M-018** — Use parameterized database queries

W-002 — Authorization is implemented route by route

Critical · design weakness · confirmed · 5 findings

`Authorization` depends on per-handler checks instead of a policy boundary that consistently enforces role, ownership, and tenant scope. New routes can bypass protection by omitting a local check.

Confirmed findings:

- **F-009** — Insecure Direct Object Reference
- **F-038** — Coupon discount bounded only by prompt text (`routes/chat.ts:184`)
- **F-039** — Deluxe upgrade skips payment check (`routes/deluxe.ts:43`)
- **F-040** — Sensitive Routes Registered Without Authentication Middleware
- **F-065** — Basket item update without ownership check

Architecture evidence: Centralised AuthZ Policy, Role / Scope Enforcement, Ownership Check, Object-Level Ownership Check, Tenant Scoping

Affected components: C-02, C-01, C-07

Remediation:

- **Structural** — enforce authorization through a shared server-side policy layer and make ownership and tenant scope mandatory inputs to data access
- **Tactical** — **M-001** — Enforce object-level (ownership) authorization, **M-019** — Enforce object-level (ownership) authorization, **M-048** — Enforce correct server-side authorization, **M-049** — Enforce correct server-side authorization, **M-004** — Enforce server-side authorization on every endpoint, **M-050** — Enforce server-side authorization on every endpoint, **M-075** — Enforce object-level (ownership) authorization

W-003 — Build pipeline trusts mutable third-party references

Critical · design weakness · confirmed · 2 findings

CI/CD workflows resolve third-party actions and other build dependencies to mutable tags or branches instead of immutable commit digests, so a retagged or compromised upstream runs inside the pipeline with its token and secret scope.

Confirmed findings:

- ● [F-010](#) — Dependency lockfile disabled by config
- ● [F-022](#) — Unpinned third-party action (`image_actions.yml:33`)

Architecture evidence: Commit-SHA Action Pinning, Dependency Provenance Verification

Affected components: [C-05](#)

Remediation:

- **Structural** — pin every third-party action and build dependency to an immutable commit SHA (or a vetted internal mirror) and enforce SHA-pinning in CI
- **Tactical** — ● [M-020](#) — Pin third-party dependencies to immutable versions, ● [M-032](#) — Set least-privilege CI workflow permissions

W-004 — Secrets are committed to source instead of a managed store

● **Critical** · design weakness · confirmed · 5 findings

Cryptographic keys, credentials, and other high-entropy secrets are embedded as literals in source or `config` rather than resolved at runtime from a managed secret store, so anyone with repository read access obtains reusable signing material and credentials.

Architectural anti-pattern - Secrets hardcoded in source. The RSA private key used to sign all session tokens is a string literal in committed source code, meaning every developer, CI runner, and repository fork has permanent read access to the material needed to forge any user or administrator session.

Confirmed findings:

- ● [F-006](#) — Hard-coded JWT signing key (`lib/insecurity.ts:21`)
- ● [F-018](#) — Hard-coded seeded account credentials (`data/datacreator.ts:200`)
- ● [F-031](#) — Hard-coded wallet mnemonic (`routes/checkKeys.ts:10`)
- ● [F-066](#) — Security answers stored under embedded HMAC key (`data/datacreator.ts:691`)

Practice sites:

- ● [F-056](#) — Hard-coded test credential (`login.component.ts:62`) (`frontend/src/app/login/login.component.ts:62`)

Architecture evidence: Managed Secret Store, Runtime Secret Injection

Affected components: [C-02](#), [C-01](#), [C-03](#), [C-07](#)

Remediation:

- **Structural** — move every secret to a managed secret store or injected environment configuration, rotate the exposed values, and add secret-scanning to CI
- **Tactical** — ● [M-016](#) — Move cryptographic keys to a managed secret store, ● [M-002](#) — Move secrets to a managed secret store, ● [M-028](#) — Move secrets to a managed secret store, ● [M-041](#) — Move secrets to a managed secret store, ● [M-009](#) — Move cryptographic keys to a managed secret store, ● [M-076](#) — Move cryptographic keys to a managed secret store, ● [M-066](#) — Move secrets to a managed secret store

W-005 — Endpoints are reachable without enforced authentication

● **High** · design weakness · confirmed · 1 finding

Sensitive API routes and real-time channels are exposed without an enforced authentication check at the endpoint boundary. Access control depends on each handler (or the caller) remembering to require a session, so an unauthenticated client can reach privileged operations directly.

Confirmed findings:

- ● [F-017](#) — Unauthenticated WebSocket Channel

Architecture evidence: Route Authentication Middleware, Server-Side Session Enforcement

Affected components: [C-06](#)

Remediation:

- **Structural** — enforce authentication centrally at the routing and channel boundary so every exposed endpoint requires a verified session unless explicitly marked public
- **Tactical** — ● [M-027](#) — Require authentication on every exposed endpoint

W-006 — Denial of Service is implemented inconsistently

● **High** · implementation weakness · confirmed · 2 findings

Resource-consuming operations lack rate limiting or bounds, so a single actor can exhaust capacity.

Confirmed findings:

- ● [F-034](#) — Unbounded LLM consumption on `/rest/chat` (`server.ts:638`)

Practice sites:

- ● [F-061](#) — Unbounded in-memory session map (`lib/insecurity.ts:74`)

Affected components: [C-02](#), [C-01](#), [C-07](#)

Remediation:

- **Structural** — apply rate limiting, request quotas, and input-size bounds on resource-intensive endpoints
- **Tactical** —  [M-044](#) — Rate-limit and lock out repeated authentication attempts,  [M-071](#) — Rate-limit and lock out repeated authentication attempts

W-007 — Input handling lacks enforced boundary validation

 **High** · design weakness · confirmed · 1 finding

Request handlers do not validate input against one enforced server-side schema or allowlist at the boundary - validation is either absent on the vulnerable parameters or limited to rejecting selected bad patterns (a blacklist). New encodings and unanticipated input forms can therefore reach downstream parsers or interpreters.

Confirmed findings:

-  [F-026](#) — Denylisted path traversal in erasure layout (`routes/dataErasure.ts:104`)

Architecture evidence: Schema Validation, Allowlist Validation

Affected components: [C-01](#), backend-api

Remediation:

- **Structural** — enforce server-side schemas and domain-specific allowlists before input reaches parsing, persistence, or command construction
- **Tactical** —  [M-036](#) — Constrain file paths to a safe base directory

W-008 — Security-sensitive data uses weak cryptographic primitives

 **High** · implementation weakness · confirmed · 2 findings

Password, token, or integrity protection uses a weak hash, predictable random source, or insufficient work factor. The application may use a standard library, but the selected primitive does not provide the required security property.

Confirmed findings:

-  [F-025](#) — Unsalted MD5 password hashing

Practice sites:

-  [F-020](#) — Non-cryptographic RNG for a secret/token (`lib/insecurity.ts:53`) (`lib/insecurity.ts:53`)
- `data/datacreator.ts:313`

Affected components: [C-02](#), [C-01](#), backend-api

Remediation:

- **Structural** — standardise on a password KDF, a CSPRNG for secrets, and modern authenticated cryptographic primitives with centrally reviewed parameters
- **Tactical** — ● [M-035](#) — Hash passwords with a strong, salted algorithm, ● [M-030](#) — Use cryptographically secure random values

W-009 — Denial of Service is implemented inconsistently

● **Medium** · implementation weakness · confirmed · 3 findings

Resource-consuming operations lack rate limiting or bounds, so a single actor can exhaust capacity.

Confirmed findings:

- ● [F-062](#) — Unbounded CI matrix fan-out (`ci.yml:58`)

Practice sites:

- ● [F-063](#) — Unbounded chat history resend (`chat-conversation.component.ts:131`)
(`frontend/src/app/chatbot/chat-conversation/chat-conversation.component.ts:131`)
- ● [F-064](#) — Listener guard race opens unbounded providers (`routes/nftMint.ts:16`)
(`routes/nftMint.ts:16`)
- `data/mongodb.ts:9`

Affected components: [C-01](#), [C-05](#), [C-03](#), [C-08](#), [C-07](#)

Remediation:

- **Structural** — apply rate limiting, request quotas, and input-size bounds on resource-intensive endpoints
- **Tactical** — ● [M-072](#) — Offload CPU-bound work and bound execution time, ● [M-073](#) — Rate-limit expensive requests and bound input size, ● [M-074](#) — Rate-limit expensive requests and bound input size

W-010 — Injection is implemented inconsistently

● **Medium** · implementation weakness · observed-practice · 1 finding

Untrusted input reaches an interpreter (SQL, OS shell, template, XML/YAML parser) without parameterization or a strict allowlist. It is systemic when the same unsafe construction pattern recurs across handlers rather than being an isolated slip.

Practice sites:

- ● [F-021](#) — Input compiled as template source (`routes/userProfile.ts:87`) (`routes/userProfile.ts:87`)

Affected components: [C-01](#)

Remediation:

- **Structural** — adopt parameterized queries / safe APIs and centralised input validation as the only sanctioned data-access and parsing path
- **Tactical** —  [M-003](#) — Remove server-side evaluation of untrusted input,  [M-031](#) — Remove server-side evaluation of untrusted input

W-011 — Cross-Site Scripting is implemented inconsistently

 **Medium** · implementation weakness · observed-practice · 1 finding

Output is rendered without contextual encoding and without a Content Security Policy backstop, so attacker-controlled markup executes in other users' browsers.

Practice sites:

-  [F-057](#) — Session cookie without HttpOnly (`login.component.ts:104`) (`frontend/src/app/login/login.component.ts:104`)

Affected components: [C-03](#)

Remediation:

- **Structural** — encode output through the framework's contextual escaping and add a strict `Content-Security-Policy` as defence in depth
- **Tactical** —  [M-067](#) — Set secure session-cookie attributes

W-012 — Frontend rendering lacks enforced output encoding

 **Medium** · design weakness · design-risk

Browser rendering paths write HTML or DOM content directly without one enforced contextual-encoding or sanitisation boundary. Safety depends on each individual component using the right API.

Architecture evidence: Template Autoescape, Output Encoding, Content Security Policy

Remediation:

- **Structural** — use framework-safe rendering by default and isolate any required raw HTML behind one reviewed sanitisation and Trusted Types policy
-

8. Findings Register

Findings are grouped by severity (Critical → High → Medium → Low); within a tier they are ordered by attack vektor (Repo-Read → Internet-Anon → Internet-User → Victim-Required). Each finding is a card with the same fixed fields, in order: **Severity** · **Component** · **Location** → **Issue** → **Root cause** → **Evidence** → **Fix** → **Classification** (with external CWE / OWASP links).

Risk Distribution: ● Critical: 8 · ● High: 32 · ● Medium: 28 · ● Low: n/a · **Total findings: 68 STRIDE Coverage:** Spoofing: 12 · Tampering: 14 · Repudiation: 2 · Information Disclosure: 19 · Denial of Service: 8 · Elevation of Privilege: 13

The systemic root-cause view is summarized in **Top Weaknesses** in the Management Summary; evidence-backed weaknesses are documented in the [Weakness Register](#).

Findings index:

- F-001 — Cross-Site Scripting
- F-006 — Hard-coded JWT signing key (`lib/insecurity.ts:21`)...
- F-007 — Insecure JWT Verification
- F-008 — SQL injection in login query
- F-009 — Insecure Direct Object Reference
- F-010 — Dependency lockfile disabled by config — `.npmrc:1`
- F-011 — Server-side eval of stored username (`routes/userProfile.ts:61`)...
- F-012 — Mass assignment of role on registration
- F-015 — Unvalidated OAuth token (`oauth.component.ts:28`)...
- F-019 — NoSQL \$where JavaScript injection
- F-031 — Hard-coded wallet mnemonic (`routes/checkKeys.ts:10`)...
- F-038 — Coupon discount bounded only by prompt text (`routes/chat.ts:184`)...
- F-039 — Deluxe upgrade skips payment check (`routes/deluxe.ts:43`)...
- F-043 — Wallet identity accepted without proof of possession...
- F-056 — Hard-coded test credential (`login.component.ts:62`)...
- F-002 — JWT in localStorage (`request.interceptor.ts:13`)...
- F-003 — Client-side authorization decision (`app.guard.ts:54`)...
- F-013 — Weak password recovery mechanism
- F-014 — Long-lived registry publish credential (`ci.yml:327`)...
- F-016 — Predictable derived credential (`oauth.component.ts:30`)...
- F-017 — Unauthenticated WebSocket Channel
- F-018 — Hard-coded seeded account credentials (`data/datacreator.ts:200`)...
- F-020 — Non-cryptographic RNG for a secret/token (`lib/insecurity.ts:53`)...
- F-021 — Input compiled as template source (`routes/userProfile.ts:87`)...
- F-022 — Unpinned action — `.github/workflows/image_actions.yml:33`
- F-023 — Client-trusted security decision (`registerWebsocketEvents.ts:50`)...
- F-024 — Passwords passed in URL query string (`routes/changePassword.ts:14`)...
- F-025 — Unsalted MD5 password hashing
- F-026 — Denylisted path traversal in erasure layout...
- F-027 — HTTP access logs browsable without authentication (`server.ts:281`)...

- F-028 — Unvalidated URL fetch in profile image...
- F-029 — Secrets broadcast to job environment (`ci.yml:253`)...
- F-030 — Unauthenticated data exposure (`registerWebsocketEvents.ts:30`)...
- F-032 — Rate limiter keyed on client-supplied header (`server.ts:346`)...
- F-033 — No rate limit or lockout on login (`server.ts:596`) — `server.ts:596`
- F-034 — Unbounded LLM consumption on `/rest/chat` (`server.ts:638`)...
- F-035 — Inefficient regex complexity (`registerWebsocketEvents.ts:46`)...
- F-036 — Password change without current password (`routes/changePassword.ts:39`)...
- F-037 — Client-controlled wallet credit amount (`routes/wallet.ts:27`)...
- F-040 — Sensitive Routes Registered Without Authentication Middleware
- F-041 — Missing workflow permissions block
- F-044 — Open redirect (`routes/redirect.ts:19`) — `routes/redirect.ts:19`
- F-053 — Admin configuration endpoints exposed unauthenticated (`server.ts:607`)...
- F-055 — Client-gated tool-call disclosure (`chat-conversation.component.ts:68`)...
- F-065 — Basket item update without ownership check
- F-073 — Token flow accepts a stolen bearer token without...
- F-004 — Missing server-side security audit logging
- F-005 — Unmediated data-store access control (`data/mongodb.ts:10`)...
- F-042 — Missing session token revocation (`lib/insecurity.ts:54`)...
- F-045 — Cookie-only authentication on POST `/profile` ...
- F-046 — Remote script piped to shell (`ci.yml:358`)...
- F-047 — Unpinned container base image — `Dockerfile:1`
- F-048 — Missing Content-Security-Policy
- F-049 — Unescaped anchor persisted into product description...
- F-050 — Mutable image tag without build attestation (`ci.yml:345`)...
- F-051 — Credential validity oracle before 2FA (`routes/login.ts:38`)...
- F-052 — Chat tool calls streamed to every caller (`routes/chat.ts:228`)...
- F-054 — Development error handler returns stack traces (`server.ts:682`)...
- F-057 — Session cookie without `HttpOnly` (`login.component.ts:104`)...
- F-058 — Ineffective author email masking in feedback (`data/datacreator.ts:578`)...
- F-059 — Plaintext security answer written to log (`data/datacreator.ts:692`)...
- F-060 — Raw provider error returned to caller (`routes/web3Wallet.ts:36`)...
- F-061 — Unbounded in-memory session map
- F-062 — Unbounded CI matrix fan-out (`ci.yml:58`) — `.github/workflows/ci.yml:58`
- F-063 — Unbounded chat history resend (`chat-conversation.component.ts:131`)...
- F-064 — Listener guard race opens unbounded providers (`routes/nftMint.ts:16`)...
- F-066 — Security answers stored under embedded HMAC key...
- F-067 — Entitlement token derived from account email (`data/datacreator.ts:198`)...

● Critical (8)

F-006 · Hard-coded JWT signing key (lib/insecurity.ts:21)

Severity: ● Critical - secret committed to the public source repo - extractable on clone, no prior access needed · **Component:** C-02 - JWT Authentication and Session Management ·

Location: `lib/insecurity.ts:21`

Weakness: [W-004](#) - Secrets are committed to source instead of a managed store

Trust boundary gap: [tb-2](#)  External - external → auth · Authentication: The tb-2 authentication leg assumes a JWT is accepted only when signed with the server-held RSA key; the key is embedded in source, so any party can satisfy it.

Issue: An attacker reads the RSA private key literal embedded at `lib/insecurity.ts:21`, signs an arbitrary JWT payload with it using `RS256`, and presents that token to any route guarded by `isAuthorized()` (`lib/insecurity.ts:52`). The signature validates against the matching public key, so the attacker holds a session for any user id and any role without ever submitting credentials.

Full authentication bypass for every account including administrators; the attacker mints tokens offline, so no login attempt, logout, or 2FA challenge is ever observed.

Evidence: ✓ verified - `lib/insecurity.ts:21` assigns a complete PEM-encoded RSA private key to the module-level constant `privateKey`; line 54 passes it to `jwt.sign()` as the signing key for every issued session token.

Fix: Move the cryptographic key out of source control into a managed secret store and rotate it → ● [M-016](#) — Move cryptographic keys to a managed secret store (`insecurity.ts:21`)

Classification: Cryptographic Failures · STRIDE: Spoofing · [CWE-321](#) · [OWASP A04:2025](#) · walkthrough [Walkthrough §3.6](#)

F-007 · Insecure JWT Verification

Severity: ● Critical - elevated as an attack-chain keystone (individual baseline: High) ·

Component: [C-02](#) - JWT Authentication and Session Management · **Location:** Multiple locations (6)

Trust boundary gap: [tb-2](#)  External - external → auth · Authentication: This boundary states that `expressJwt` accepts a token only when it carries the server RSA signature; without an algorithms allowlist the verifier also accepts attacker-chosen HMAC signatures.

[tb-1](#)  External - external → backend · Authentication: The authentication leg of tb-1 assumes `expressJwt` proves caller identity, but the middleware accepts any algorithm the token header names.

Instances (6): ● `lib/insecurity.ts:52`, ● `lib/insecurity.ts:55`, ● `lib/insecurity.ts:53`, ● `lib/insecurity.ts:56`, ● `lib/insecurity.ts:189`, ● `routes/verify.ts:120`

Issue: An attacker reads the RSA public key from `encryptionkeys/jwt.pub` (exposed at `lib/insecurity.ts:20`) and forges a JWT with `alg: HS256`, using that PEM text as the HMAC secret. Because `isAuthorized()` configures `express-jwt` with no `algorithms` allowlist, the verifier trusts the attacker-supplied `alg` header and accepts the token as a valid server-issued session.

Any unauthenticated internet client can impersonate any account, including administrators, on every route behind the primary authentication boundary tb-2.

Evidence: ✓ verified - `lib/insecurity.ts:52` calls `expressJwt` with the object `{ secret: publicKey }` cast to `any` and supplies no `algorithms` property; `lib/insecurity.ts:189` repeats the same omission in `jwt.verify(token, publicKey, callback)`.

```
// lib/insecurity.ts:52
    return str
  }

  export const isAuthorized = () => expressJwt(({ secret: publicKey }) as any)
  export const denyAll = () => expressJwt({ secret: '' + Math.random() } as any)
  export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256'
})
    export const verify = (token: string) => token ? (jwt.verify as ((token: string, secret: string) => boolean))(token, publicKey) : false
```

Fix: Pin the signature algorithm explicitly and reject `alg:none` and unknown algorithms → ● [M-017](#) — Enforce JWT signature and algorithm verification (`insecurity.ts:52`)

Classification: Broken Authentication · STRIDE: Spoofing · [CWE-347](#) · [OWASP A07:2025](#) · walkthrough [Walkthrough §3.1](#)

F-008 · SQL injection in login query

Severity: ● Critical · **Component:** [C-02](#) - JWT Authentication and Session Management ·

Location: Multiple locations (2)

Weakness: [W-001](#) - Database access relies on concatenated queries

Trust boundary gap: [tb-4](#) - auth → sqlite-db · Query construction: This boundary assumes every credential lookup reaches SQLite through Sequelize parameter binding; this login query is assembled as raw text with the client-supplied email inside it.

Instances (2): ● `routes/login.ts:34` , ● `routes/search.ts:23`

Issue: An attacker submits `' OR 1=1--` as the login email. Because `req.body.email` is concatenated straight into the raw SQL string passed to `models.sequelize.query()`, the WHERE clause always matches, returning the first `users` row, and line 47 issues that account a valid session token.

The same primitive supports `UNION SELECT` reads of the `Users` and `SecurityAnswers` tables. Unauthenticated administrator login plus arbitrary read of the credential store, which contains MD5 password hashes and TOTP secrets for every user.

Evidence: ✓ verified - `routes/login.ts:34` interpolates `req.body.email` and the MD5 of `req.body.password` directly into the SQL string, bypassing `Sequelize` parameter binding.

```
// routes/login.ts:34

return (req: Request, res: Response, next: NextFunction) => {
  verifyPreLoginChallenges(req) // vuln-code-snippet hide-line
```

```
models.sequelize.query(`SELECT * FROM Users WHERE email = '${req.body.email} || ''}' AND password = '${security.hash(req.body.password || '')}' AND deletedAt IS NULL`, { model: UserModel, plain: true }) //
vuln-code-snippet vuln-line loginAdminChallenge loginBenderChallenge loginJimChallenge
.then((authenticatedUser) => { // vuln-code-snippet neutral-line loginAdminChallenge
loginBenderChallenge loginJimChallenge
const user = utils.queryResultToJson(authenticatedUser)
if (user.data?.id && user.data.totpSecret !== '') {
```

Fix: Switch all SQL execution to parameterised queries or ORM-bound parameters → ● [M-018](#)
 — Use parameterized database queries (login.ts:34)

Classification: Injection · STRIDE: Tampering · [CWE-89](#) · [OWASP A05:2025](#) · walkthrough
[Walkthrough §3.2](#)

F-009 · Insecure Direct Object Reference

Severity: ● Critical · **Component:** [C-01](#) - Express.js Backend API · **Location:** Multiple locations (20)

Weakness: [W-002](#) - Authorization is implemented route by route

Instances (20): ● routes/address.ts:11, ● routes/address.ts:18, ● routes/address.ts:29, ● routes/basketItems.ts:68, ● routes/dataExport.ts:26, ● routes/delivery.ts:34, ● routes/deluxe.ts:25, ● routes/deluxe.ts:30 ... (+12 more)

Issue: Server-side authorization MUST derive the resource owner from the authenticated session (req.user , req.session , or req.auth), never from attacker-controlled request data. Trusting req.body.UserId etc. enables horizontal privilege escalation across all authenticated tenants.

Evidence: ○ ambiguous

```
// routes/address.ts:11

export function getAddress () {
  return async (req: Request, res: Response) => {
    const addresses = await AddressModel.findAll({ where: { UserId: req.body.UserId } })
    res.status(200).json({ status: 'success', data: addresses })
  }
}
```

Fix: Tie every object lookup to the requesting user's identity and reject cross-tenant references → ○ [M-001](#) — Enforce object-level (ownership) authorization (address.ts:11) · ● [M-019](#) — Enforce object-level (ownership) authorization (address.ts:11)

Classification: Broken Access Control · STRIDE: Tampering · [CWE-639](#) · [OWASP A01:2025](#)

F-010 · Dependency lockfile disabled by config

Severity: ● Critical · **Component:** [C-05](#) - CI/CD Pipeline · **Location:** .npmrc:1

Weakness: [W-003](#) - Build pipeline trusts mutable third-party references

Issue: An attacker publishes a malicious version of any package in the transitive dependency tree - by typosquat, maintainer account takeover, or a compromised release pipeline upstream. Because `.npmrc` sets `package-lock=false`, no `package-lock.json` is ever generated or committed, so every `npm install` in CI and in the Docker build re-resolves the entire graph against the live registry and picks up the attacker's version the moment it is published.

None of those installs pass `--ignore-scripts`, so the malicious package's lifecycle script executes on the runner with the job's full secret environment and can also rewrite application source before it is packaged into the released image. A single poisoned package in the dependency graph yields arbitrary code execution inside jobs that hold the Docker Hub publish token and the Heroku production deploy key, and simultaneously backdoors the official Juice Shop container image consumed by training and CTF environments worldwide.

Evidence: ✓ verified - `.npmrc:1` contains `'package-lock=false'` and no `package-lock.json` exists in the repository, so dependency resolution is non-deterministic by configuration. `.github/workflows/ci.yml:51, 71, 110, 147, 203` and `238` all run bare `'npm install'` without `--ignore-scripts`, and `Dockerfile:5` runs `'npm install --omit=dev'` in the stage whose output is copied into the released image.

```
// .npmrc:1
package-lock=false
```

Fix: ● [M-020](#) — Pin third-party dependencies to immutable versions (`.npmrc:1`)

Classification: Supply-Chain Integrity · STRIDE: Tampering · [CWE-829](#) · [OWASP A03:2025](#) · walkthrough [Walkthrough §3.7](#)

F-011 · Server-side eval of stored username (routes/userProfile.ts:61)

Severity: ● Critical · **Component:** [C-01](#) - Express.js Backend API · **Location:** `routes/userProfile.ts:61`

Issue: An attacker sets their username to a `#{...}`-wrapped JavaScript payload via `POST /profile`; `routes/updateUserProfile.ts:38` persists it verbatim. On the next `GET /profile`, `routes/userProfile.ts:54` detects the pattern, extracts the inner text at line 56, and passes it to `eval()` at line 61 - running the attacker's JavaScript inside the Node process with server privileges instead of performing template substitution.

An authenticated attacker executes arbitrary code inside the Node process, reading the SQLite database file, the hard-coded signing key, and any other secret on the training host.

Evidence: ✓ verified - `routes/userProfile.ts:61` calls `eval()` on the substring extracted at line 56 from the stored username, and `routes/updateUserProfile.ts:38` writes `req.body.username` to that column with no character or pattern restriction.

```
// routes/userProfile.ts:61
    if (!code) {
        throw new Error('Username is null')
    }
    username = eval(code) // eslint-disable-line no-eval
} catch (err) {
    username = '\\' + username
}
```

Fix: Replace runtime code generation (eval/Function/template render) with a data-only execution path → ● [M-021](#) — Remove server-side evaluation of untrusted input
(userProfile.ts:61)

Classification: Code Execution via Unsafe Deserialization or Eval · STRIDE: Elevation of Privilege · [CWE-94](#) · [OWASP A08:2025](#) · walkthrough [Walkthrough §3.4](#)

F-012 · Mass assignment of role on registration

Severity: ● Critical - reaches a privileged operation on an unauthenticated endpoint ·
Component: [C-01](#) - Express.js Backend API · **Location:** Multiple locations (2)

Trust boundary gap: [tb-1](#) 🌐 External - external → backend · Validation: The validation leg assumes every request body is schema-checked before business logic, yet the generated User resource binds an unlisted privileged attribute.

Instances (2): `server.ts:484` , `routes/verify.ts:53`

Issue: An unauthenticated attacker posts to `/api/Users` with an extra role field set to admin. The finale resource generated for the User model excludes only password and `totpSecret` from writable attributes, so role is bound straight from the request body into the created row.

The middleware chain reaching that resource, through `server.ts:422` , only trims fields and records challenge state, and `routes/verify.ts:53` confirms the server observes `req.body.role` arriving with the admin value. Any internet attacker self-provisions an administrator account, reaching all training account credentials, payment card test data, and personal data submitted during challenges.

Evidence: ✓ verified - `server.ts:484` declares `exclude: ['password', 'totpSecret']` for the auto-generated User resource, leaving `role` writable, and `routes/verify.ts:53` reads `req.body.role === roles.admin` on the same registration path.

```
// server.ts:484
finale.initialize({ app, sequelize: seq })

const autoModels = [
  { name: 'User', exclude: ['password', 'totpSecret'], model: UserModel },
  { name: 'Product', exclude: [], model: ProductModel },
  { name: 'Feedback', exclude: [], model: FeedbackModel },
  { name: 'BasketItem', exclude: [], model: BasketItemModel },
```

Fix: ● [M-022](#) — Allowlist client-controlled fields (`server.ts:484`)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · [CWE-915](#) · [OWASP A01:2025](#) · walkthrough [Walkthrough §3.3](#)

F-001 · Cross-Site Scripting

Severity: ● Critical · **Component:** [C-03](#) - Angular SPA Web Frontend · **Location:** Multiple locations (4)

Instances (4): ● `frontend/src/app/search-result/search-result.component.ts:143` , ● `frontend/src/app/search-result/search-result.component.ts:110` , ● `frontend/src/app/administration/administration.component.ts:73` , ● `frontend/src/app/about/about.component.ts:119`

Issue: An attacker sends a victim a link of the form `/#/search?q=<img src=x onerror=...>`; `filterTable` reads the `q` query parameter straight from the route snapshot, wraps it in `bypassSecurityTrustHtml()`, and the template renders it as trusted markup - the attacker's script executes in the victim's origin and reads the JWT that `request.interceptor.ts` keeps in `localStorage`. A single link yields full account takeover of any signed-in customer: the stolen bearer token authorises order history, saved addresses and payment card endpoints for the lifetime of the token.

Evidence: ✓ verified - `search-result.component.ts:136` reads `queryParams` from `this.route.snapshot.queryParams.q`; line 143 assigns `this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam)`, marking the URL-controlled value as trusted HTML without sanitization. `request.interceptor.ts:13` shows the JWT is readable from the same document via `localStorage`.

```
// frontend/src/app/search-result/search-result.component.ts:143
    this.io.socket().emit('verifyLocalXssChallenge', queryParam)
  } // vuln-code-snippet hide-end
  this.dataSource.filter = queryParam.toLowerCase()
  this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam) // vuln-code-snippet vuln-
line localXssChallenge xssBonusChallenge
  if (this.gridDataSourceSubscription) {
    this.gridDataSourceSubscription.unsubscribe()
  }
```

Fix: Output-encode untrusted strings at every sink and remove all `bypassSecurityTrustHtml` calls → ● [M-011](#) — Encode output instead of bypassing the framework sanitizer (`search-result.component.ts:143`)

Classification: Cross-Site Scripting (XSS) · STRIDE: Tampering · [CWE-79](#) · [OWASP A05:2025](#) · walkthrough [Walkthrough §3.5](#)

● **High (32)**

F-018 · Hard-coded seeded account credentials (data/datacreator.ts:200)

Severity: ● High · **Component:** C-01 - Express.js Backend API · **Location:** data/datacreator.ts:200

Weakness: W-004 - Secrets are committed to source instead of a managed store

Issue: An attacker reads the repository-tracked static user data that `createUsers` loads, then authenticates to a deployed instance as any seeded account, including the admin role account, and supplies a valid TOTP code because the shared secret persisted into the user row comes from that same tracked file. Full administrative impersonation of the authoritative data store's owner account, exposing the user credential table, payment card test data, and delivery addresses named as sensitive assets for this component.

Evidence: ○ ambiguous - `createUsers` destructures `password`, `role`, and `totpSecret` from `loadStaticUserData()` at `data/datacreator.ts:190` and writes them verbatim into `UserModel.create` at lines 193-200; line 199 shows `role` is compared against `security.roles.admin`, confirming a privileged account is among the seeded set.

```
// data/datacreator.ts:200
    deluxeToken: role === security.roles.deluxe ? security.deluxeToken(completeEmail) : '',
    profileImage: `assets/public/images/uploads/${profileImage ?? (role === security.roles.admin ?
'defaultAdmin.png' : 'default.svg')}`,
    totpSecret,
    lastLoginIp
  })
```

Fix: Move the credential out of source control into a secret store and rotate it → ● M-002 — Move secrets to a managed secret store (`datacreator.ts:200`) · ● M-028 — Move secrets to a managed secret store (`datacreator.ts:200`)

Classification: Cryptographic Failures · STRIDE: Spoofing · CWE-798 · OWASP A04:2025

F-031 · Hard-coded wallet mnemonic (routes/checkKeys.ts:10)

Severity: ● High · **Component:** C-07 - Web3 / Wallet / NFT Surface · **Location:** routes/checkKeys.ts:10

Weakness: W-004 - Secrets are committed to source instead of a managed store

Issue: Anyone who reads the repository copies the BIP39 mnemonic literal, runs the same `HDNodeWallet.fromPhrase` derivation used on line 11, and obtains the application wallet's private key. The attacker can then sign transactions as that wallet on any EVM chain and satisfy the endpoint's own key check at line 18 without ever compromising the running service.

The private key of the wallet identity this component speaks for is public, so its on-chain identity can be impersonated and any value at the derived addresses is spendable. The admitted architecture assumption scopes the wallet to demonstration keys on Sepolia, which limits exposure to the demo wallet rather than production funds, but the same mnemonic derives usable addresses on every EVM chain.

Evidence: ✓ verified - `routes/checkKeys.ts:10` assigns a twelve-word BIP39 mnemonic as a string literal in source, and line 11 derives the wallet, private key, public key, and address from it; line 18 then treats possession of that private key as proof of wallet ownership.

```
// routes/checkKeys.ts:10
try {
  const { HDNodeWallet } = await import('ethers')
  const mnemonic = 'purpose betray marriage blame crunch monitor spin slide donate sport lift clutch'

  const mnemonicWallet = HDNodeWallet.fromPhrase(mnemonic)
  const privateKey = mnemonicWallet.privateKey
```

Fix: Move the credential out of source control into a secret store and rotate it →  [M-041](#) — Move secrets to a managed secret store (`checkKeys.ts:10`)

Classification: Cryptographic Failures · STRIDE: Information Disclosure · [CWE-798](#) · [OWASP A04:2025](#)

F-003 · Client-side authorization decision (`app.guard.ts:54`)

Severity:  High · **Component:** [C-03](#) - Angular SPA Web Frontend · **Location:** `frontend/src/app/app.guard.ts:54`

Issue: An attacker edits the JWT in `localStorage` to set `data.role` to `admin`, then navigates to `/administration`. Because `AdminGuard` reads the role from an unverified client-side `jwtDecode` call rather than a server-validated claim, both the administration and accounting views load and issue data requests under the attacker's real identity.

Any user who edits their stored token gains access to the administration and accounting interfaces, exposing the admin UI surface and turning every server endpoint that lacks its own role check into an immediate privilege escalation.

Evidence: ✓ verified - `app.guard.ts:38` decodes the token with `jwtDecode`, which parses the payload without verifying the signature. `app.guard.ts:54` grants the administration route when `payload.data.role` equals `roles.admin`, and `app.routing.ts:81` wires that guard as the only gate on the administration path. `LoginGuard` at `app.guard.ts:18` likewise treats the mere presence of a `localStorage` token as proof of authentication, and `DeLuxeGuard` at line 86 reuses the same unverified claim.

Fix:  [M-013](#) — Enforce authorization on the server (`app.guard.ts:54`)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · [CWE-602](#) · [OWASP A01:2025](#)

F-013 · Weak password recovery mechanism

Severity: ● High · **Component:** C-02 - JWT Authentication and Session Management ·

Location: Multiple locations (2)

Instances (2): `routes/resetPassword.ts:41` , `routes/resetPassword.ts:10`

Issue: An attacker submits a target's email and a guessed or researched security answer to `POST /rest/user/reset-password`. The handler requires only a matching HMAC answer (keyed on the constant at `lib/insecurity.ts:42`) - no reset token, no proof of mailbox control - and overwrites the password immediately, giving the attacker full account access in one request.

Account takeover of any user whose security answer is guessable or publicly researchable, including administrative accounts, with the legitimate owner locked out by the password rewrite.

Evidence: ✓ verified - `routes/resetPassword.ts:41` compares `security.hmac(answer)` to the stored answer and, on match, immediately updates the user password at line 44; nothing in the handler binds the reset to a one-time token, a session, or a delivered channel.

```
// routes/resetPassword.ts:41
    }]
  })
  if ((data != null) && security.hmac(answer) === data.answer) {
    const user = await UserModel.findByPk(data.UserId)
    if (user) {
```

Fix: ● M-023 — Replace security-question reset in `routes/resetPassword.ts` with emailed one-time tokens (`resetPassword.ts:41`)

Classification: Broken Authentication · STRIDE: Spoofing · CWE-640 · OWASP A07:2025

F-014 · Long-lived registry publish credential (ci.yml:327)

Severity: ● High (*raw Critical*) · **Component:** C-05 - CI/CD Pipeline · **Location:** `.github/workflows/ci.yml:327`

Issue: An attacker who obtains the static `DOCKERHUB_TOKEN` secret - through a compromised action in the same workflow, a log leak, or maintainer account takeover - authenticates to Docker Hub as the project and pushes a backdoored image to `bkimminich/juice-shop:latest`. Because the credential is a long-lived password rather than a short-lived OIDC workload identity, the token keeps working until someone notices and rotates it, and nothing in the pull path lets a consumer distinguish the attacker's push from a genuine release.

Any holder of the leaked token publishes images under the project's own Docker Hub identity, and every downstream `docker pull bkimminich/juice-shop` - training environments, CTF platforms, and CI systems worldwide - executes the attacker's code while believing it came from the maintainers.

Evidence: ✓ verified - `.github/workflows/ci.yml:327` authenticates the release push with `password: ${ secrets.DOCKERHUB_TOKEN }`, a persistent registry credential; the workflow declares no `id-token: write` permission and contains no signing or attestation step, so OIDC Trusted Publishing is not in use.

```
// .github/workflows/ci.yml:327
with:
  username: ${ secrets.DOCKERHUB_USERNAME }
  password: ${ secrets.DOCKERHUB_TOKEN }
- name: "Set tag & labels for ${ github.ref }"
  run: |
```

Fix: ● [M-024](#) — Use workload identity for package publishing (`ci.yml:327`)

Classification: Broken Authentication · STRIDE: Spoofing · [CWE-522](#) · [OWASP A07:2025](#)

F-015 · Unvalidated OAuth token (oauth.component.ts:28)

Severity: ● High · **Component:** [C-03](#) - Angular SPA Web Frontend · **Location:** `frontend/src/app/oauth/oauth.component.ts:28`

Issue: An attacker lures a victim to a crafted URL carrying `#access_token=<attacker's own Google token>`. The SPA matches the fragment, parses the token with no state or nonce comparison, and submits it as the victim's login.

The victim's browser is silently bound to the attacker-controlled account, so subsequent orders and payment data are recorded under the attacker's identity. A victim silently logged into an attacker-controlled shop account has any basket contents, delivery addresses, and order history they enter afterwards stored in that account and visible to the attacker.

Evidence: ✓ verified - `oauth.component.ts:28` passes `parseRedirectUrlParams().access_token` straight into `userService.oauthLogin`; `parseRedirectUrlParams` (line 71) reads the raw fragment supplied by `app.routing.ts:257` and splits it on `&` and `=` only. `login.component.ts:148` builds the authorization request with `client_id`, `response_type=token`, `scope` and `redirect_uri` and emits no state or nonce, so no value exists that the callback could compare against.

```
// frontend/src/app/oauth/oauth.component.ts:28

ngOnInit (): void {
  this.userService.oauthLogin(this.parseRedirectUrlParams().access_token).subscribe({
    next: (profile: any) => {
      const password = btoa(profile.email.split('').reverse().join(''))
    }
  })
}
```

Fix: ● [M-025](#) — Bind the OAuth callback in `oauth.component.ts` to a generated state value (`oauth.component.ts:28`)

Classification: Broken Authentication · STRIDE: Spoofing · [CWE-290](#) · [OWASP A07:2025](#)

F-016 · Predictable derived credential (oauth.component.ts:30)

Severity: ● High · **Component:** C-03 - Angular SPA Web Frontend · **Location:** frontend/src/app/oauth/oauth.component.ts:30

Issue: An attacker who knows a victim's Google account address computes that victim's Juice Shop password as `base64(reverse(email))` - the OAuth callback derives the local account password deterministically from the email claim. The attacker then authenticates against the normal login endpoint without ever touching Google.

Google OAuth accounts have passwords derived from the account's public email address; any attacker who knows a victim's email can compute the password and gain full access to their order history, saved addresses, payment cards, and basket.

Evidence: ✓ verified - `oauth.component.ts:30` builds the password as `btoa(profile.email.split('').reverse().join(''))`; line 31 registers the account with that value via `userService.save`, and line 46 re-derives the identical string to call `userService.login`, so the same public-identifier-derived secret is both created and accepted as an authentication credential.

```
// frontend/src/app/oauth/oauth.component.ts:30
this.userService.oauthLogin(this.parseRedirectUrlParams().access_token).subscribe({
  next: (profile: any) => {
    const password = btoa(profile.email.split('').reverse().join(''))
    this.userService.save({ email: profile.email, password, passwordRepeat: password }).subscribe({

      next: () => {
```

Fix: ● M-026 — Replace the e-mail-derived OAuth password (`oauth.component.ts:30`)

Classification: OAuth / OIDC Misconfiguration · STRIDE: Spoofing · [CWE-1391](#) · [OWASP A07:2025](#)

F-017 · Unauthenticated WebSocket Channel

Severity: ● High · **Component:** C-06 - Real-time WebSocket Channel · **Location:** Multiple locations (2)

Weakness: W-005 - Endpoints are reachable without enforced authentication

Trust boundary gap: [tb-1](#) 🌐 External - external → backend · Authentication: The WebSocket transport crosses the external-to-backend boundary without the `security.isAuthenticated()` `expressJwt` check that the boundary assumption relies on for every state-changing entry point.

Instances (2): ● `lib/startup/registerWebsocketEvents.ts:23` , ● `lib/startup/registerWebsocketEvents.ts:36`

Issue: An anonymous internet client opens a Socket.IO connection to port 3000 and the connection handler attaches every event listener without verifying a JWT, cookie, or any handshake credential, so the attacker receives the queued push stream and can emit every registered state-changing event without presenting an identity. Any unauthenticated party on the internet obtains a live push channel that leaks solved-challenge notifications including CTF flags and accepts four state-changing event handlers, defeating the stated component role of gating event handling behind JWT verification.

Evidence: ✓ verified - `lib/startup/registerWebsocketEvents.ts:23` registers `io.on('connection', ...)` with no handshake verification and no `io.use()` middleware anywhere in the 56-line file; the imported security module is used only for `isRedirectAllowed` at line 46. The Socket.IO server is constructed on the raw HTTP server at line 20, so the Express `expressJwt` chain never runs for this transport.

```
// lib/startup/registerWebsocketEvents.ts:23
globalWithSocketIO.io = io

io.on('connection', (socket: any) => {
  if (firstConnectedSocket === null) {
    socket.emit('server started')
  }
})
```

Fix: ● [M-027](#) — Require authentication on every exposed endpoint
(`registerWebsocketEvents.ts:23`)

Classification: Unauthenticated Management Plane · STRIDE: Spoofing · [CWE-306](#) · [OWASP A01:2025](#)

F-019 · NoSQL \$where JavaScript injection

Severity: ● High · **Component:** [C-01](#) - Express.js Backend API · **Location:** Multiple locations (2)

Instances (2): `routes/trackOrder.ts:18` , `routes/showProductReviews.ts:36`

Issue: An unauthenticated attacker calls `GET /rest/track-order/:id` with a path segment that closes the quoted comparison and appends a JavaScript predicate. Because `routes/trackOrder.ts:18` concatenates the `id` into a `$where` expression the document store evaluates as JavaScript, an always-true predicate returns every order document, exposing other customers' order identifiers, totals, and masked emails.

An unauthenticated attacker enumerates all orders in the store, disclosing other customers' order history and purchase totals submitted during training exercises.

Evidence: ✓ verified - `routes/trackOrder.ts:18` builds the `$where` expression by string concatenation of the `id` derived from `req.params.id` at line 15; the sanitizing branch at line 15 is skipped whenever the `reflectedXssChallenge` flag is enabled, leaving only a 60-character truncation.

```
// routes/trackOrder.ts:18

challengeUtils.solveIf(challenges.reflectedXssChallenge, () => { return utils.contains(id, '<iframe
src="javascript:alert(`xss`)">') })
db.ordersCollection.find({ $where: `this.orderId === '${id}'` }).then((order: any) => {
  const result = utils.queryResultToJson(order)
  challengeUtils.solveIf(challenges.noSqlOrdersChallenge, () => { return result.data.length > 1 })
})
```

Fix: Replace string concatenation in query operators with parameter binding →  [M-029](#) — Use parameterized database queries (`trackOrder.ts:18`)

Classification: Injection · STRIDE: Tampering · [CWE-943](#) · [OWASP A05:2025](#)

F-020 · Non-cryptographic RNG for a secret/token (`lib/insecurity.ts:53`)

Severity:  High · **Component:** [C-02](#) - JWT Authentication and Session Management ·

Location: `lib/insecurity.ts:53`

Weakness: [W-008](#) - Security-sensitive data uses weak cryptographic primitives

Issue: A predictable token/secret lets an attacker guess or brute-force session identifiers, reset links, or OTPs.

Evidence: ✓ verified

```
// lib/insecurity.ts:53

export const isAuthorized = () => expressJwt(({ secret: publicKey }) as any)
export const denyAll = () => expressJwt({ secret: '' + Math.random() } as any)
export const authorize = (user = {}) => jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256'
})

export const verify = (token: string) => token ? (jws.verify as ((token: string, secret: string) => boole
an))(token, publicKey) : false
```

Fix: Switch to a cryptographically secure RNG (`crypto.randomBytes` / `OS /dev/urandom`) →  [M-030](#) — Use cryptographically secure random values (`insecurity.ts:53`)

Classification: Cryptographic Failures · STRIDE: Tampering · [CWE-330](#) · [OWASP A04:2025](#)

F-021 · Input compiled as template source (`routes/userProfile.ts:87`)

Severity:  High · **Component:** [C-01](#) - Express.js Backend API · **Location:** `routes/`
`userProfile.ts:87`

Weakness: [W-010](#) - Injection is implemented inconsistently

Issue: Persisted record read reaches an executable expression. Observed path conditions: if (username) { . The unsafe path is observed; attacker control, access prerequisites, and active configuration require verification.

Evidence:  ambiguous

```
// routes/userProfile.ts:87
try {
  const pug = (await import('pug')).default
  const fn = pug.compile(template)
  const CSP = `img-src 'self' ${user?.profileImage}; script-src 'self' 'unsafe-eval'`
```

Fix:  [M-003](#) — Remove server-side evaluation of untrusted input (`userProfile.ts:87`) · 
[M-031](#) — Remove server-side evaluation of untrusted input (`userProfile.ts:87`)

Classification: Injection · STRIDE: Tampering · [CWE-1336](#) · [OWASP A05:2025](#)

F-022 · Unpinned action

Severity:  High · **Component:** [C-05](#) - CI/CD Pipeline · **Location:** `.github/workflows/image_actions.yml:33`

Weakness: [W-003](#) - Build pipeline trusts mutable third-party references

Issue: An attacker who compromises the calibreapp/image-actions repository, or a maintainer who force-pushes it, changes the code at the mutable 'main' branch ref. The next image-compression run resolves that ref fresh and executes the new code inside the repository's CI context with `GITHUB_TOKEN` in hand.

The same substitution works against the other tag-pinned actions, because a Git tag can be moved to any commit by whoever controls the action repository. Malicious code in a substituted action runs with the workflow's `GITHUB_TOKEN` and any secrets exposed to that job, letting the attacker commit to the repository, alter the CodeQL results that gate security review, or pivot into the release jobs that hold registry and deploy credentials.

Evidence:  verified - `.github/workflows/image_actions.yml:33` pins calibreapp/image-actions to the mutable branch ref '@main'; lines 30 and 42 pin actions/checkout and peter-evans/create-pull-request to movable tags '@v6' and '@v8'; `.github/workflows/ci.yml:188` uses coverallsapp/github-action@v2 and `.github/workflows/codeql-analysis.yml:23`, 34 and 36 use `github/codeql-action/* @v3`. The rest of `ci.yml` pins every action to a full 40-character commit SHA (for example line 29 and line 338), so these are gaps in an otherwise applied practice.

```
// .github/workflows/image_actions.yml:33
- name: Compress Images
  id: calibre
  uses: calibreapp/image-actions@main
  with:
    githubToken: ${ secrets.GITHUB_TOKEN }
```

Fix:  [M-032](#) — Set least-privilege CI workflow permissions (`image_actions.yml:33`)

Classification: Supply-Chain Integrity · STRIDE: Tampering · [CWE-1357](#) · [OWASP A03:2025](#)

F-023 · Client-trusted security decision (registerWebsocketEvents.ts:50)

Severity: ● High · **Component:** [C-06](#) - Real-time WebSocket Channel · **Location:** `lib/startup/registerWebsocketEvents.ts:50`

Trust boundary gap: [tb-1](#) 🌐 External - external → backend · Validation: The handler reaches persistence logic with a payload that was never schema-checked, breaking the boundary assumption that request bodies are validated before business logic runs.

Issue: An unauthenticated client emits `verifyCloseNotificationsChallenge` with any two-element array; the handler treats that client-supplied value as proof of completion and calls `challengeUtils.solveIf`, which persists `challenge.solved = true` and broadcasts the challenge notification to every connected socket. Attacker-supplied event payloads drive persisted application state and the anti-cheat and webhook side effects that follow it, so scoreboard and challenge-completion records no longer reflect work actually performed against the application.

Evidence: ✓ verified - `lib/startup/registerWebsocketEvents.ts:50` decides challenge completion solely from the shape of the client-supplied data argument; the sibling handlers at lines 41, 42 and 46 do the same with string matching. Each calls `challengeUtils.solveIf`, which reaches `challengeUtils.solve` at `lib/challengeUtils.ts:30-31` and writes `solved = true` to the database.

```
// lib/startup/registerWebsocketEvents.ts:50

socket.on('verifyCloseNotificationsChallenge', (data: any) => {
  challengeUtils.solveIf(challenges.closeNotificationsChallenge, () => { return Array.isArray(data) &
& data.length > 1 })
})
})
```

Fix: ● [M-033](#) — Use trusted client IPs for rate limiting (`registerWebsocketEvents.ts:50`)

Classification: Insecure Real-Time Channel · STRIDE: Tampering · [CWE-807](#) · [OWASP A01:2025](#)

F-024 · Passwords passed in URL query string (routes/changePassword.ts:14)

Severity: ● High · **Component:** [C-02](#) - JWT Authentication and Session Management · **Location:** `routes/changePassword.ts:14`

Issue: The change-password handler reads the current and the new password from `req.query`, so both secrets travel in the request URL. The URL is recorded by reverse proxies, access logs, browser history and any `Referer` header the page emits, letting an attacker with log or history access read a live account password in cleartext.

Current and replacement passwords are persisted in cleartext in infrastructure logs and browser history, giving anyone with log access working credentials for the account.

Evidence: ✓ verified - `routes/changePassword.ts:14-17` assigns `query.current`, `query.new` and `query.repeat` to the password variables, which means the endpoint is invoked with the credentials embedded in the request line rather than in a request body.

```
// routes/changePassword.ts:14
export function changePassword () {
  return async ({ query, headers, connection }: Request, res: Response, next: NextFunction) => {
    const currentPassword = query.current as string
    const newPassword = query.new as string
    const newPasswordInString = newPassword?.toString()
```

Fix: 🚫 [M-034](#) — Accept change-password secrets in a POST body (`changePassword.ts:14`)

Classification: Cryptographic Failures · STRIDE: Information Disclosure · [CWE-598](#) · [OWASP A04:2025](#)

F-025 · Unsalted MD5 password hashing

Severity: 🔴 High · **Component:** [C-02](#) - JWT Authentication and Session Management · **Location:** Multiple locations (2)

Weakness: [W-008](#) - Security-sensitive data uses weak cryptographic primitives

Instances (2): `lib/insecurity.ts:41`, `data/datacreator.ts:193`

Issue: The hash helper applies a single unsalted MD5 round and is the only transform protecting stored passwords: `routes/login.ts:34` compares MD5 output during login and `routes/2fa.ts:107` reuses it for password re-confirmation before 2FA changes. An attacker who reads the Users table, for example through the SQL injection at `routes/login.ts:34`, recovers plaintext passwords with commodity rainbow tables.

Disclosure of the Users table converts directly into plaintext credentials for every account, enabling credential stuffing against the users' other services.

Evidence: ✓ verified - `lib/insecurity.ts:41` defines hash as `crypto.createHash('md5').update(data).digest('hex')` with no salt, no per-user parameter and no iteration count, and that value is what login compares against the stored password column.

Fix: Replace the broken hash with a salted password-hashing function (bcrypt/Argon2id) → 🚫 [M-035](#) — Hash passwords with a strong, salted algorithm (`insecurity.ts:41`)

Classification: Cryptographic Failures · STRIDE: Information Disclosure · [CWE-916](#) · [OWASP A04:2025](#)

F-026 · Denylisted path traversal in erasure layout (routes/dataErasure.ts:104)

Severity: ● High · **Component:** C-01 - Express.js Backend API · **Location:** routes/dataErasure.ts:104

Weakness: W-007 - Input handling lacks enforced boundary validation

Issue: An authenticated attacker posts a layout field such as ../../../../etc/passwd to POST /dataerasure. routes/dataErasure.ts:104 resolves that value with path.resolve and screens it only against a three-string denylist at line 105 covering ftp, ctf.key, and encryptionkeys.

Any other absolute path passes and is spread into the template context at line 108, so the renderer reads the attacker-chosen file and the first hundred characters are returned at line 115. An authenticated attacker reads arbitrary files the server process can access, disclosing configuration, the SQLite database path, and any secret not covered by the three denylisted substrings.

Evidence: ✓ verified - routes/dataErasure.ts:104 resolves req.body.layout to an absolute path and line 105 applies only a substring denylist; no base-directory containment check or allowlist constrains the resolved path before it reaches res.render at line 107.

```
// routes/dataErasure.ts:104

if (req.body.layout) {
  const filePath: string = path.resolve(req.body.layout).toLowerCase()
  const isForbiddenFile: boolean = (filePath.includes('ftp') || filePath.includes('ctf.key') ||
filePath.includes('encryptionkeys'))
  if (!isForbiddenFile) {
```

Fix: Resolve and normalise every constructed path and reject anything that escapes the intended base directory → ● M-036 — Constrain file paths to a safe base directory (dataErasure.ts:104)

Classification: Insecure File Handling · STRIDE: Information Disclosure · CWE-22 · OWASP A06:2025

F-027 · HTTP access logs browsable without authentication (server.ts:281)

Severity: ● High · **Component:** C-01 - Express.js Backend API · **Location:** server.ts:281

Trust boundary gap: tb-1 🌐 External - external → backend · Authentication: The authentication leg of tb-1 assumes every sensitive route sits behind security.isAuthenticated(), but the log directory listing is registered with no authentication middleware.

Issue: An unauthenticated attacker browses `/support/logs`, which `server.ts:281` backs with a `serve-index` directory listing over the logs directory, and downloads each daily file through the handler. Those files are the `morgan` combined-format access log configured, which records full request lines including query strings, so any token, coupon, continue-code, or reset parameter that ever travelled in a URL is readable by anyone.

An unauthenticated attacker harvests session tokens and sensitive query parameters from historical traffic, allowing impersonation of any user whose credentials or tokens appeared in a URL.

Evidence: ✓ verified - `server.ts:281` mounts `serveIndex` over the logs directory and `server.ts:283` serves individual log files, with no `security.isAuthenticated()` middleware on either registration; `server.ts:338` confirms those files carry `morgan` combined-format request lines.

```
// server.ts:281

/* /logs directory browsing */ // vuln-code-snippet neutral-line accessLogDisclosureChallenge
app.use('/support/logs', serveIndexMiddleware, serveIndex('logs', { icons: true, view:
'details' }))) // vuln-code-snippet vuln-line accessLogDisclosureChallenge
app.use('/support/logs', verify.accessControlChallenges()) // vuln-code-snippet hide-line
app.use('/support/logs/:file', serveLogFiles()) // vuln-code-snippet vuln-line
accessLogDisclosureChallenge
```

Fix: Strip secrets and PII from every log sink and rotate any token that already leaked → [M-037](#) — Require administrator authorization on the `/support/logs` routes (`server.ts:281`)

Classification: Missing Audit Logging & Accountability · STRIDE: Information Disclosure · [CWE-532](#) · [OWASP A09:2025](#)

F-028 · Unvalidated URL fetch in profile image (routes/profileImageUrlUpload.ts:24)

Severity: ● High · **Component:** [C-01](#) - Express.js Backend API · **Location:** `routes/profileImageUrlUpload.ts:24`

Issue: An authenticated attacker posts an `imageUrl` of `http://169.254.169.254/latest/meta-data/` or a private RFC1918 address to POST `/profile/image/url`. `routes/profileImageUrlUpload.ts:24` passes that value straight to `fetch` with no scheme, host, or address-range allowlist, and the response body is streamed to disk under the web-served uploads directory at line 29.

When the `fetch` fails, line 36 stores the raw URL as the user's `profileImage`, which also confirms the target's reachability. An attacker probes the hosting network from the server's position and retrieves internal or cloud-metadata responses, which land in a publicly served uploads directory and pivot outside the intended training boundary.

Evidence: ✓ verified - `routes/profileImageUrlUpload.ts:24` calls `fetch(url)` on `req.body.imageUrl` read at line 19, with no allowlist, scheme check, or DNS-rebinding guard anywhere in the handler.

```
// routes/profileImageUrlUpload.ts:24
if (loggedInUser) {
  try {
    const response = await fetch(url)
    if (!response.ok || !response.body) {
      throw new Error('url returned a non-OK status code or an empty body')
    }
  }
}
```

Fix: Validate the URL scheme + host against an explicit allow-list before issuing outbound requests → ● [M-038](#) — Validate and allowlist outbound request targets (`profileImageUrlUpload.ts:24`)

Classification: Server-Side Request Forgery · STRIDE: Information Disclosure · [CWE-918](#) · [OWASP A01:2025](#)

F-029 · Secrets broadcast to job environment (`ci.yml:253`)

Severity: ● High · **Component:** [C-05](#) - CI/CD Pipeline · **Location:** `.github/workflows/ci.yml:253`

Issue: The e2e-test step exports four repository secrets as environment variables for a third-party action that then installs and starts the full application. Any code running anywhere inside that step - the action itself, a Cypress plugin, or any of the hundreds of npm packages resolved without a lockfile - can read `process.env` and post all four secrets to an attacker-controlled endpoint.

The secrets are scoped to the whole step rather than to the single command that needs each one, so the blast radius of any one compromised package is the entire secret set. Exfiltration of `GITHUB_TOKEN` grants repository write within the run, the Alchemy API key allows blockchain-API abuse billed to the project, the Cypress record key allows tampering with recorded test evidence, and the solutions webhook leaks the challenge-solution notification channel.

Evidence: ✓ verified - `.github/workflows/ci.yml:250-253` sets `SOLUTIONS_WEBHOOK`, `ALCHEMY_API_KEY`, `CYPRESS_RECORD_KEY` and `GITHUB_TOKEN` as env for the `cypress-io/github-action` step, and lines 264-268 repeat the pattern for `macOS`. Line 238 installs the dependency tree with a bare `'npm install'` and line 245 starts the application under that step, so untrusted package code executes with those variables in scope.

Fix: Restrict the response to the minimum fields needed and never echo secrets → ● [M-039](#) — Stop exposing internal information to clients (`ci.yml:253`)

Classification: Error Information Disclosure · STRIDE: Information Disclosure · [CWE-200](#) · [OWASP A02:2025](#)

F-030 · Unauthenticated data exposure (registerWebsocketEvents.ts:30)

Severity: ● High · **Component:** C-06 - Real-time WebSocket Channel · **Location:** lib/startup/registerWebsocketEvents.ts:30

Issue: An anonymous client connects and the server replays the entire process-global notifications array to the new socket; each notification carries the CTF flag string built in lib/challengeUtils.ts:52, so the attacker harvests the flag of every challenge other players have solved without solving any of them. Every CTF flag released by another participant's solve is delivered to any anonymous internet client, defeating the scoring integrity of a CTF deployment, and the operator's showSolvedNotifications setting does not prevent the transmission because it is enforced only in the client.

Evidence: ✓ verified - lib/startup/registerWebsocketEvents.ts:30 emits every queued notification to a socket that has not been authenticated. The notification object built at lib/challengeUtils.ts:62-70 includes flag: utils.ctfFlag(challenge.name), and its hidden field is derived from challenges.showSolvedNotifications, which suppresses client-side rendering only - the flag value is transmitted regardless.

Fix: Restrict the response to the minimum fields needed and never echo secrets → ● M-040 — Stop exposing internal information to clients (registerWebsocketEvents.ts:30)

Classification: Error Information Disclosure · STRIDE: Information Disclosure · CWE-200 · OWASP A02:2025

F-032 · Rate limiter keyed on client-supplied header (server.ts:346)

Severity: ● High · **Component:** C-01 - Express.js Backend API · **Location:** server.ts:346

Issue: An attacker defeats the reset-password rate limiter by varying one request header. The keyGenerator returns headers['X-Forwarded-For'] whenever that header is present, falling back to the socket address only when it is absent.

Because the header is attacker-controlled and nothing validates it against a trusted proxy chain, sending a fresh value per request gives each attempt its own bucket, so the hundred-request window never engages against a single attacker. An attacker performs unlimited security-answer guesses against the password-reset endpoint, and the only rate limit protecting an account-takeover path becomes ineffective.

Evidence: ✓ verified - server.ts:346 returns the raw headers['X-Forwarded-For'] value as the rate-limit key with no trusted-proxy validation, and server.ts:342 enables trust proxy globally without restricting which upstream addresses may set the header.

```
// server.ts:346
windowMs: 5 * 60 * 1000,
max: 100,
keyGenerator ({ headers, ip }: { headers: any, ip: any }) { return headers['X-Forwarded-For'] ?? ip } // vuln-code-snippet vuln-line resetPasswordMortyChallenge
```

```
)))  
// vuln-code-snippet end resetPasswordMortyChallenge
```

Fix: Apply rate limiting and lock-out thresholds on authentication endpoints → 🕒 [M-042](#) — Rate-limit and lock out repeated authentication attempts (`server.ts:346`)

Classification: Broken Authentication · STRIDE: Denial of Service · [CWE-307](#) · [OWASP A07:2025](#)

F-033 · No rate limit or lockout on login (`server.ts:596`)

Severity: 🟠 High · **Component:** [C-01](#) - Express.js Backend API · **Location:** `server.ts:596`

Issue: An attacker runs a credential-stuffing or password-spraying campaign against POST `/rest/user/login`. The route registered carries no rate-limit middleware, and a search of every `rateLimit` registration in `server.ts` finds limiters only on `/rest/user/reset-password` and the three 2FA routes.

No failed-attempt counter or account lockout exists in `routes/login.ts`, so an attacker sustains unlimited guesses while the unsalted MD5 comparison at `routes/login.ts:34` keeps each attempt cheap for the server and expensive for the defender. An attacker brute-forces training account passwords at full speed and can saturate the single Node process with login work, degrading the shared training instance for all users.

Evidence: ✓ verified - `server.ts:596` registers login with only the handler; the complete set of `rateLimit` call sites in `server.ts` is lines 343, 459, 466, and 472, none of which covers `/rest/user/login`, and `routes/login.ts` contains no attempt counter.

```
// server.ts:596  
  
/* Custom Restful API */  
app.post('/rest/user/login', login())  
app.get('/rest/user/change-password', utils.asyncHandler(changePassword()))  
app.post('/rest/user/reset-password', utils.asyncHandler(resetPassword()))
```

Fix: Apply rate limiting and lock-out thresholds on authentication endpoints → 🕒 [M-043](#) — Rate-limit and lock out repeated authentication attempts (`server.ts:596`)

Classification: Broken Authentication · STRIDE: Denial of Service · [CWE-307](#) · [OWASP A07:2025](#)

F-034 · Unbounded LLM consumption on `/rest/chat` (`server.ts:638`)

Severity: 🟠 High · **Component:** [C-01](#) - Express.js Backend API · **Location:** `server.ts:638`

Weakness: [W-006](#) - Denial of Service is implemented inconsistently

Trust boundary gap: [tb-5](#) - backend → external · Egress content: The egress-content leg assumes controlled traffic to the Ollama provider, but an unauthenticated route relays unbounded attacker-sized prompts to it.

Issue: LLM10 - Unbounded Consumption: an unauthenticated attacker posts an oversized messages array to `/rest/chat` in a loop. The route carries no authentication and no rate-limit middleware, and the `streamText` call at `routes/chat.ts:203` sets no `maxOutputTokens` and no per-caller budget.

Only `stopWhen: stepCountIs (10)` at line 209 bounds tool iterations, and `maxRetries` at line 208 multiplies each failed request against the Ollama backend, so a single caller can exhaust the model service and the single Node process. An unauthenticated attacker exhausts the Ollama service and the single backend process, making the training instance unavailable to every other user and driving unbounded inference cost.

Evidence: ✓ verified - `server.ts:638` registers `/rest/chat` with only `utils.asyncHandler` and no authorization or rate limiter, and `routes/chat.ts:203-209` configures `streamText` with a step cap but no token cap, no input-size cap, and no per-caller quota.

```
// server.ts:638

/* Chat API endpoint */
app.post('/rest/chat', utils.asyncHandler(chat()))

/* Web3 API endpoints */
```

Fix: Bound the request rate and the per-request resource budget on this endpoint → 🕒 M-044 — Rate-limit and lock out repeated authentication attempts (`server.ts:638`)

Classification: Denial of Service · STRIDE: Denial of Service · [CWE-770](#) · [OWASP A06:2025](#)

F-035 · Inefficient regex complexity (`registerWebsocketEvents.ts:46`)

Severity: 🟡 High · **Component:** [C-06](#) - Real-time WebSocket Channel · **Location:** `lib/startup/registerWebsocketEvents.ts:46`

Issue: An unauthenticated client emits `verifySvgInjectionChallenge` with a several-hundred-kilobyte string of repeated `'./././.'` sequences that never contains `'/redirect'`; the handler runs a regular expression whose leading greedy `.*` and lazy `[\w/-]*?` backtrack quadratically over the input, blocking the single Node.js event loop that also serves the HTTP API on the same listener.

A single unauthenticated message stalls the Node.js event loop, making the WebSocket channel and the HTTP API that share the port 3000 listener unresponsive for the duration of the scan, and the message can be repeated at will because the channel has no authentication or rate limit.

Evidence: ✓ verified - `lib/startup/registerWebsocketEvents.ts:46` applies `/.../././.[\w/-]?` / redirect \? to=https?: \\/\ /cataas.com \ /cat` / directly to the client-supplied data value with no length, type, or rate check anywhere in the handler or in the connection setup at lines 20-23. The prefix . retries at every offset and the lazy class rescans to end-of-input at each './././.' anchor.`

```
// lib/startup/registerWebsocketEvents.ts:46

socket.on('verifySvgInjectionChallenge', (data: any) => {
    challengeUtils.solveIf(challenges.svgInjectionChallenge, () => { return data?.match(/.*\\.\\.\\/.
\\.\\.\\w/-]?*?\\redirect?to=https?:\\/\\/cataas.com\\/cat.*/) && security.isRedirectAllowed(data) })
})
```

Fix: ● [M-045](#) — Bound socket payload size and rewrite the SVG-challenge regex (registerWebsocketEvents.ts:46)

Classification: Denial of Service · STRIDE: Denial of Service · [CWE-1333](#) · [OWASP A06:2025](#)

F-036 · Password change without current password (routes/changePassword.ts: 39)

Severity: ● High · **Component:** C-02 - JWT Authentication and Session Management · **Location:** routes/changePassword.ts:39

Issue: The current-password comparison is wrapped in a truthiness guard, so a request that simply omits the current parameter skips verification entirely and line 51 writes the new password. Anyone holding a session token - stolen through a shared browser, obtained via the forgery paths in `lib/insecurity.ts`, or left valid because sessions are never revoked - converts temporary token possession into permanent ownership of the account.

Temporary possession of a session token becomes permanent account takeover, and against an administrator session it yields lasting administrative control with the owner locked out.

Evidence: ✓ verified - routes/changePassword.ts:39 reads `if (currentPassword && security.hash(currentPassword) !== loggedInUser.data.password)`, which makes the whole check conditional on the attacker choosing to send the parameter.

```
// routes/changePassword.ts:39
}

if (currentPassword && security.hash(currentPassword) !== loggedInUser.data.password) {
  res.status(401).send(res.__('Current password is not correct.'))
  return
}
```

Fix: 🕒 [M-046](#) — Require the current password unconditionally (`changePassword.ts:39`)

Classification: Broken Authentication · STRIDE: Elevation of Privilege · [CWE-620](#) · [OWASP A07:2025](#)

F-037 · Client-controlled wallet credit amount (routes/wallet.ts:27)

Severity: ● High · **Component:** C-01 - Express.js Backend API · **Location:** routes/wallet.ts:27

Issue: An authenticated user posts an arbitrary balance value together with the id of any card they own to PUT `/rest/wallet/balance`. `routes/wallet.ts:24` confirms only that the card row belongs to the caller; no charge is attempted and no amount ceiling is checked.

`routes/wallet.ts:27` then increments the wallet by the raw `req.body.balance` value, so the caller decides how much store credit to mint and spends it at checkout through the wallet branch at `routes/order.ts:148`. Any authenticated user mints unlimited store credit and obtains goods and deluxe membership without payment, corrupting the order and wallet data the training scenarios depend on.

Evidence: ✓ verified - `routes/wallet.ts:27` passes `req.body.balance` directly to `WalletModel.increment` with no bound, no server-side price source, and no payment capture between the card lookup at line 24 and the credit.

Fix: 🟡 [M-047](#) — Enforce authorization on the server (`wallet.ts:27`)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · [CWE-602](#) · [OWASP A01:2025](#)

F-040 · Sensitive Routes Registered Without Authentication Middleware

Severity: 🟠 High · **Component:** [C-01](#) - Express.js Backend API · **Location:** Multiple locations (18)

Weakness: [W-002](#) - Authorization is implemented route by route

Instances (18): 🟡 `server.ts:633`, 🟠 `server.ts:310`, 🟠 `server.ts:311`, 🟠 `server.ts:408`, 🟠 `server.ts:420`, 🟠 `server.ts:421`, 🟠 `server.ts:422`, 🟠 `server.ts:438` ... (+10 more)

Issue: State-changing operations on sensitive resources MUST require a proven session. A registration line that lacks any auth marker either trusts the URL itself or relies on a downstream check that the static signature cannot prove exists.

Evidence: 🟡 ambiguous

```
// server.ts:310
/* File Upload */
app.post('/file-upload', uploadToMemory.single('file'), ensureFileIsPassed, metrics.observeFileUploadMetricsMiddleware(), checkUploadSize, checkFileType, handleZipFileUpload, handleXmlUpload, handleYamlUpload)
app.post('/profile/image/file', uploadToMemory.single('file'), ensureFileIsPassed, metrics.observeFileUploadMetricsMiddleware(), utils.asyncHandler(profileImageFileUpload()))
app.post('/profile/image/url', uploadToMemory.single('file'), utils.asyncHandler(profileImageUrlUpload()))
app.post('/rest/memories', uploadToDisk.single('image'), ensureFileIsPassed, security.appendUserId(), metrics.observeFileUploadMetricsMiddleware(), utils.asyncHandler(addMemory()))
```

Fix: 🟡 [M-004](#) — Enforce server-side authorization on every endpoint (`server.ts:310`) · 🟡 [M-050](#) — Enforce server-side authorization on every endpoint (`server.ts:310`)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · [CWE-862](#) · [OWASP A01:2025](#)

F-041 · Missing workflow permissions block

Severity: ● High · **Component:** [C-05](#) - CI/CD Pipeline · **Location:** Multiple locations (2)

Instances (2): ● `.github/workflows/image_actions.yml:35` , ● `.github/workflows/ci.yml:190`

Issue: The CI/CD workflow defines eleven jobs and never declares a permissions key, so `GITHUB_TOKEN` in every job carries the repository's default scope, which on legacy-default repositories is read-write across contents, packages, issues and releases. The workflow then hands that ambient token to third-party action code: coverallsapp/github-action, itself pinned to a movable tag, receives it directly.

A single compromised step - a substituted action or a malicious package installed by the unlocked npm install - inherits repository write and can push commits, create releases, or publish packages under the project's identity. Compromise of any single step in the workflow escalates to repository write, letting the attacker commit backdoored source to master, publish a release, or alter the packages the project distributes - all attributed to the project's own automation.

Evidence: ✓ verified - `.github/workflows/ci.yml:190` passes 'github-token: \${ secrets.GITHUB_TOKEN }' to the coverallsapp/github-action@v2 step declared at line 188, and lines 253, 268 and 389 pass the same token to three further third-party steps. A search of `ci.yml` found zero occurrences of a 'permissions:' key, while `codeql-analysis.yml:11-14` shows the project applies least-privilege token scoping elsewhere.

```
// .github/workflows/ci.yml:190
uses: coverallsapp/github-action@v2
with:
  github-token: ${ secrets.GITHUB_TOKEN }
  files: frontend-lcov.info server-lcov.info api-lcov.info
  custom-config-test:
```

Fix: Add explicit server-side authorisation checks on every protected route → ● [M-051](#) —
Apply least-privilege filesystem access (`ci.yml:190`)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · [CWE-284](#) · [OWASP A01:2025](#)

F-073 · Token flow accepts a stolen bearer token without (login.component.ts: 148)

Severity: ● High · **Component:** [C-03](#) - Angular SPA Web Frontend · **Location:** `frontend/src/app/login/login.component.ts:148`

Issue: Exfiltrated token accepted for a new session; absence of token binding / PKCE removes the last server-side revocation opportunity.

Evidence: ✓ verified

```
// frontend/src/app/login/login.component.ts:148

googleLogin () {
  this.windowRefService.nativeWindow.location.replace(`${oauthProviderUrl}?client_id=${this.clientId}&response_type=token&scope=email&redirect_uri=${this.redirectUri}`)
}
}
```

Fix: Strengthen authentication: enforce a vetted JWT verifier with explicit algorithm, MFA where appropriate → 🟡 [M-078](#) — Harden the authentication flow (`login.component.ts:148`)

Classification: Broken Authentication · STRIDE: Spoofing · [CWE-287](#) · [OWASP A07:2025](#)

F-002 · JWT in localStorage (request.interceptor.ts:13)

Severity: 🟡 High · **Component:** [C-03](#) - Angular SPA Web Frontend · **Location:** `frontend/src/app/Services/request.interceptor.ts:13`

Issue: An attacker who gains script execution in the shop origin through any sanitizer-bypass sink reads the session JWT from `localStorage` - the interceptor stores the bearer token where every same-origin script can reach it - then replays it on each API call. Any script execution in the page yields a replayable session token plus the account e-mail address, granting the attacker the victim's full API authority until the token expires.

Evidence: ✓ verified - `request.interceptor.ts:13` reads `localStorage.getItem('token')` and line 16 attaches it as the `Authorization` bearer header on every outgoing request; line 20 does the same for the user's e-mail address. `app.guard.ts:18` and `basket.service.ts:64` read the same key, confirming the token lives in script-readable storage throughout the app.

Fix: 🟡 [M-012](#) — Store session tokens in HttpOnly, Secure cookies (`request.interceptor.ts:13`)

Classification: Insecure Client-Side Storage · STRIDE: Information Disclosure · [CWE-922](#) · [OWASP A04:2025](#)

F-038 · Coupon discount bounded only by prompt text (routes/chat.ts:184)

Severity: 🟡 High · **Component:** [C-01](#) - Express.js Backend API · **Location:** `routes/chat.ts:184`

Weakness: [W-002](#) - `Authorization` is implemented route by route

Trust boundary gap: [tb-5](#) - backend → external · Egress content: The egress-content leg assumes nothing attacker-controlled reaches the model unfiltered, yet raw user messages steer a tool that mints redeemable coupons.

Issue: LLM06 - Excessive Agency: an attacker injects instructions into the chat messages array so the model calls `generateCoupon` with a discount far above the stated maximum. The coupon policy, including the 10 percent ceiling and the damaged-order precondition, exists only as system-prompt text at `routes/chat.ts:98-105`.

The tool's execute body at `routes/chat.ts:184` passes the model-chosen discount straight to `security.generateCoupon` with no numeric bound, no order verification, and no caller check, and the resulting code is redeemable through `PUT /rest/basket/:id/coupon/:coupon` at `server.ts:605`. An attacker mints arbitrarily large discount coupons through the chat agent and redeems them at checkout - an economic bypass outside the defined challenge scope.

Evidence: ✓ verified - `routes/chat.ts:184` passes the model-supplied value directly to `security.generateCoupon(discount)`. The `zod` schema at `routes/chat.ts:179` declares the maximum only in a describe string, imposing no runtime constraint.

```
// routes/chat.ts:184
    challengeUtils.solveIf(challenges.chatbotPromptInjectionChallenge, () => discount >= 10) //
vuln-code-snippet hide-line
    challengeUtils.solveIf(challenges.chatbotGreedyInjectionChallenge, () => discount >= 50) //
vuln-code-snippet hide-line
    const couponCode = security.generateCoupon(discount) // vuln-code-snippet vuln-line
chatbotPromptInjectionChallenge
    return { couponCode, discount } // vuln-code-snippet neutral-line
chatbotPromptInjectionChallenge
  }
```

Fix: 🟡 M-048 — Enforce correct server-side authorization (`chat.ts:184`)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · [CWE-863](#) · [OWASP A01:2025](#) · walkthrough [Walkthrough §3.8](#)

F-039 · Deluxe upgrade skips payment check (`routes/deluxe.ts:43`)

Severity: 🟡 High · **Component:** C-01 - Express.js Backend API · **Location:** `routes/deluxe.ts:43`

Weakness: W-002 - `Authorization` is implemented route by route

Issue: An authenticated customer posts to `/rest/deluxe-membership` with a `paymentMode` value that is neither `wallet` nor `card` - for example, the empty string. Because `routes/deluxe.ts:24` and `routes/deluxe.ts:34` each guard their payment branch on an exact string match, both branches are skipped and control falls through to the role write at `routes/deluxe.ts:43`, which grants the deluxe role and mints a `deluxeToken`.

No default branch rejects an unrecognized payment mode. Any authenticated customer escalates to the deluxe role without payment, bypassing the per-product purchase limits enforced at `routes/basketItems.ts:92` and the deluxe-only surfaces.

Evidence: ✓ verified - `routes/deluxe.ts:24` and `:34` are independent `if` statements with no `else` and no default rejection, so `routes/deluxe.ts:43` updates the user role unconditionally once the customer lookup at line 19 succeeds.

```
// routes/deluxe.ts:43

    try {
      const updatedUser = await user.update({ role: security.roles.deluxe, deluxeToken: security.deluxeToken(user.email) })
      challengeUtils.solveIf(challenges.freeDeluxeChallenge, () => {
        return security.verify(utils.jwtFrom(req)) && req.body.paymentMode !== 'wallet' && req.body.paymentMode !== 'card'
      })
    }
```

Fix: 🟡 [M-049](#) — Enforce correct server-side authorization (`deluxe.ts:43`)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · [CWE-863](#) · [OWASP A01:2025](#)

🟡 Medium (28)

F-056 · Hard-coded test credential (`login.component.ts:62`)

Severity: 🟡 Medium · **Component:** [C-03](#) - Angular SPA Web Frontend · **Location:** `frontend/src/app/login/login.component.ts:62`

Weakness: [W-004](#) - Secrets are committed to source instead of a managed store

Issue: An attacker downloads the public JavaScript bundle and reads a complete account credential pair out of the login component, because the username and password for the testing account are declared as literal public fields and are therefore compiled into every browser-delivered build. A working credential pair is published to every visitor; if the account exists in the deployed environment, an attacker authenticates with those credentials directly, and in either case the hardcoded literal reveals the internal account-naming convention.

Evidence: ✓ verified - `login.component.ts:61` declares `testingUsername = 'testing@juice-sh.op'` and line 62 declares `testingPassword = 'IamU**** (17 chars)'` as public class fields of `LoginComponent`. A repository search found no other reference to either field in `frontend/src/app/login`, so the literals are shipped without being consumed; whether the corresponding account exists server-side was not verified from this component.

Fix: Move the credential out of source control into a secret store and rotate it → 🟡 [M-066](#) — Move secrets to a managed secret store (`login.component.ts:62`)

Classification: Cryptographic Failures · STRIDE: Information Disclosure · [CWE-798](#) · [OWASP A04:2025](#)

F-066 · Security answers stored under embedded HMAC key (data/datacreator.ts:691)

Severity: ● Medium · **Component:** C-01 - Express.js Backend API · **Location:** data/datacreator.ts:691

Weakness: W-004 - Secrets are committed to source instead of a managed store

Issue: An attacker who reads the stored SecurityAnswer digests recomputes candidate answers offline using the HMAC key embedded in the repository, then drives the password recovery flow to take over the matching account including the administrative one. Account takeover through the password recovery path for any user whose security answer is stored, defeating the recovery control that guards the credentials and payment card data held in this store.

Evidence: ○ ambiguous - data/datacreator.ts:691 persists security answers through SecurityAnswerModel.create, whose setter applies security.hmac at models/securityAnswer.ts:45; that helper is defined at lib/insecurity.ts:42 as an HMAC-SHA256 under the literal key pa4qacea4VK9t9nGv7yZtwmj, so the digest is unsalted, uniterated, and reproducible by anyone with repository access.

Fix: Move the cryptographic key out of source control into a managed secret store and rotate it → ● M-009 — Move cryptographic keys to a managed secret store (datacreator.ts:691) · ● M-076 — Move cryptographic keys to a managed secret store (datacreator.ts:691)

Classification: Cryptographic Failures · STRIDE: Elevation of Privilege · CWE-321 · OWASP A04:2025

F-004 · Missing server-side security audit logging

Severity: ● Medium · **Component:** C-02 - JWT Authentication and Session Management · **Location:** Multiple locations (8)

Instances (8): ● routes/login.ts:50, ● routes/login.ts:34, ● routes/chat.ts:227, ● frontend/src/app/chatbot/chat-conversation/chat-conversation.component.ts:107, ● data/mongodb.ts:10, ● lib/challengeUtils.ts:33, ● data/datacreator.ts:284, ● routes/checkKeys.ts:18

Issue: No handler in the authentication component writes a security event. A failed login returns 401 without recording the attempt, and successful login, password change, password reset and 2FA disable are equally silent.

Credential attacks and token forgery against the authentication boundary cannot be detected or reconstructed, and account actions cannot be attributed to a specific principal after the fact.

Evidence: ✓ verified - A case-insensitive search for logger, `winston`, audit and console logging calls across all nine component files returned zero hits, so the 401 branch at `routes/login.ts:50` and every other authentication outcome in this component produce no audit record.

Fix: 🕒 [M-014](#) — Add security audit logging (`login.ts:50`)

Classification: Missing Audit Logging & Accountability · STRIDE: Repudiation · [CWE-778](#) · [OWASP A09:2025](#)

F-005 · Unmediated data-store access control (`data/mongodb.ts:10`)

Severity: 🟡 Medium · **Component:** [C-08](#) - `marsdb` Embedded Store · **Location:** `data/mongodb.ts:10`

Issue: An attacker who reaches any consumer route backed by the orders collection obtains read and write access to every order document that the route's own query selects, because the collection exported is a bare module-scope singleton that applies no ownership predicate, tenant scope, or authorization hook at the storage layer. `Authorization` for order and review data therefore rests entirely on each importing module repeating its own check, with no store-level backstop when one omits it.

Order and product-review records for all customers are reachable through a single unrestricted handle, so one consumer that omits an ownership check exposes or mutates other customers' order data without any second line of defence.

Evidence: ✓ verified - `data/mongodb.ts:9-10` constructs both MarsDB collections with only a collection-name argument and exports them directly; a full read of the 11-line module plus a zero-hit search for authorization, ownership, tenant, and repository terms shows no mediation layer between importers and the storage engine.

Fix: Add explicit server-side authorisation checks on every protected route → 🕒 [M-015](#) — Apply least-privilege filesystem access (`mongodb.ts:10`)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · [CWE-284](#) · [OWASP A01:2025](#)

F-042 · Missing session token revocation (`lib/insecurity.ts:54`)

Severity: 🟡 Medium · **Component:** [C-02](#) - JWT Authentication and Session Management · **Location:** `lib/insecurity.ts:54`

Issue: `authorize()` issues a six-hour `RS256` token and the session store interface declares only put, get, `tokenOf`, from and `updateFrom`, with no removal operation. A token that an attacker stole or forged stays valid for its whole lifetime, and neither a password change (`routes/changePassword.ts:51`) nor disabling 2FA (`routes/2fa.ts:162`) invalidates sessions that were issued earlier.

A compromised session survives the victim's own remediation: after a password reset the attacker keeps acting as the victim until the six-hour expiry elapses.

Evidence: ✓ verified - `lib/insecurity.ts:54` signs tokens with `expiresIn` six hours; the `IAuthenticatedUsers` interface at lines 31-39 and its implementation at lines 70-91 expose no delete or revoke member, so nothing can retire an issued token before expiry.

Fix: 🕒 [M-052](#) — Add server-side token revocation to the session store (`insecurity.ts:54`)

Classification: Broken Authentication · STRIDE: Spoofing · [CWE-613](#) · [OWASP A07:2025](#)

F-043 · Wallet identity accepted without proof of possession (routes/web3Wallet.ts:16)

Severity: 🟡 Medium · **Component:** [C-07](#) - Web3 / Wallet / NFT Surface · **Location:** `routes/web3Wallet.ts:16`

Trust boundary gap: [tb-1](#) 🌐 External - external → backend · Validation: The boundary assumes every request body is schema-checked before business logic, but this handler writes the raw body field into a security decision set before any validation runs.

Issue: An unauthenticated attacker POSTs to `/rest/web3/walletExploitAddress` with any wallet address, including one belonging to another user. Because `routes/web3Wallet.ts:16` inserts the claimed address into `walletsConnected` before any validation, when the on-chain `ContractExploited` event later reports that address, the handler at lines 26-30 treats the attacker's unproven claim as an identity match and solves `web3WalletChallenge` on their behalf.

Any anonymous caller can register any wallet address without proving private-key ownership. On-chain actions are then attributed to the registrant rather than the key holder, so the component's wallet-ownership verification boundary carries no cryptographic meaning.

Evidence: ✓ verified - `routes/web3Wallet.ts:16` adds `req.body.walletAddress` to the `walletsConnected` trust set outside the `try` block and with no signature, nonce, or session check; lines 27-29 then use membership in that set as the sole authority for attributing an on-chain event to the caller. A grep for signature verification across the component's four route files returned zero hits.

Fix: 🕒 [M-053](#) — Require a signed nonce challenge before registering a wallet address (`web3Wallet.ts:16`)

Classification: Broken Authentication · STRIDE: Spoofing · [CWE-290](#) · [OWASP A07:2025](#)

F-046 · Remote script piped to shell (ci.yml:358)

Severity: 🟡 Medium · **Component:** [C-05](#) - CI/CD Pipeline · **Location:** `.github/workflows/ci.yml:358`

Issue: The heroku job downloads an installer script over the network and pipes it straight into a shell without pinning a version, checking a checksum, or verifying a signature. An attacker who compromises the cli-assets.heroku.com distribution point, or who can intercept the fetch, substitutes the script body and gains arbitrary command execution in a job that already holds the production Heroku API key.

Substituted installer content executes with the `HEROKU_API_KEY` in the environment, allowing the attacker to exfiltrate the production deploy credential and push arbitrary code to the live juice-shop Heroku application.

Evidence: ✓ verified - `.github/workflows/ci.yml:358` runs `'curl https://cli-assets.heroku.com/install.sh | sh'` with no `--fail` flag, no pinned release, and no checksum verification; line 371 shows the same job passes `secrets.HEROKU_API_KEY` to the deploy step, and line 351 shows the job runs on every master and develop push.

Fix: 🕒 [M-056](#) — Verify token signatures before trusting claims (`ci.yml:358`)

Classification: Supply-Chain Integrity · STRIDE: Tampering · [CWE-345](#) · [OWASP A03:2025](#)

F-047 · Unpinned container base image

Severity: 🟡 Medium · **Component:** [C-05](#) - CI/CD Pipeline · **Location:** `Dockerfile:1`

Issue: The build resolves its base images by floating tag rather than by content digest, so the bytes underlying 'node:24' and the untagged distroless runtime image can change between any two builds. An attacker who compromises an upstream base image, or who can influence registry tag resolution, substitutes a backdoored layer that is inherited by the installer stage that compiles the application and by the runtime stage that ships to users, with no `diff` in this repository to signal the change.

A substituted base layer executes inside the build that produces the released image and inside every container started from it, giving the attacker code execution in the shipped artifact; builds are also not reproducible, so a compromised build cannot be distinguished from a clean one after the fact.

Evidence: ✓ verified - `Dockerfile:1` declares 'FROM node:24 AS installer' with a floating major tag and no `@sha256` digest, and `Dockerfile:22` declares 'FROM gcr.io/distroless/nodejs24-debian13' with no tag at all, which resolves to `:latest`. `docker-compose.test.yml:7` pulls 'bkimminich/juice-shop:latest' for the docker-test job invoked at `.github/workflows/ci.yml:311`.

Fix: Replace the unmaintained dependency with a maintained equivalent or fork it under ownership → 🕒 [M-057](#) — Pin the container base image to an immutable digest (`Dockerfile:1`)

Classification: Supply-Chain Integrity · STRIDE: Tampering · [CWE-1104](#) · [OWASP A03:2025](#)

F-048 · Missing Content-Security-Policy

Severity: 🟡 Medium · **Component:** [C-03](#) - Angular SPA Web Frontend · **Location:** -

Issue: An attacker who lands script in any of the eight sanitizer-bypass sinks faces no second line of defence, because the SPA ships no `Content-Security-Policy`, so injected code may load remote scripts and exfiltrate the `localStorage` JWT to an arbitrary host. Every XSS finding in this component escalates from script execution to unrestricted outbound exfiltration of session tokens and customer data, and no violation reporting exists to detect it.

Evidence: 🔍 ambiguous - A repository-relative search for 'Content-Security-Policy' across `frontend/src` returned zero hits, covering `index.html` and every component template, so no policy is declared client-side to constrain script sources or form-action targets.

Fix: Add the missing protection mechanism for this surface (CSP / CSRF token / headers) → 🔍 [M-005](#) — Manual review: verify Missing Content-Security-Policy at the cited location · 🔍 [M-058](#) — Add a script-src and frame-ancestors Content-Security-Policy

Classification: Denial of Service · STRIDE: Tampering · [CWE-693](#) · [OWASP A06:2025](#)

F-049 · Unescaped anchor persisted into product description (data/datacreator.ts:406)

Severity: 🟡 Medium · **Component:** [C-01](#) - Express.js Backend API · **Location:** `data/datacreator.ts:406`

Issue: An attacker who controls the destination page of the seeded product link uses the retained `window.opener` handle to navigate the victim's Juice Shop tab to a credential-harvesting clone, because the anchor persisted into the product description carries `target="_blank"` without `rel="noopener noreferrer"`. Tab-hijacking phishing against authenticated shop users and referrer leakage of the product page URL to the third-party destination, plus stored HTML attribute injection if the configured URL contains a double quote.

Evidence: 🔍 ambiguous - `data/datacreator.ts:406` concatenates `urlForProductTamperingChallenge` into an href attribute without escaping and emits `target="_blank"` with no rel attribute; the resulting description is persisted by `ProductModel.create` at line 426 and rendered unsanitized through `[innerHTML]` at `frontend/src/app/product-details/product-details.component.html:16`. A recon signal independently flags this line as `window-opener-noopener-missing`.

Fix: 🔍 [M-006](#) — Manual review: verify Unescaped anchor persisted into product description (`datacreator.ts:406`) · 🔍 [M-059](#) — Add `rel=noopener noreferrer` and escape the href (`datacreator.ts:406`)

Classification: Cross-Site Scripting (XSS) · STRIDE: Tampering · [CWE-116](#) · [OWASP A05:2025](#)

F-050 · Mutable image tag without build attestation (ci.yml:345)

Severity: 🟡 Medium · **Component:** [C-05](#) - CI/CD Pipeline · **Location:** `.github/workflows/ci.yml:345`

Issue: Every master push overwrites the same `bkimminich/ juice-shop:latest` tag, and the only provenance carried by the image is a set of OCI labels populated from build arguments that the publisher supplies. Nothing records which workflow run produced the image currently behind the tag, and the previous image is silently replaced.

Release publications cannot be attributed to a specific workflow run or source commit, so a malicious or accidental bad release cannot be traced, scoped, or distinguished from the legitimate build it replaced during incident response.

Evidence: ✓ verified - `.github/workflows/ci.yml:345` pushes `'bkimminich/juice-shop:${{ env.DOCKER_TAG }}` where `DOCKER_TAG` is set to the mutable value `'latest'` at line 331; lines 347-348 pass `VCS_REF` and `BUILD_DATE` as build arguments that `Dockerfile:35-36` writes into `org.opencontainers.image.revision` and `.created` labels. The docker job contains no signing or attestation step, and the only pipeline record is a Slack notification.

Fix: 🟡 [M-060](#) — Add security audit logging (`ci.yml:345`)

Classification: Missing Audit Logging & Accountability · STRIDE: Repudiation · [CWE-778](#) · [OWASP A09:2025](#)

F-051 · Credential validity oracle before 2FA (`routes/login.ts:38`)

Severity: 🟡 Medium · **Component:** [C-02](#) - JWT Authentication and Session Management · **Location:** `routes/login.ts:38`

Issue: When the submitted password is correct but the account has a TOTP secret, the login endpoint answers 401 with the body status `totp_token_required` and a `tmpToken`, while wrong credentials produce a plain 401 text response. An attacker distinguishes the two responses and confirms valid email and password pairs at scale without ever passing the second factor, and simultaneously learns which accounts have 2FA enabled.

Credential-stuffing campaigns can validate harvested password lists against this endpoint and select the accounts without a second factor for follow-up takeover.

Evidence: ✓ verified - `routes/login.ts:38-46` returns a JSON body with status `totp_token_required` and a signed `tmpToken` on correct credentials, whereas `routes/login.ts:50` returns the generic invalid-credentials text, making the two outcomes trivially distinguishable.

Fix: 🟡 [M-061](#) — Neutralise the credential-validity oracle (`login.ts:38`)

Classification: Error Information Disclosure · STRIDE: Information Disclosure · [CWE-204](#) · [OWASP A02:2025](#)

F-053 · Admin configuration endpoints exposed unauthenticated (`server.ts:607`)

Severity: 🟡 Medium · **Component:** [C-01](#) - Express.js Backend API · **Location:** `server.ts:607`

Trust boundary gap: [tb-1](#)  External - external → backend · Authentication: The authentication leg of tb-1 assumes management routes sit behind `security.isAuthenticated()`, yet the admin configuration route is registered without it.

Issue: An unauthenticated attacker requests `GET /rest/admin/application-configuration` and `GET /rest/admin/application-version`. Both are registered and `server.ts:607` with only `utils.asyncHandler` and no `security.isAuthenticated()` middleware, and no earlier `app.use` covers the `/rest/admin` prefix.

An unauthenticated attacker reads the deployment's configuration and version, learning which conditional vulnerable code paths are enabled before choosing an exploit.

Evidence: ✓ verified - `server.ts:606` and `server.ts:607` register the two `/rest/admin` routes with no authorization middleware, and the authorization block at `server.ts:355-446` contains no `app.use` entry matching the `/rest/admin` prefix.

Fix: Restrict the response to the minimum fields needed and never echo secrets →  [M-063](#) — Stop exposing internal information to clients (`server.ts:607`)

Classification: Error Information Disclosure · STRIDE: Information Disclosure · [CWE-200](#) · [OWASP A02:2025](#)

F-054 · Development error handler returns stack traces (`server.ts:682`)

Severity:  Medium · **Component:** [C-01](#) - Express.js Backend API · **Location:** `server.ts:682`

Issue: An attacker triggers an unhandled error, for example by posting a malformed body that makes the JSON parser throw, and receives the response produced by the `errorhandler` middleware registered unconditionally. That middleware is the Express development handler and renders the full stack trace, absolute file paths, and the framework version string set, mapping the deployment's internal layout for follow-up attacks.

An unauthenticated attacker learns absolute server paths, dependency versions, and internal call structure, shortening reconnaissance for the injection and traversal paths elsewhere in this component.

Evidence: ✓ verified - `server.ts:682` registers `errorhandler()` as the terminal middleware with no `NODE_ENV` guard or production branch, and `server.ts:730` sets its title to the application name plus the resolved Express version.

Fix: Replace developer error pages with a generic message in production responses →  [M-064](#) — Return generic error messages to clients (`server.ts:682`)

Classification: Error Information Disclosure · STRIDE: Information Disclosure · [CWE-209](#) · [OWASP A02:2025](#)

F-057 · Session cookie without HttpOnly (login.component.ts:104)

Severity: ● Medium · **Component:** C-03 - Angular SPA Web Frontend · **Location:** frontend/src/app/login/login.component.ts:104

Weakness: W-011 - Cross-Site Scripting is implemented inconsistently

Issue: An attacker with script execution in the shop origin reads the session token a second way, because the login flow writes the same JWT into a cookie through the JavaScript cookie API with only an expiry set, leaving `HttpOnly`, `Secure` and `SameSite` unspecified so the value stays script-readable and is attached to cross-site requests. The session token is exposed to script a second time and, without `SameSite`, is also sent on cross-site requests, widening both XSS token theft and cross-site request forgery against authenticated endpoints.

Evidence: ✓ verified - `login.component.ts:104` calls `this.cookieService.put('token', authentication.token, { expires })` with no `httpOnly`, `secure` or `sameSite` option; `oauth.component.ts:50` writes the identical cookie the same way. A cookie written by client JavaScript cannot carry `HttpOnly`, so the token remains readable by any script in the document.

Fix: Set `HttpOnly` on every session cookie → ● M-067 — Set secure session-cookie attributes (`login.component.ts:104`)

Classification: Insecure Client-Side Storage · STRIDE: Information Disclosure · CWE-1004 · OWASP A04:2025

F-058 · Ineffective author email masking in feedback (data/datacreator.ts:578)

Severity: ● Medium · **Component:** C-01 - Express.js Backend API · **Location:** data/datacreator.ts:578

Issue: An unauthenticated attacker reads the public feedback collection and recovers registered user email addresses, because `createFeedback` masks only the first three characters of the author address and leaves the remaining local part and the full domain in the stored comment. Disclosure of registered user email addresses, including the administrative account, to anonymous readers, supplying the identifier needed for targeted credential stuffing and password recovery attacks against this store's accounts.

Evidence: ○ ambiguous - `data/datacreator.ts:578` builds the stored comment as `${comment} (***${author.slice(3)})`, so an address such as `admin@juice-sh.op` is persisted as `***in@juice-sh.op` with the domain and the tail of the local part intact; that comment is written to the Feedback table by `FeedbackModel.create` at line 579.

Fix: ● M-007 — Manual review: verify Ineffective author email masking (`datacreator.ts:578`) · ● M-068 — Mask the full local part and domain of the author address (`datacreator.ts:578`)

Classification: Error Information Disclosure · STRIDE: Information Disclosure · CWE-359 · OWASP A02:2025

F-059 · Plaintext security answer written to log (data/datacreator.ts:692)

Severity: ● Medium · **Component:** C-01 - Express.js Backend API · **Location:** data/datacreator.ts:692

Issue: An attacker with read access to the application log harvests the plaintext security answers that `createSecurityAnswer` interpolates into its error message, then drives the password recovery flow to take over the corresponding accounts. Disclosure of credential-equivalent recovery secrets for named user accounts to anyone holding log read access, enabling account takeover without touching the password column.

Evidence: ○ ambiguous - `data/datacreator.ts:692` interpolates the raw answer parameter into `logger.error`, and the same plaintext reaches this path for the geo-stalking answers passed at lines 349 and 353; the value is a credential equivalent because `models/securityAnswer.ts:45` stores only a keyed digest of it.

Fix: Strip secrets and PII from every log sink and rotate any token that already leaked → ●
M-008 — Manual review: verify Plaintext security answer written to log (`datacreator.ts:692`) ·
● M-069 — Redact the answer value from the `createSecurityAnswer` error log
(`datacreator.ts:692`)

Classification: Missing Audit Logging & Accountability · STRIDE: Information Disclosure ·
CWE-532 · OWASP A09:2025

F-060 · Raw provider error returned to caller (routes/web3Wallet.ts:36)

Severity: ● Medium · **Component:** C-07 - Web3 / Wallet / NFT Surface · **Location:** routes/web3Wallet.ts:36

Issue: An unauthenticated attacker calls POST `/rest/web3/walletExploitAddress` while the Alchemy endpoint is unreachable or rejects the key. The catch block returns `utils.getErrorMessage(error)`, which is the unmodified `Error.message`, so the caller receives the internal failure text produced while constructing the provider whose connection URL at line 20 embeds `ALCHEMY_API_KEY`.

Anonymous callers learn internal failure detail about the blockchain provider integration, and any provider error text that echoes the connection URL discloses the `ALCHEMY_API_KEY` that authorises the application's on-chain reads.

Evidence: ✓ verified - `routes/web3Wallet.ts:36` serialises the caught error directly into the 500 response body, and `lib/utils.ts:210-213` shows `getErrorMessage` performs no redaction - it returns `error.message` for Error instances and `String(error)` otherwise. Line 20 interpolates `process.env.ALCHEMY_API_KEY` into the WebSocket provider URL that the failing call operates on.

Fix: Replace developer error pages with a generic message in production responses → ●
M-070 — Return generic error messages to clients (`web3Wallet.ts:36`)

Classification: Error Information Disclosure · STRIDE: Information Disclosure · [CWE-209](#) · [OWASP A02:2025](#)

F-061 · Unbounded in-memory session map

Severity: ● Medium · **Component:** [C-02](#) - JWT Authentication and Session Management · **Location:** Multiple locations (2)

Weakness: [W-006](#) - Denial of Service is implemented inconsistently

Instances (2): `lib/insecurity.ts:74` , `routes/web3Wallet.ts:16`

Issue: `authenticatedUsers.put` stores every issued token together with the full user record in a plain object that has no size cap, no TTL and no eviction, and `updateAuthenticatedUsers` adds an entry for each distinct token presented in a cookie. An attacker who supplies a stream of distinct verifiable tokens grows the map until the Node heap is exhausted and the process dies, taking all sessions with it.

Sustained token submission exhausts process memory and crashes the single Node backend, making login and all authenticated functionality unavailable.

Evidence: ✓ verified - `lib/insecurity.ts:74` writes `this.tokenMap[token] = user` and `lib/insecurity.ts:191` calls that put for any cookie token that passes verification; neither the interface at lines 31-39 nor the implementation at lines 70-91 removes an entry, and expired tokens are never purged.

Fix: Bound the request rate and the per-request resource budget on this endpoint → ● [M-071](#) — Rate-limit and lock out repeated authentication attempts (`insecurity.ts:74`)

Classification: Denial of Service · STRIDE: Denial of Service · [CWE-770](#) · [OWASP A06:2025](#)

F-062 · Unbounded CI matrix fan-out (ci.yml:58)

Severity: ● Medium · **Component:** [C-05](#) - CI/CD Pipeline · **Location:** `.github/workflows/ci.yml:58`

Weakness: [W-009](#) - Denial of Service is implemented inconsistently

Issue: Three test jobs each fan out across three operating systems and three Node versions, and several further jobs run without any repository guard. The workflow declares no concurrency group, so a superseded run is never cancelled, and no job declares timeout-minutes, so a hung job runs to the platform maximum.

Sustained trigger activity saturates the shared GitHub Actions runner quota and the paid Cypress Cloud recording plan, stalling releases and pull-request feedback for the whole project until quota resets or an administrator intervenes.

Evidence: ✓ verified - `.github/workflows/ci.yml:57-59` declares a 3x3 matrix repeated for the frontend-test, server-test and api-test jobs, and the workflow triggers on all pushes except two branches (lines 3-6) and on every `pull_request` (line 14). A search of `.github/workflows` found zero occurrences of job-level 'timeout-minutes:' and no 'concurrency:' key in `ci.yml`; the 'timeout_minutes' values at lines 77, 115, 155 and 207 are inputs to the nick-invision/retry action and bound only that one retry, not the job. The custom-config-test step at line 207 permits 30 minutes multiplied by three attempts.

Fix: Bound the request rate and the per-request resource budget on this endpoint → [M-072](#)
— Offload CPU-bound work and bound execution time (`ci.yml:58`)

Classification: Denial of Service · STRIDE: Denial of Service · [CWE-400](#) · [OWASP A06:2025](#)

F-064 · Listener guard race opens unbounded providers (routes/nftMint.ts:16)

Severity: ● Medium · **Component:** [C-07](#) - Web3 / Wallet / NFT Surface · **Location:** `routes/nftMint.ts:16`

Weakness: [W-009](#) - Denial of Service is implemented inconsistently

Issue: An unauthenticated attacker sends many concurrent GET `/rest/web3/nftMintListen` requests. Every request evaluates the `isEventListenerCreated` guard before any of them reaches the assignment at line 29, because two awaits sit in between, so each request constructs its own WebSocketProvider and registers another NFTMinted listener, exhausting outbound sockets, provider quota, and heap until the process degrades.

The Node process accumulates unbounded WebSocket connections to the blockchain provider and duplicate contract listeners from anonymous traffic, degrading the service and burning the provider quota the NFT mint features depend on.

Evidence: ✓ verified - `routes/nftMint.ts:16` reads the `isEventListenerCreated` flag that is only set at line 29, with an await on the `ethers` dynamic import at line 17 and provider and contract construction at lines 18 and 23 in between; nothing serialises entry, and no rate limit exists in the component's route files.

Fix: Bound the request rate and the per-request resource budget on this endpoint → [M-074](#)
— Rate-limit expensive requests and bound input size (`nftMint.ts:16`)

Classification: Denial of Service · STRIDE: Denial of Service · [CWE-400](#) · [OWASP A06:2025](#)

F-065 · Basket item update without ownership check

Severity: ● Medium · **Component:** [C-01](#) - Express.js Backend API · **Location:** Multiple locations (2)

Weakness: [W-002](#) - `Authorization` is implemented route by route

Trust boundary gap: [tb-1](#)  External - external → backend · **Authorization:** The authorization leg assumes every object reference is checked against the authenticated subject, but this basket item update resolves the id without an owner predicate.

Instances (2): `server.ts:426` , `routes/nftMint.ts:42`

Issue: An authenticated attacker sends PUT `/api/BasketItems/:id` with another user's basket item id. `server.ts:426` applies `security.appendUserId()` and a quantity check, then hands the request to the auto-generated finale `BasketItem` resource.

An authenticated attacker alters other customers' basket contents and quantities, corrupting their orders and the purchase data used by the training scenarios.

Evidence: ✓ verified - `server.ts:426` chains only `appendUserId` and `quantityCheckBeforeBasketItemUpdate` , and `routes/basketItems.ts:68` resolves the item with `where: { id: req.params.id }` with no `UserId` or `BasketId` constraint on the caller.

Fix: Tie every object lookup to the requesting user's identity and reject cross-tenant references →  [M-075](#) — Enforce object-level (ownership) authorization (`server.ts:426`)

Classification: Broken Access Control · STRIDE: Elevation of Privilege · [CWE-639](#) · [OWASP A01:2025](#)

F-067 · Entitlement token derived from account email (`data/datacreator.ts:198`)

Severity:  Medium · **Component:** [C-01](#) - Express.js Backend API · **Location:** `data/datacreator.ts:198`

Issue: An attacker forges the deluxe entitlement for any account by recomputing the token from the account email alone, because `createUsers` persists a `deluxeToken` that is a deterministic function of a public identifier with no per-grant secret, nonce, or expiry. Vertical privilege gain to deluxe membership on the authoritative data store without payment, and no ability to revoke a granted entitlement because the token cannot be rotated independently of the account email.

Evidence:  ambiguous - `data/datacreator.ts:198` stores `deluxeToken` as `security.deluxeToken(completeEmail)` , and `lib/insecurity.ts:149-151` computes it as an HMAC over email plus the constant deluxe role; the verification at line 167 compares the presented claim against that same recomputation, so the token carries no state beyond the public email.

Fix:  [M-010](#) — Manual review: verify Entitlement token derived from account email (`datacreator.ts:198`) ·  [M-077](#) — Persist a random per-user deluxe entitlement token (`datacreator.ts:198`)

Classification: Cryptographic Failures · STRIDE: Elevation of Privilege · [CWE-1391](#) · [OWASP A04:2025](#)

F-052 · Chat tool calls streamed to every caller (routes/chat.ts:228)

Severity: ● Medium · **Component:** C-01 - Express.js Backend API · **Location:** routes/chat.ts:228

Trust boundary gap: tb-5 - backend → external · Response trust: The response-trust leg assumes LLM response content is handled before forwarding, but raw tool-call events are relayed to any caller unfiltered.

Issue: LLM07 - System Prompt Leakage: an unauthenticated caller posts to /rest/chat and receives every tool-call event verbatim. routes/chat.ts:228 writes the tool name and the full JSON arguments into the SSE stream for all callers; the role and show_tool_calls cookie test sits inside a challenge-detection predicate and gates nothing.

An unauthenticated attacker learns the agent's internal tool names, argument schemas, and the confidential 15 percent escalation discount policy, enabling targeted prompt injection against the coupon path.

Evidence: ✓ verified - routes/chat.ts:228 unconditionally writes the tool name and stringified arguments to the response stream; the role comparison at routes/chat.ts:225 is an argument to challengeUtils.solveIf and never guards the write at line 228.

Fix: Restrict the response to the minimum fields needed and never echo secrets → ● M-062 — Stop exposing internal information to clients (chat.ts:228)

Classification: Error Information Disclosure · STRIDE: Information Disclosure · CWE-200 · OWASP A02:2025

F-055 · Client-gated tool-call disclosure (chat-conversation.component.ts:68)

Severity: ● Medium · **Component:** C-03 - Angular SPA Web Frontend · **Location:** frontend/src/app/chatbot/chat-conversation/chat-conversation.component.ts:68

Issue: LLM07 - System Prompt Leakage: any visitor sets document.cookie = ' show_tool_calls =true' and reloads the chat, because the conversation component decides whether to display the assistant's raw tool calls purely from that client-writable cookie, revealing internal tool names and their argument payloads that the UI otherwise keeps hidden behind an administrator-only toggle. Internal agent tool names and argument payloads are exposed to any unauthenticated visitor, handing an attacker a map of the chatbot's server-side capabilities to target with prompt injection.

Evidence: ✓ verified - chat-conversation.component.ts:68 sets showToolCalls from cookieService.get('show_tool_calls') === 'true', and chat-conversation.component.html:25 renders message.tool_calls whenever that signal is true. The cookie is written from the admin-only screen at administration.component.ts:64 with no server-side authorization, and chat.service.ts:74 shows the tool_calls are already delivered to every client in the stream.

Fix: Restrict the response to the minimum fields needed and never echo secrets → ● M-065 — Stop exposing internal information to clients (chat-conversation.component.ts:68)

Classification: Error Information Disclosure · STRIDE: Information Disclosure · [CWE-200](#) · [OWASP A02:2025](#)

F-063 · Unbounded chat history resend (chat-conversation.component.ts:131)

Severity: ● Medium · **Component:** [C-03](#) - Angular SPA Web Frontend · **Location:** `frontend/src/app/chatbot/chat-conversation/chat-conversation.component.ts:131`

Weakness: [W-009](#) - Denial of Service is implemented inconsistently

Issue: LLM10 - Unbounded Consumption: a user keeps one chat conversation open and sends message after message, and because `sendMessage` rebuilds the request from the entire accumulated message list with no turn or character cap, each request carries a strictly larger prompt, driving token cost and backend latency up without limit. A single user inflates per-request token consumption without bound, raising model cost and starving the shared chat backend of capacity for other shoppers.

Evidence: ✓ verified - `chat-conversation.component.ts:126` maps `this.messages().slice(0, -1)` into `apiMessages` with no length or size bound, and line 131 passes that whole array to `chatService.streamMessages`; the array is additionally rehydrated from browser storage at line 73, so a persisted conversation resumes at its full accumulated size. No `max_tokens`, turn cap or client-side throttle appears anywhere in the component or in `chat.service.ts`.

Fix: Bound the request rate and the per-request resource budget on this endpoint → ● [M-073](#) — Rate-limit expensive requests and bound input size (`chat-conversation.component.ts:131`)

Classification: Denial of Service · STRIDE: Denial of Service · [CWE-400](#) · [OWASP A06:2025](#)

F-044 · Open redirect (routes/redirect.ts:19)

Severity: ● Medium · **Component:** [C-07](#) - Web3 / Wallet / NFT Surface · **Location:** `routes/redirect.ts:19`

Trust boundary gap: [tb-1](#) 🌐 External - external → backend · Validation: The redirect destination crosses the external-to-backend boundary outbound without an exact-match allowlist, so the boundary's validation guarantee does not hold for egress destinations.

Issue: An unauthenticated attacker requests `/redirect?to=https://evil.example/?x=https://github.com/juice-shop/juice-shop`. The allowlist gate accepts the URL because it only requires an allowlisted string to appear somewhere in the destination, and line 19 issues a 302 to the attacker-controlled host, so the victim leaves the application's own domain and lands on a phishing page that inherits the shop's perceived trust.

Visitors are sent from the application's own origin to an attacker-chosen host, enabling credential phishing and the exfiltration of any token carried in a redirect chain; the crypto-asset framing of this component makes wallet-drainer landing pages a realistic destination.

Evidence: ✓ verified - `routes/redirect.ts:19` redirects to the unmodified query parameter. The route's own challenge logic proves the gate at line 16 is not a prefix match: line 18 solves `redirectChallenge` exactly when `isUnintendedRedirect(toUrl)` is true, and lines 27-32 define that as a URL that starts with none of `security.redirectAllowlist` entries. A destination that passes line 16 while matching no allowlist prefix is therefore an expected, reachable state.

Fix: 🕒 [M-054](#) — Validate redirect targets against an allowlist (`redirect.ts:19`)

Classification: Open Redirect · STRIDE: Spoofing · [CWE-601](#) · [OWASP A01:2025](#)

F-045 · Cookie-only authentication on POST /profile (routes/updateUserProfile.ts:17)

Severity: 🟡 Medium · **Component:** [C-01](#) - Express.js Backend API · **Location:** `routes/updateUserProfile.ts:17`

Issue: An attacker hosts a page that auto-submits a form to POST `/profile` with a chosen username field. `routes/updateUserProfile.ts:17` authenticates the request solely from `req.cookies.token`, which `server.ts:183` and the cookie writes at `lib/insecurity.ts:192` leave without `SameSite` or `HttpOnly` attributes, and the handler performs the username write at line 38 with no CSRF token or `origin` check.

An attacker rewrites a victim's profile username without their consent, and can plant the payload that later triggers server-side code execution when the victim opens their profile page.

Evidence: ✓ verified - `routes/updateUserProfile.ts:17` reads the session exclusively from the token cookie and line 38 writes `req.body.username` with no anti-CSRF token; the cookie is set without `SameSite` or `HttpOnly` at `lib/insecurity.ts:192`.

Fix: Enforce a same-origin or signed CSRF token on every state-changing endpoint → 🕒 [M-055](#) — Add anti-CSRF protection to state-changing requests (`updateUserProfile.ts:17`)

Classification: Cross-Site Request Forgery (CSRF) · STRIDE: Tampering · [CWE-352](#) · [OWASP A01:2025](#)

9. Abuse Cases

*Abuse cases describe end-to-end attack scenarios that chain individual findings into an exploitation path. Each case is **mandatory** - defined in the org profile / plugin library and evaluated against every repository. Every chain step references a finding from [§8 Findings Register](#); each step is code-confirmed against the repository and the chain verdict is folded deterministically from the per-step results, never rated by hand.*

#	Scenario	Actor	Combined Risk	Verdict
AC-T-001	Account Takeover via Stored XSS + Token Hijacking	external-attacker	● Critical	▲ Fully viable
AC-T-002	Bulk Data Exfiltration via Broken Object Authorization	authenticated-user	● Critical	⦿ Partially blocked
AC-T-004	Privilege Escalation via Mass-Assignment on Registration	external-attacker	● Critical	▲ Fully viable
AC-T-005	Authentication Bypass via Exposed Secret Material	external-attacker	● High	? Inconclusive
AC-T-006	Remote Code Execution via Server-Side Injection	external-attacker	● Critical	▲ Fully viable

Verdict: ▲ Fully viable - no effective control blocks this chain · ⦿ Partially blocked - at least one step has a compensating control but the chain is not fully closed · ✓ Mitigated - chain is broken at a verified step · ? Inconclusive - could not be verified end-to-end. Step: ✗ Refuted - the matched pairing does not hold on the code.

AC-T-001 — Account Takeover via Stored XSS + Token Hijacking

Source: mandatory · **Actor:** external-attacker - unauthenticated external attacker · **Combined Risk:** ● Critical · **Verdict:** ▲ Fully viable

Goal: Obtain persistent authenticated access as an arbitrary user without valid credentials.

Prerequisite: Attacker can submit content that is later rendered to other users (e.g. feedback, comments, profile fields).

Attack chain

Step	Finding	Outcome
1	● F-001 — Cross-Site Scripting (<code>search-result.component.ts:143</code>)	Attacker JavaScript executes in the victim's browser session.
2	● F-055 — Client-gated tool-call disclosure (<code>chat-conversation.component.ts:68</code>)	Token exfiltrated from local/session storage via the Step 1 payload.
3		Exfiltrated token accepted for a new

Step	Finding	Outcome
	● F-073 — Token flow accepts a stolen bearer token without (<code>login.component.ts:148</code>)	session; absence of token binding / PKCE removes the last server-side revocation opportunity.

Why combined risk exceeds individual ratings

Individually the XSS sink and the web-readable token storage rate below Critical, but chained they form a repeatable credential-theft path: a single stored payload causes indefinite session compromise for every user who views the affected page.

Blocking mitigations

Implementing any single mitigation below severs the chain at the named step, so the end-to-end abuse can no longer complete:

- ● **M-011** — Encode output instead of bypassing the framework sanitizer (**P1**): remediating
- ● **F-001** — Cross-Site Scripting breaks the chain at **Step 1**, removing the link the rest of the chain depends on.
- ① **M-065** — Stop exposing internal information to clients (**P3**): remediating ● **F-055** — Client-gated tool-call disclosure breaks the chain at **Step 2**, removing the link the rest of the chain depends on.
- ② **M-078** — Harden the authentication flow (**P2**): remediating ● **F-073** — Token flow accepts a stolen bearer token without breaks the chain at **Step 3**, removing the link the rest of the chain depends on.

AC-T-002 — Bulk Data Exfiltration via Broken Object Authorization

Source: mandatory · **Actor:** authenticated-user - authenticated low-privilege user ·
Combined Risk: ● Critical · **Verdict:** ① Partially blocked

Goal: Enumerate and exfiltrate other users' records, then escalate own permissions via unguarded mass assignment.

Prerequisite: Attacker holds a valid, non-privileged user account.

Attack chain

Step	Finding	Outcome
1	● F-065 — Basket item update without ownership check (<code>server.ts:426</code>)	Attacker enumerates and retrieves records for arbitrary object IDs; no ownership comparison is performed.
2		

Step	Finding	Outcome
	● F-012 — Mass assignment of role on registration (<code>server.ts:484</code>)	Update endpoint persists an unfiltered <code>role</code> (or equivalent) field supplied in the request body.

Why combined risk exceeds individual ratings

The ownership gap exposes every record, and the mass-assignment gap lets the same low-privilege actor self-elevate - together they turn a single compromised account into full tenant data access and role escalation.

Blocking mitigations

Implementing any single mitigation below severs the chain at the named step, so the end-to-end abuse can no longer complete:

- 🕒 **M-075** — Enforce object-level (ownership) authorization (**P3**): remediating 🟡 **F-065** — Basket item update without ownership check breaks the chain at **Step 1**, removing the link the rest of the chain depends on.
- ● **M-022** — Allowlist client-controlled fields (**P1**): remediating ● **F-012** — Mass assignment of role on registration breaks the chain at **Step 2**, removing the link the rest of the chain depends on.

AC-T-004 — Privilege Escalation via Mass-Assignment on Registration

Source: mandatory · **Actor:** external-attacker - unauthenticated external attacker ·
Combined Risk: ● Critical · **Verdict:** ▲ Fully viable

Goal: Obtain an administrator account without any existing privilege.

Prerequisite: Self-registration is open (one unauthenticated POST).

Attack chain

Step	Finding	Outcome
1	● F-012 — Mass assignment of role on registration (<code>server.ts:484</code>)	The account-creation handler persists the request body wholesale, so a client-supplied <code>role</code> (or <code>isAdmin</code>) field is written verbatim.

Why combined risk exceeds individual ratings

A single unauthenticated request mints an admin account when the registration handler trusts a client-supplied role field - the most direct full-compromise path in role-based apps with open sign-up.

Blocking mitigations

Implementing any single mitigation below severs the chain at the named step, so the end-to-end abuse can no longer complete:

- **M-022** — Allowlist client-controlled fields (**P1**): remediating ● **F-012** — Mass assignment of role on registration breaks the chain at **Step 1**, removing the link the rest of the chain depends on.

AC-T-005 — Authentication Bypass via Exposed Secret Material

Source: mandatory · **Actor:** external-attacker - unauthenticated external attacker ·
Combined Risk: ● High · **Verdict:** ? Inconclusive

Goal: Forge trusted tokens / credentials and impersonate any user.

Prerequisite: Signing material or other secrets are reachable (committed to a public repo, served by an unauthenticated route, or in an exposed directory).

Attack chain

Step	Finding	Outcome
1	● F-031 — Hard-coded wallet mnemonic (<code>checkKeys.ts:10</code>)	A private key, signing secret, or credential file is committed to the source repository or served without authentication.
2	<i>no matching finding</i> <code>lib/insecurity.ts:21</code>	The exposed key/secret is the same one the server trusts, so a token signed with it (or the leaked credential) is accepted as authentic.

Why combined risk exceeds individual ratings

Exposed signing material collapses the entire authentication boundary: any attacker who reads the key can mint a valid token for any identity or role, with no credential ever required.

Blocking mitigations

Implementing any single mitigation below severs the chain at the named step, so the end-to-end abuse can no longer complete:

- ● [M-041](#) — Move secrets to a managed secret store (**P2**): remediating ● [F-031](#) — Hard-coded wallet mnemonic breaks the chain at **Step 1**, removing the link the rest of the chain depends on.

AC-T-006 — Remote Code Execution via Server-Side Injection

Source: mandatory · **Actor:** external-attacker - unauthenticated external attacker ·
Combined Risk: ● Critical · **Verdict:** ▲ Fully viable

Goal: Execute arbitrary code in the application process.

Prerequisite: An input reaches a server-side interpreter / template / eval.

Attack chain

Step	Finding	Outcome
1	● F-011 — Server-side eval of stored username (<code>userProfile.ts:61</code>)	Attacker-controlled input is passed to <code>eval</code> , a server-side template engine, an unsafe sandbox, or an unsafe deserializer.

Why combined risk exceeds individual ratings

A single injection into a server-side interpreter yields code execution in the application process - the highest-impact outcome, granting full filesystem and network access from one unauthenticated request.

Blocking mitigations

Implementing any single mitigation below severs the chain at the named step, so the end-to-end abuse can no longer complete:

- ● [M-021](#) — Remove server-side evaluation of untrusted input (**P1**): remediating ● [F-011](#) — Server-side eval of stored username breaks the chain at **Step 1**, removing the link the rest of the chain depends on.

Generic catalog — evaluated, not applicable

These common abuse-case scenarios from the standard library were checked against this codebase and did not apply. They are listed so the assessment's abuse-case coverage is explicit, not silent.

Scenario	Source	Why not applicable
Privilege Escalation to Admin via JWT Algorithm Confusion	mandatory	no finding matched the required chain step(s) for this scenario
Privileged Action via Prompt Injection into a Tool-Calling Model	mandatory	no finding matched the required chain step(s) for this scenario

10. Mitigation Register

Each mitigation block lists the findings it **Addresses**, the CWEs it **Prevents**, and the **Priority** (P1 = before deployment, P2 = current sprint, P3 = next quarter, P4 = backlog). The **Why / How / Verification** fields are populated only when authored; if a field is omitted, refer to the linked finding's *Evidence* line for file:line context and to the threat-category description in [§8 Findings Register](#) for the underlying weakness.

Mitigations index:

- [M-011](#) — Encode output instead of bypassing the framework sanitizer
- [M-016](#) — Move cryptographic keys to a managed secret store
- [M-017](#) — Enforce JWT signature and algorithm verification
- [M-018](#) — Use parameterized database queries
- [M-019](#) — Enforce object-level (ownership) authorization
- [M-020](#) — Pin third-party dependencies to immutable versions
- [M-021](#) — Remove server-side evaluation of untrusted input
- [M-022](#) — Allowlist client-controlled fields
- [M-013](#) — Enforce authorization on the server
- [M-024](#) — Use workload identity for package publishing
- [M-025](#) — Bind the OAuth callback in `oauth.component.ts` to a generated state value
- [M-026](#) — Replace the e-mail-derived OAuth password
- [M-027](#) — Require authentication on every exposed endpoint
- [M-028](#) — Move secrets to a managed secret store
- [M-029](#) — Use parameterized database queries
- [M-030](#) — Use cryptographically secure random values
- [M-031](#) — Remove server-side evaluation of untrusted input
- [M-032](#) — Set least-privilege CI workflow permissions
- [M-033](#) — Use trusted client IPs for rate limiting
- [M-034](#) — Accept change-password secrets in a POST body
- [M-035](#) — Hash passwords with a strong, salted algorithm
- [M-036](#) — Constrain file paths to a safe base directory
- [M-037](#) — Require administrator authorization on the `/support/logs` routes
- [M-038](#) — Validate and allowlist outbound request targets
- [M-039](#) — Stop exposing internal information to clients
- [M-040](#) — Stop exposing internal information to clients
- [M-041](#) — Move secrets to a managed secret store

- 🕒 M-042 — Rate-limit and lock out repeated authentication attempts
- 🕒 M-043 — Rate-limit and lock out repeated authentication attempts
- 🕒 M-044 — Rate-limit and lock out repeated authentication attempts
- 🕒 M-045 — Bound socket payload size and rewrite the SVG-challenge regex
- 🕒 M-046 — Require the current password unconditionally
- 🕒 M-047 — Enforce authorization on the server
- 🕒 M-048 — Enforce correct server-side authorization
- 🕒 M-049 — Enforce correct server-side authorization
- 🕒 M-050 — Enforce server-side authorization on every endpoint
- 🕒 M-051 — Apply least-privilege filesystem access
- 🕒 M-078 — Harden the authentication flow
- 🕒 M-001 — Enforce object-level (ownership) authorization
- 🕒 M-002 — Move secrets to a managed secret store
- 🕒 M-003 — Remove server-side evaluation of untrusted input
- 🕒 M-004 — Enforce server-side authorization on every endpoint
- 🕒 M-005 — Manual review: verify Missing Content-Security-Policy at the cited...
- 🕒 M-006 — Manual review: verify Unescaped anchor persisted into product...
- 🕒 M-007 — Manual review: verify Ineffective author email masking
- 🕒 M-008 — Manual review: verify Plaintext security answer written to log
- 🕒 M-009 — Move cryptographic keys to a managed secret store
- 🕒 M-010 — Manual review: verify Entitlement token derived from account email
- 🕒 M-012 — Store session tokens in HttpOnly, Secure cookies
- 🕒 M-014 — Add security audit logging
- 🕒 M-015 — Apply least-privilege filesystem access
- 🕒 M-023 — Replace security-question reset in `routes/resetPassword.ts` with...
- 🕒 M-052 — Add server-side token revocation to the session store
- 🕒 M-053 — Require a signed nonce challenge before registering a wallet address
- 🕒 M-054 — Validate redirect targets against an allowlist
- 🕒 M-055 — Add anti-CSRF protection to state-changing requests
- 🕒 M-056 — Verify token signatures before trusting claims
- 🕒 M-057 — Pin the container base image to an immutable digest
- 🕒 M-058 — Add a script-src and frame-ancestors Content-Security-Policy
- 🕒 M-059 — Add rel=noopener noreferrer and escape the href
- 🕒 M-060 — Add security audit logging
- 🕒 M-061 — Neutralise the credential-validity oracle
- 🕒 M-062 — Stop exposing internal information to clients
- 🕒 M-063 — Stop exposing internal information to clients
- 🕒 M-064 — Return generic error messages to clients
- 🕒 M-065 — Stop exposing internal information to clients
- 🕒 M-066 — Move secrets to a managed secret store
- 🕒 M-067 — Set secure session-cookie attributes
- 🕒 M-068 — Mask the full local part and domain of the author address
- 🕒 M-069 — Redact the answer value from the createSecurityAnswer error log
- 🕒 M-070 — Return generic error messages to clients
- 🕒 M-071 — Rate-limit and lock out repeated authentication attempts
- 🕒 M-072 — Offload CPU-bound work and bound execution time
- 🕒 M-073 — Rate-limit expensive requests and bound input size

- 🕒 [M-074](#) — Rate-limit expensive requests and bound input size
- 🕒 [M-075](#) — Enforce object-level (ownership) authorization
- 🕒 [M-076](#) — Move cryptographic keys to a managed secret store
- 🕒 [M-077](#) — Persist a random per-user deluxe entitlement token

P1 — Immediate

M-011 — Encode output instead of bypassing the framework sanitizer

Addresses:

- [F-001](#) — Cross-Site Scripting (`search-result.component.ts:143`)

Prevents CWEs:

- [CWE-79](#) - Cross-site Scripting

Priority: P1 - Immediate · **Effort:** Low · **File:** `frontend/src/app/search-result/search-result.component.ts:143`

How:

1. Delete the `bypassSecurityTrustHtml` call at `search-result.component.ts:143` and assign the raw string to `searchValue` so the template binding escapes it, since the search term needs no markup.
2. If highlighted markup is genuinely required, build it from escaped text with a dedicated pipe rather than trusting the whole parameter.
3. Add a component test that sets `q` to `<img src=x onerror=alert(1)>` and asserts the rendered DOM contains no `img` element.

Example implementation in `frontend/src/app/search-result/search-result.component.ts:143`: it applies **Encode output instead of bypassing the framework sanitizer**. The ordered steps above remain authoritative.

```
// Never call bypassSecurityTrust*; let Angular sanitize.
// template: <div [innerHTML]="product.description"></div>
// component: no DomSanitizer.bypassSecurityTrustHtml(...)
this.product.description = raw // bound directly; Angular escapes
```

Verification: Run the search-result component spec with `q='<img src=x onerror=alert(1)>'` and assert `fixture.nativeElement.querySelector('img')` is `null`.

Reference: [CWE-79](#): Improper Neutralization of Input During Web Page Generation

M-016 — Move cryptographic keys to a managed secret store

Addresses:

- [F-006](#) — Hard-coded JWT signing key (`insecurity.ts:21`)

Weaknesses addressed: [W-004](#)

Prevents CWEs:

- [CWE-321](#) - Use of Hard-coded Cryptographic Key

Priority: P1 - Immediate · **Effort:** Medium · **File:** `lib/insecurity.ts:21`

How:

1. Delete the `privateKey` literal at `lib/insecurity.ts:21` and read the signing key from an injected secret (environment variable or mounted secret file) at process start, failing startup when it is absent.
2. Rotate the key pair and invalidate all tokens signed with the old key by rejecting tokens whose `iat` predates the rotation.
3. Add a CI secret-scanning step (for example `gitleaks`) that fails the build on any `BEGIN RSA PRIVATE KEY` match in tracked source.

*Example implementation in `lib/insecurity.ts:21`: it applies **Move cryptographic keys to a managed secret store**. The ordered steps above remain authoritative.*

```
// Load the RSA private key from an environment variable / KMS – never
// the source tree. Rotate the prior key and revoke outstanding tokens.
const privateKey = process.env.JWT_PRIVATE_KEY
if (!privateKey) throw new Error('JWT_PRIVATE_KEY not set')
const token = jwt.sign(claims, privateKey, { algorithm: 'RS256' })
```

Verification: Run `grep -rn 'BEGIN RSA PRIVATE KEY' lib/ routes/` and expect zero hits; then start the server without the key environment variable set and assert the process exits non-zero instead of serving tokens.

Reference: [CWE-321: Use of Hard-coded Cryptographic Key](#)

M-017 — Enforce JWT signature and algorithm verification

Addresses:

- [F-007](#) — Insecure JWT Verification (`insecurity.ts:52`)

Prevents CWEs:

- [CWE-347](#) - Improper Verification of Cryptographic Signature

Priority: P1 - Immediate · **Effort:** Low · **File:** `lib/insecurity.ts:52`

How:

1. Pass `algorithms: ['RS256']` to the `expressJwt` configuration at `lib/insecurity.ts:52` and to the `jwt.verify` call at `lib/insecurity.ts:189`, and remove the `as any` casts that currently hide the missing option from the type checker.
2. Upgrade `express-jwt` to a version whose types require the `algorithms` option, making omission a compile error.
3. Add a unit test that submits an `HS256` token signed with the public key PEM and asserts a 401 response.

*Example implementation in `lib/insecurity.ts:52`: it applies **Enforce JWT signature and algorithm verification**. The ordered steps above remain authoritative.*

```
export const isAuthorized = () => expressJwt({ secret: publicKey, algorithms: ['RS256'] })
```

Verification: Run the new unit test: an `HS256` token signed with the contents of `encryptionkeys/jwt.pub` must yield 401 on `GET /rest/user/whoami`, while a genuine `RS256` token still yields 200.

Reference: [CWE-347: Improper Verification of Cryptographic Signature](#)

M-018 — Use parameterized database queries

Addresses:

- [F-008](#) — SQL injection in login query (`login.ts:34`)

Weaknesses addressed: [W-001](#)

Prevents CWEs:

- [CWE-89](#) - SQL Injection

Priority: P1 - Immediate · **Effort:** Low · **File:** `routes/login.ts:34`

How:

1. Replace the raw query at `routes/login.ts:34` with `UserModel.findOne({ where: { email, password: hashed, deletedAt: null } })` so Sequelize binds the values, or pass replacements to `sequelize.query` instead of interpolating.
2. Validate the email field against a schema before the lookup.
3. Add a regression test that logs in with the email `' OR 1=1--` and asserts 401.

*Example implementation in `routes/login.ts:34`: it applies **Use parameterized database queries**. The ordered steps above remain authoritative.*

```
// Reject string interpolation; use parameter binding.  
await sequelize.query(  
  `SELECT * FROM users WHERE email = ? AND password = ? AND deletedAt IS NULL`  
  , { replacements: [email, hashed] }  
)
```

```
'SELECT * FROM Users WHERE email = :email AND password = :password',
{ replacements: { email, password: hash(password) }, type: QueryTypes.SELECT }
)
```

Verification: Run the regression test: `POST /rest/user/login` with email `' OR 1=1--` must return 401 and must not include an authentication token in the response body.

Reference: [OWASP Cheat Sheet: SQL Injection Prevention Cheat Sheet](#)

M-019 — Enforce object-level (ownership) authorization

Addresses:

- [F-009](#) — Insecure Direct Object Reference (`address.ts:11`)

Weaknesses addressed: [W-002](#)

Prevents CWEs:

- [CWE-639](#) - `Authorization` Bypass Through User-Controlled Key (IDOR)

Priority: P1 - Immediate · **Effort:** Medium · **File:** `routes/address.ts:11`

How: Replace `req.body.UserId / userId / ownerId` with `req.user.id` (or equivalent session-derived identity) in every WHERE/filter clause.

1. Replace `req.body.UserId/userId/ownerId` with `req.user.id` (or equivalent session-derived identity) in every `WHERE /filter` clause.
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

*Example implementation in `routes/address.ts:11`: it applies **Enforce object-level (ownership) authorization**. The ordered steps above remain authoritative.*

```
// Ownership check before touching a resource.
const basket = await Basket.findByPk(req.params.id)
if (!basket || basket.UserId !== req.user.id) return res.status(403).end()
```

Verification: Re-run the `AUTHZ-001` scanner check; it reports no match at `routes/address.ts:11`, and the regression test passes.

M-020 — Pin third-party dependencies to immutable versions

Addresses:

- [F-010](#) — Dependency lockfile disabled by config (`.npmrc:1`)

Weaknesses addressed: [W-003](#)

Prevents CWEs:

- [CWE-829](#)

Priority: P1 - Immediate · **Effort:** Medium · **File:** `.npmrc:1`

How:

1. Delete the `package-lock=false` line from `.npmrc`, run `npm install` once locally to generate `package-lock.json`, and commit the lockfile for both the root project and `frontend/`.
2. Replace every `npm install` in `.github/workflows/ci.yml` (lines 51, 71, 110, 147, 203, 238) and in `Dockerfile:5` with `npm ci --ignore-scripts`, adding explicit build steps for the few packages that genuinely need a lifecycle script.
3. Add a Dependabot or Renovate configuration with a `minimumReleaseAge` cooldown of at least one day, and add `actions/dependency-review-action` to the `pull_request` path so a malicious or vulnerable manifest change is blocked before merge.
4. Add a CI assertion that fails the build when `package-lock.json` is missing or when `npm ci` reports the lockfile is out of sync with `package.json`.

Verification: Run `git ls-files package-lock.json` and expect the path to be listed, then run `npm ci --ignore-scripts` on a clean checkout; it must succeed, and `grep -r "npm install" .github/workflows/ci.yml` must return no hits.

Reference: [CWE-829: Inclusion of Functionality from Untrusted Control Sphere](#)

M-021 — Remove server-side evaluation of untrusted input

Addresses:

- [F-011](#) — Server-side eval of stored username (`userProfile.ts:61`)

Prevents CWEs:

- [CWE-94](#) - Code Injection

Priority: P1 - Immediate · **Effort:** Low · **File:** `routes/userProfile.ts:61`

How:

1. Delete the `#{...}` extraction and `eval` at `routes/userProfile.ts:54-61` and pass the stored username to `pug` as a plain locals value so the template engine escapes it.
2. Validate the username on write at `routes/updateUserProfile.ts:38` against an allowlist of printable characters excluding `#`, `{`, and `}`.
3. Add an integration test that stores a username of `#{1+1}` and asserts the rendered profile contains the literal text rather than 2.

Example implementation in `routes/userProfile.ts:61` : it applies **Remove server-side evaluation of untrusted input**. The ordered steps above remain authoritative.

```
// Replace `eval()` / sandbox with a strict whitelist parser.
import { evaluate } from 'mathjs'
const allowed = /^[^d+\-*/().\s]+$ /
if (!allowed.test(expr)) throw new Error('invalid expression')
return evaluate(expr)
```

Verification: Run the new integration test: `GET /profile` for a user whose username is `#{1+1}` must render that exact string and must not render `2`.

Reference: [CWE-94](#): Improper Control of Generation of Code (Code Injection)

M-022 — Allowlist client-controlled fields

Addresses:

- [F-012](#) — Mass assignment of role on registration (`server.ts:484`)

Prevents CWEs:

- [CWE-915](#)

Priority: P1 - Immediate · **Effort:** Low · **File:** `server.ts:484`

How:

1. Add `role` and `deluxeToken` to the exclude array at `server.ts:484` so neither attribute is bindable from a request body.
2. Replace the implicit finale binding for registration with an explicit allowlist of `email`, `password`, and `securityAnswer`, and set `role` server-side to the customer default.
3. Add an integration test posting a registration body containing `role: admin` and asserting the created user's role is `customer`.

Example implementation in `server.ts:484` : it applies **Allowlist client-controlled fields**. The ordered steps above remain authoritative.

```
// Whitelist mutable fields server-side; never trust the body.
const ALLOWED = ['email', 'username'] as const
const patch = Object.fromEntries(
  Object.entries(req.body).filter(([k]) => ALLOWED.includes(k as any))
)
await user.update(patch)
```

Verification: Run the new integration test: registering with `role: admin` must produce a stored user whose `role` field reads `customer`.

Reference: [CWE-915](#): Improperly Controlled Modification of Dynamically-Determined Object Attributes

P2 — This Sprint

M-013 — Enforce authorization on the server

Addresses:

- [F-003](#) — Client-side authorization decision (`app.guard.ts:54`)

Prevents CWEs:

- [CWE-602](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `frontend/src/app/app.guard.ts:54`

How:

1. Verify on every administration, accounting, and deluxe API endpoint that the authenticated principal holds the required role server-side, independent of the client route guard.
2. Replace the unverified `jwtDecode` role read at `app.guard.ts:54` with a role supplied by an authenticated server call, so the client never derives authority from a token it cannot validate.
3. Have `LoginGuard` at `app.guard.ts:18` treat a token as a hint only and rely on a 401 from the API to drive logout, rather than on the key being present.
4. Add an integration test that calls each admin API with a non-admin session and asserts HTTP 401 or 403.

Verification: Using a non-admin session token, call `/api/Users` and every admin-only endpoint directly and assert each returns 401 or 403 even when the client route is forced open.

Reference: [CWE-602](#): Client-Side Enforcement of Server-Side Security

M-024 — Use workload identity for package publishing

Addresses:

- [F-014](#) — Long-lived registry publish credential (`ci.yml:327`)

Prevents CWEs:

- [CWE-522](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `.github/workflows/ci.yml:327`

How:

1. Add permissions: `{ id-token: write, contents: read }` to the docker job in `.github/workflows/ci.yml`, switch `docker/login-action` to OIDC-based registry authentication, then delete the `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` repository secrets.
2. Add a `cosign` keyless signing step after `docker/build-push-action` and publish the resulting signature and SLSA provenance attestation alongside the image.
3. Add a release smoke test that runs `cosign verify --certificate-identity-regex` against the freshly pushed tag and fails the job when no valid signature is found.

Verification: Run `cosign verify --certificate-oidc-issuer https://token.actions.githubusercontent.com --certificate-identity-regex "^https://github.com/juice-shop/juice-shop/" bkimminich/juice-shop:latest`; it must exit 0 and print a verified Rekor entry naming the `ci.yml` docker job.

Reference: [OWASP: CICD SEC 06 Insufficient Credential Hygiene](#)

M-025 — Bind the OAuth callback in `oauth.component.ts` to a generated state value

Addresses:

- [F-015](#) — Unvalidated OAuth token (`oauth.component.ts:28`)

Prevents CWEs:

- [CWE-290](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:**

`frontend/src/app/oauth/oauth.component.ts:28`

How:

1. Generate a cryptographically random state (and nonce) before the redirect in `login.component.ts:148`, store it in `sessionStorage`, and include it in the authorization URL.
2. In `oauth.component.ts:28` reject the callback unless the returned state exactly equals the stored value, and clear the stored value after a single use.
3. Migrate off the implicit flow to authorization code with PKCE so no access token ever traverses the URL fragment.
4. Add a test that replays a callback fragment containing an unknown state and asserts `oauthLogin` is never invoked.

Verification: Open the app with a handcrafted `#access_token=X&state=bogus` fragment and assert no request reaches `/rest/user/authentication-details` and the user stays logged out.

Reference: RFC 6749 [§10.12](#) - Cross-Site Request Forgery

M-026 — Replace the e-mail-derived OAuth password

Addresses:

- [F-016](#) — Predictable derived credential (`oauth.component.ts:30`)

Prevents CWEs:

- [CWE-1391](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:**

`frontend/src/app/oauth/oauth.component.ts:30`

How:

1. Stop creating a local password in the client: remove the `btoa(...)` derivation at `oauth.component.ts:30` and :46 and have the backend issue the session directly from the verified OAuth profile, so the browser never synthesises a credential.
2. Mark federated accounts as password-less server-side so the interactive login endpoint rejects password authentication for them.
3. Add a component test asserting that the OAuth callback performs no `userService.save /` login call carrying a password field.

Verification: Register a test user via the Google flow, then POST `/rest/user/login` with `{email, password: btoa(reverse(email))}` and assert the response is 401 rather than a token.

Reference: [CWE-1391](#): Use of Weak Credentials

M-027 — Require authentication on every exposed endpoint

Addresses:

- [F-017](#) — Unauthenticated WebSocket Channel (`registerWebsocketEvents.ts:23`)

Weaknesses addressed: [W-005](#)

Prevents CWEs:

- [CWE-306](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `lib/startup/registerWebsocketEvents.ts:23`

How:

1. Register an `io.use()` handshake middleware before `io.on('connection')` that reads the bearer token from `socket.handshake.auth` and verifies it with the same token-verification routine the Express `expressJwt` middleware uses, calling `next(new Error('unauthorized'))` when verification fails, and attach the resolved subject to `socket.data` for later authorization checks.

2. Add an integration test that opens a socket.io-client connection without a token and asserts a `connect_error` is received rather than a connect event, and a second test that a valid token connects successfully.

Verification: Start the server, then run: `node -e`

```
"const io=require('socket.io-client');const s=io('http://localhost:3000',{transports:
['websocket']});s.on('connect',
()=>{console.log('CONNECTED');process.exit(1)});s.on('connect_error',e=>{console.log('REJ
ECTED '+e.message);process.exit(0)})" - expect 'REJECTED' and exit code 0.
```

Reference: [CWE-306: Missing Authentication for Critical Function](#)

M-028 — Move secrets to a managed secret store

Addresses:

- [F-018](#) — Hard-coded seeded account credentials (`datacreator.ts:200`)

Weaknesses addressed: [W-004](#)

Prevents CWEs:

- [CWE-798](#) - Use of Hard-coded Credentials

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `data/datacreator.ts:200`

How:

1. Replace the password and `totpSecret` values read from static user data in `data/datacreator.ts:190-200` with values resolved from environment variables, generating a random password per account when the variable is unset and printing it once to the startup log.
2. Add a startup assertion that refuses to boot when a seeded account still carries a repository-tracked password and `NODE_ENV` is not a test or training profile.
3. Add a test asserting that UserModel rows created by the seeder do not match any credential literal present in the static data file.

*Example implementation in `data/datacreator.ts:200` : it applies **Move secrets to a managed secret store**. The ordered steps above remain authoritative.*

```
// Read secrets from env / vault, not the source.
const hmacKey = process.env.ORDER_HMAC_KEY
if (!hmacKey) throw new Error('ORDER_HMAC_KEY not set')
const sig = createHmac('sha256', hmacKey).update(payload).digest('hex')
```

Verification: Run the seeder, then execute a login request against the seeded admin email using the password from the static data file and assert the response is HTTP 401 rather than a token.

Reference: [CWE-798](#): Use of Hard-coded Credentials

M-029 — Use parameterized database queries

Addresses:

- [F-019](#) — NoSQL \$where JavaScript injection (`trackOrder.ts:18`)

Prevents CWEs:

- [CWE-943](#) - Special-Element Injection in Data Query

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/trackOrder.ts:18`

How:

1. Change `routes/trackOrder.ts:18` to `ordersCollection.find({ orderId: id })` so the value is matched as data rather than compiled as a JavaScript expression.
2. Validate `req.params.id` against the documented order-id format before the query and reject non-conforming values with HTTP 400.
3. Add an integration test requesting `/rest/track-order/' || '1'=='1` and asserting the response contains at most one order document.

*Example implementation in `routes/trackOrder.ts:18`: it applies **Use parameterized database queries**. The ordered steps above remain authoritative.*

```
// Reject `$where` and operator keys from user-controlled input.
function safeQuery(filter) {
  for (const k of Object.keys(filter)) {
    if (k.startsWith('$')) throw new Error('operator not allowed')
  }
  return collection.find(filter)
}
```

Verification: Run the new integration test: the injected predicate must yield a single synthesized result rather than the full order collection.

Reference: [CWE-943](#): Improper Neutralization of Special Elements in Data Query Logic

M-030 — Use cryptographically secure random values

Addresses:

- [F-020](#) — Non-cryptographic RNG for a secret/token (`insecurity.ts:53`)

Weaknesses addressed: [W-008](#)

Prevents CWEs:

- [CWE-330](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `lib/insecurity.ts:53`

How: Use `crypto.randomBytes` / `crypto.randomUUID` / `webcrypto.getRandomValues` for any security value.

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the CRYPTO-002 scanner check; it reports no match at `lib/insecurity.ts:53`, and the regression test passes.

M-031 — Remove server-side evaluation of untrusted input

Addresses:

- [F-021](#) — Input compiled as template source (`userProfile.ts:87`)

Weaknesses addressed: [W-010](#)

Prevents CWEs:

- [CWE-1336](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/userProfile.ts:87`

How: Keep input as data; remove dynamic code or template-source compilation

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the INJ-NODE-008 scanner check; it reports no match at `routes/userProfile.ts:87`, and the regression test passes.

M-032 — Set least-privilege CI workflow permissions

Addresses:

- [F-022](#) — Unpinned third-party action (`image_actions.yml:33`)

Weaknesses addressed: [W-003](#)

Prevents CWEs:

- [CWE-1357](#)

Priority: P2 - This Sprint · **Effort:** Low · **File:** `.github/workflows/image_actions.yml:33`

How:

1. Replace '@main', '@v6' and '@v8' in `.github/workflows/image_actions.yml` lines 33, 30 and 42, '@v2' in `.github/workflows/ci.yml` line 188, and '@v3' in `.github/workflows/codeql-analysis.yml` lines 23, 34 and 36 with the full 40-character commit SHA of the intended release, keeping the human-readable version in a trailing comment as the file already does elsewhere.
2. Enable the GitHub repository setting that allows only SHA-pinned actions, or add a pinactlint / zizmor step to the `pull_request` path that fails when a 'uses:' value is not a 40-character SHA.

Verification: Run `'grep -rnE "uses:\s+[^ ]+@(v[0-9]|main|master)" .github/workflows/';` it must return no matches.

Reference: [CWE-1357: Reliance on Insufficiently Trustworthy Component](#)

M-033 — Use trusted client IPs for rate limiting

Addresses:

- [F-023](#) — Client-trusted security decision (`registerWebsocketEvents.ts:50`)

Prevents CWEs:

- [CWE-807](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `lib/startup/registerWebsocketEvents.ts:50`

How:

1. Derive challenge completion from server-observed state rather than from the socket payload: record the triggering condition where the server already sees it (the request, response, or stored entity that proves the behaviour) and have the websocket handler only consult that recorded state, rejecting payloads that do not correspond to an authenticated subject's own recorded activity.
2. Add a test that emits `verifyCloseNotificationsChallenge` with [1,2] from a fresh connection and asserts the challenge row remains unsolved.

Verification: Emit the forged event against a running instance, then run: `npx sequelize ? no - query the challenge state via the application's own scoreboard API (GET /api/Challenges/?name= ) and expect solved to remain false.`

Reference: [CWE-807](#)

M-034 — Accept change-password secrets in a POST body

Addresses:

- [F-024](#) — Passwords passed in URL query string (`changePassword.ts:14`)

Prevents CWEs:

- [CWE-598](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/changePassword.ts:14`

How:

1. Change the route to a POST (or PUT) and read current, new and repeat from `req.body` at `routes/changePassword.ts:14` instead of `req.query`.
2. Update the Angular client call site to send the JSON body and remove the query-parameter form.
3. Reject the request when the password fields appear in `req.query`, so a stale client cannot reintroduce the leak.

Verification: Add a test asserting that GET `/rest/user/change-password?new=x & repeat=x` returns 400 or 405 while the POST variant with a JSON body succeeds.

Reference: [CWE-598: Use of GET Request Method With Sensitive Query Strings](#)

M-035 — Hash passwords with a strong, salted algorithm

Addresses:

- [F-025](#) — Unsalted MD5 password hashing (`insecurity.ts:41`)

Weaknesses addressed: [W-008](#)

Prevents CWEs:

- [CWE-916](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `lib/insecurity.ts:41`

How:

1. Replace the `MD5` helper at `lib/insecurity.ts:41` with `argon2id` or `bcrypt` at a tuned cost, storing the full encoded hash including its per-user salt.
2. Verify passwords with the library's constant-time comparison and rehash transparently on successful login so existing `MD5` records migrate as users sign in.
3. Keep a per-record algorithm marker so the login path can distinguish legacy `MD5` rows from migrated ones and force a reset for rows that never migrate.

Example implementation in `lib/insecurity.ts:41`: it applies **Hash passwords with a strong, salted algorithm**. The ordered steps above remain authoritative.

```
// Replace md5/sha1 password hashes with bcrypt or argon2.
import bcrypt from 'bcrypt'
const hash = await bcrypt.hash(plaintext, 12)
const ok = await bcrypt.compare(plaintext, storedHash)
```

Verification: Add a unit test asserting that hashing the same password twice yields different stored values and that verification still succeeds, which fails against the current `MD5` implementation.

Reference: [OWASP Cheat Sheet: Password Storage Cheat Sheet](#)

M-036 — Constrain file paths to a safe base directory

Addresses:

- [F-026](#) — Denylisted path traversal in erasure layout (`dataErasure.ts:104`)

Weaknesses addressed: [W-007](#)

Prevents CWEs:

- [CWE-22](#)

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/dataErasure.ts:104`

How:

1. Replace the denylist at `routes/dataErasure.ts:105` with an allowlist of known layout names, mapping `req.body.layout` to a fixed template path and rejecting any value not in the map.
2. If a path must remain configurable, resolve it and assert the result starts with `path.resolve('views')` plus a separator before calling `res.render` at line 107.
3. Add an integration test posting `layout= ../../../../etc/passwd` and asserting HTTP 400 with no file content in the response body.

Verification: Run the new integration test: the traversal payload must return 400 and the response body must not contain the string `root`✗.

Reference: [CWE-22](#): Improper Limitation of a Pathname to a Restricted Directory

M-037 — Require administrator authorization on the /support/logs routes

Addresses:

- [F-027](#) — HTTP access logs browsable without authentication (`server.ts:281`)

Prevents CWEs:

- [CWE-532](#)

Priority: P2 - This Sprint · **Effort:** Low · **File:** `server.ts:281`

How:

1. Insert `security.isAuthenticated()` plus an administrator role check ahead of the `serveIndex` and `serveLogFiles` registrations at `server.ts:281` and `server.ts:283` , or remove both routes from the deployed configuration.
2. Configure `morgan` at `server.ts:338` with a token format that omits query strings, so future logs cannot carry secrets even if exposed.
3. Add an integration test asserting that an unauthenticated GET `/support/logs` returns HTTP 401.

Verification: Run the new integration test: unauthenticated GET `/support/logs` and GET `/support/logs/access.log` must both return 401.

Reference: [CWE-532](#): Insertion of Sensitive Information into Log File

M-038 — Validate and allowlist outbound request targets

Addresses:

- [F-028](#) — Unvalidated URL fetch in profile image (`profileImageUrlUpload.ts:24`)

Prevents CWEs:

- [CWE-918](#) - Server-Side Request Forgery (SSRF)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/profileImageUrlUpload.ts:24`

How:

1. Before `routes/profileImageUrlUpload.ts:24` , parse the URL and reject anything whose scheme is not https or whose resolved address falls in a loopback, link-local, or private range, re-checking the address after DNS resolution to defeat rebinding.
2. Bound the fetch with an explicit timeout and a maximum response size, and reject responses whose `Content-Type` is not an image type before writing the stream at line 29.
3. Remove the fallback at line 36 that persists the attacker-supplied URL as the profile image on failure.

Example implementation in `routes/profileImageUrlUpload.ts:24` : it applies **Validate and allowlist outbound request targets**. The ordered steps above remain authoritative.

```
// Allowlist external image hosts; block private IP ranges.
const ALLOWED_HOSTS = new Set(['images.example.com', 'cdn.example.com'])
const url = new URL(input)
if (!ALLOWED_HOSTS.has(url.hostname)) throw new Error('host not allowed')
if (/^(10\.|172\.(1[6-9]|2[0-9]|3[01])\.|192\.168\.|127\.)/.test(url.hostname))
  throw new Error('private range blocked')
```

Verification: Run an integration test posting `imageUrl = http://169.254.169.254/` and assert HTTP 400 with no file created under `assets/public/images/uploads`.

Reference: [CWE-918](#): Server-Side Request Forgery (SSRF)

M-039 — Stop exposing internal information to clients

Addresses:

- [F-029](#) — Secrets broadcast to job environment (`ci.yml:253`)

Prevents CWEs:

- [CWE-200](#) - Exposure of Sensitive Information

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `.github/workflows/ci.yml:253`

How:

1. Remove `GITHUB_TOKEN` from the `cypress` step environment at `.github/workflows/ci.yml:253` and 268 unless a specific plugin requires it, and pass the remaining secrets only to the command that consumes them rather than to the whole step.
2. Split application start-up from the Cypress run so that the npm-installed application process does not inherit `CYPRESS_RECORD_KEY` or the solutions webhook.
3. Rotate `ALCHEMY_API_KEY`, `CYPRESS_RECORD_KEY` and `E2E_SOLUTIONS_WEBHOOK`, and constrain the Alchemy key to the test network and to a request quota.

Example implementation in `.github/workflows/ci.yml:253` : it applies **Stop exposing internal information to clients**. The ordered steps above remain authoritative.

```
// Remove directory-listing middleware; require auth on management endpoints.
// app.use('/ftp', serveIndex(...)) // delete
app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: Add a step after the Cypress run that asserts `'env | grep -cE " ALCHEMY_API_KEY | CYPRESS_RECORD_KEY | SOLUTIONS_WEBHOOK "'` returns 0 in the application start-up context, and confirm the e2e suite still passes on a develop push.

Reference: [CWE-200: Exposure of Sensitive Information to an Unauthorized Actor](#)

M-040 — Stop exposing internal information to clients

Addresses:

- [F-030](#) — Unauthenticated data exposure (`registerWebsocketEvents.ts:30`)

Prevents CWEs:

- [CWE-200](#) - Exposure of Sensitive Information

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `lib/startup/registerWebsocketEvents.ts:30`

How:

1. Strip the flag field from the notification payload before emitting on the connect-time replay and the broadcast, and send a flag only to the authenticated socket whose own subject solved that challenge; when `challenges.showSolvedNotifications` is disabled, suppress the emit server-side rather than marking the payload hidden.
2. Add a test that connects a second, unrelated socket after a solve and asserts the received 'challenge solved' payload has no flag property.

*Example implementation in `lib/startup/registerWebsocketEvents.ts:30` : it applies **Stop exposing internal information to clients**. The ordered steps above remain authoritative.*

```
// Remove directory-listing middleware; require auth on management endpoints.  
// app.use('/ftp', serveIndex(...)) // delete  
app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: Solve one challenge, connect a fresh anonymous socket, capture the first 'challenge solved' event and assert `JSON.stringify(payload)` contains no 'flag' key - expect the assertion to pass.

Reference: [CWE-200: Exposure of Sensitive Information to an Unauthorized Actor](#)

M-041 — Move secrets to a managed secret store

Addresses:

- [F-031](#) — Hard-coded wallet mnemonic (`checkKeys.ts:10`)

Weaknesses addressed: [W-004](#)

Prevents CWEs:

- [CWE-798](#) - Use of Hard-coded Credentials

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/checkKeys.ts:10`

How:

1. Replace the literal at `routes/checkKeys.ts:10` with a value read from the runtime secret store or environment, fail the handler closed when the value is absent, and derive a new wallet so the exposed mnemonic no longer controls any address the application uses.
2. Add a CI secret-scanning rule and a unit test that fails the build when a BIP39-length word sequence or private-key literal appears under `routes/`.

*Example implementation in `routes/checkKeys.ts:10`: it applies **Move secrets to a managed secret store**. The ordered steps above remain authoritative.*

```
// Read secrets from env / vault, not the source.
const hmacKey = process.env.ORDER_HMAC_KEY
if (!hmacKey) throw new Error('ORDER_HMAC_KEY not set')
const sig = createHmac('sha256', hmacKey).update(payload).digest('hex')
```

Verification: `grep -rE "([a-z]+ ){11}[a-z]+" routes/checkKeys.ts` returns no match and the service still starts with the mnemonic supplied only through the environment.

Reference: [CWE-798: Use of Hard-coded Credentials](#)

M-042 — Rate-limit and lock out repeated authentication attempts

Addresses:

- [F-032](#) — Rate limiter keyed on client-supplied header (`server.ts:346`)

Prevents CWEs:

- [CWE-307](#) - Improper Restriction of Excessive Authentication Attempts

Priority: P2 - This Sprint · **Effort:** Low · **File:** `server.ts:346`

How:

1. Remove the custom `keyGenerator` at `server.ts:346` and let `express-rate-limit` derive the key from `req.ip`, which Express resolves correctly once trust proxy is restricted.
2. Replace `app.enable('trust proxy')` at `server.ts:342` with `app.set('trust proxy', )` so only the real reverse proxy may set X-Forwarded-For.
3. Add a per-account failed-answer counter in `routes/resetPassword.ts` so the limit survives address rotation.

*Example implementation in `server.ts:346`: it applies **Rate-limit and lock out repeated authentication attempts**. The ordered steps above remain authoritative.*

```
// Per-IP + per-account rate limiting on auth endpoints.
import rateLimit from 'express-rate-limit'
```

```
app.use('/rest/user/login',
  rateLimit({ windowMs: 60_000, max: 5, standardHeaders: true })))
```

Verification: Run an integration test issuing 150 reset-password attempts each with a distinct X-Forwarded-For value and assert that requests past the configured ceiling return 429.

Reference: [CWE-307](#): Improper Restriction of Excessive Authentication Attempts

M-043 — Rate-limit and lock out repeated authentication attempts

Addresses:

- [F-033](#) — No rate limit or lockout on login (`server.ts:596`)

Prevents CWEs:

- [CWE-307](#) - Improper Restriction of Excessive Authentication Attempts

Priority: P2 - This Sprint · **Effort:** Low · **File:** `server.ts:596`

How:

1. Register an `express-rate-limit` middleware ahead of the login handler at `server.ts:596` keyed on the trusted client address, not on a client-supplied header.
2. Add a per-account failed-attempt counter in `routes/login.ts` that applies exponential backoff or a temporary lock after a threshold of consecutive failures.
3. Add an integration test issuing 30 rapid failed logins for one account and asserting that later attempts receive HTTP 429.

*Example implementation in `server.ts:596`: it applies **Rate-limit and lock out repeated authentication attempts**. The ordered steps above remain authoritative.*

```
// Per-IP + per-account rate limiting on auth endpoints.
import rateLimit from 'express-rate-limit'
app.use('/rest/user/login',
  rateLimit({ windowMs: 60_000, max: 5, standardHeaders: true })))
```

Verification: Run the new integration test: after the configured threshold of failed logins from one address, further POST `/rest/user/login` calls must return 429.

Reference: OWASP ASVS V11.1 - Rate limiting on authentication endpoints

M-044 — Rate-limit and lock out repeated authentication attempts

Addresses:

- [F-034](#) — Unbounded LLM consumption on `/rest/chat` (`server.ts:638`)

Weaknesses addressed: [W-006](#)

Prevents CWEs:

- [CWE-770](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `server.ts:638`

How:

1. Register an `express-rate-limit` middleware on `/rest/chat` at `server.ts:638` keyed on the authenticated subject where available and on the trusted client address otherwise.
2. Set `maxOutputTokens` on the `streamText` call at `routes/chat.ts:203` and reject requests whose serialized messages array exceeds a fixed byte budget before the model call.
3. Add an alert on the existing `juice_shop_llm_input_tokens_total` counter so sustained consumption is visible, and add a load test asserting the configured request-per-minute ceiling is enforced.

Verification: Run the load test: requests to `/rest/chat` beyond the configured per-minute ceiling must return HTTP 429, and a single response must never exceed the configured `maxOutputTokens`.

Reference: [CWE-770](#): Allocation of Resources Without Limits or Throttling

M-045 — Bound socket payload size and rewrite the SVG-challenge regex

Addresses:

- [F-035](#) — Inefficient regex complexity (`registerWebsocketEvents.ts:46`)

Prevents CWEs:

- [CWE-1333](#)

Priority: P2 - This Sprint · **Effort:** Low · **File:** `lib/startup/registerWebsocketEvents.ts:46`

How:

1. Reject the payload in the handler unless `typeof data === 'string'` and its length is under a small explicit limit, lower `maxHttpBufferSize` in the Server options at line 20 to that limit, and `replace` the backtracking pattern with an anchored, non-ambiguous expression (or a plain `indexOf` on `'/redirect?to='`) that cannot rescan the input.
2. Add a test that emits `verifySvgInjectionChallenge` with a 200 KB string of repeated `'../..'` and asserts the server answers a subsequent HTTP request within one second.

Verification: Emit the 200 KB payload, then run: `curl -s -o /dev/null -w '%{ time_total }' http://localhost:3000/rest/admin/application-version` - expect a value under 1 second.

Reference: [CWE-1333](#)

M-046 — Require the current password unconditionally

Addresses:

- [F-036](#) — Password change without current password (`changePassword.ts:39`)

Prevents CWEs:

- [CWE-620](#)

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/changePassword.ts:39`

How:

1. Reject the request at `routes/changePassword.ts:39` when `currentPassword` is absent, and compare it with the stored credential before any update, using a constant-time comparison over the KDF output from [F-025](#) — Unsalted MD5 password hashing.
2. Invalidate all other sessions of the account after a successful change, using the revocation introduced for [F-042](#) — Missing session token revocation (`lib/insecurity.ts:54`).
3. Add a test that omits the current parameter and asserts 401 with the stored password unchanged.

Verification: Run the new test: a change-password request without the current parameter must return 401 and the password column of the target user must be unchanged afterwards.

Reference: [CWE-620: Unverified Password Change](#)

M-047 — Enforce authorization on the server

Addresses:

- [F-037](#) — Client-controlled wallet credit amount (`wallet.ts:27`)

Prevents CWEs:

- [CWE-602](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/wallet.ts:27`

How:

1. Before the increment at `routes/wallet.ts:27` , capture the payment against the referenced card through the payment provider and credit only the settled amount returned by that call.
2. Validate `req.body.balance` as a positive number within a configured per-transaction ceiling and reject anything else with HTTP 400.

3. Add an integration test crediting 999999 and asserting HTTP 400 with the stored wallet balance unchanged.

Verification: Run the new integration test: PUT `/rest/wallet/balance` with balance 999999 must return 400 and the wallet row must retain its prior value.

Reference: [CWE-602](#): Client-Side Enforcement of Server-Side Security

M-048 — Enforce correct server-side authorization

Addresses:

- [F-038](#) — Coupon discount bounded only by prompt text (`chat.ts:184`)

Weaknesses addressed: [W-002](#)

Prevents CWEs:

- [CWE-863](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `routes/chat.ts:184`

How:

1. Constrain the `zod` schema at `routes/chat.ts:179` with `z.number().int().min(1).max(10)` so an out-of-range discount is rejected before `execute` runs.
2. Inside the execute body at `routes/chat.ts:181`, resolve the caller with `getUserId` and verify server-side that a damaged order exists for that caller before calling `security.generateCoupon` at line 184.
3. Add an integration test that drives the model to request a 50 percent coupon and asserts the tool returns an error with no coupon code emitted.

Verification: Run the new integration test: a chat exchange that induces `generateCoupon` with discount 50 must yield a tool error and no redeemable coupon code in the stream.

Reference: OWASP Top 10 for LLM Applications 2025 - LLM06 Excessive Agency

M-049 — Enforce correct server-side authorization

Addresses:

- [F-039](#) — Deluxe upgrade skips payment check (`deluxe.ts:43`)

Weaknesses addressed: [W-002](#)

Prevents CWEs:

- [CWE-863](#)

Priority: P2 - This Sprint · **Effort:** Low · **File:** `routes/deluxe.ts:43`

How:

1. Replace the two independent `if` statements at `routes/deluxe.ts:24` and `:34` with a single `switch` or `if/else-if` chain whose default branch returns HTTP 400 before reaching the role update at line 43.
2. Record a settled-payment reference on the user or an orders row and assert it is present immediately before the update call.
3. Add an integration test posting `paymentMode: ""` and asserting HTTP 400 with the user's role still customer.

Verification: Run the new integration test: `POST /rest/deluxe-membership` with an unrecognized `paymentMode` must return 400 and leave the stored role as `customer`.

Reference: [CWE-863](#): Incorrect Authorization

M-050 — Enforce server-side authorization on every endpoint

Addresses:

- [F-040](#) — Sensitive Routes Registered Without Authentication Middleware (`server.ts:310`)

Weaknesses addressed: [W-002](#)

Prevents CWEs:

- [CWE-862](#) - Missing Authorization

Priority: P2 - This Sprint · **Effort:** Medium · **File:** `server.ts:310`

How: Add an explicit auth middleware (`isAuthorized()` / `passport.authenticate()` / `requireAuth`) to the route registration, or mark the route intentionally public via `app.get(...)` for read-only access only.

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

*Example implementation in `server.ts:310` : it applies **Enforce server-side authorization on every endpoint**. The ordered steps above remain authoritative.*

```
// Add server-side role check on every admin route.
router.use('/admin/*', (req, res, next) => {
  if (req.user?.role !== 'admin') return res.status(403).end()
  next()
})
```

Verification: Re-run the AUTHZ-008 scanner check; it reports no match at `server.ts:310`, and the regression test passes.

M-051 — Apply least-privilege filesystem access

Addresses:

- [F-041](#) — Missing workflow permissions block (`ci.yml:190`)

Prevents CWEs:

- [CWE-284](#) - Improper Access Control

Priority: P2 - This Sprint · **Effort:** Low · **File:** `.github/workflows/ci.yml:190`

How:

1. Add a top-level 'permissions: { contents: read }' block to `.github/workflows/ci.yml` and grant wider scopes only on the individual jobs that need them, following the pattern already used in `.github/workflows/codeql-analysis.yml:11-14`.
2. Set the organisation or repository default workflow permission to read-only so that any workflow missing a permissions block fails closed rather than open.
3. Confirm the coverage-report, e2e-test and notify-slack steps still function with the reduced scope, and remove the explicit `GITHUB_TOKEN` inputs wherever the action can use the job token directly.

*Example implementation in `.github/workflows/ci.yml:190`: it applies **Apply least-privilege filesystem access**. The ordered steps above remain authoritative.*

```
// Centralize access decisions in a single middleware.  
function requireRole(role: 'admin' | 'user') {  
  return (req, res, next) =>  
    req.user?.role === role ? next() : res.status(403).end()  
}
```

Verification: Run `grep -c "^permissions:" .github/workflows/ci.yml` and expect 1; then inspect a completed run's 'Set up job' log and confirm the `GITHUB_TOKEN` permissions list shows only `contents: read` for the test jobs.

Reference: [OWASP: CICD SEC 05 Insufficient PBAC](#)

M-078 — Harden the authentication flow

Addresses:

- [F-073](#) — Token flow accepts a stolen bearer token without (`login.component.ts:148`)

Prevents CWEs:

- [CWE-287](#)

Priority: P2 - This Sprint · **Effort:** Medium · **File:**

`frontend/src/app/login/login.component.ts:148`

How: Use authorization-code flow with PKCE

1. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the security scan; the CWE-287 finding at `frontend/src/app/login/login.component.ts:148` is cleared, and the regression test passes.

P3 — Next Quarter

M-001 — Enforce object-level (ownership) authorization

Addresses:

- [F-009](#) — Insecure Direct Object Reference (`address.ts:11`)

Weaknesses addressed: [W-002](#)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/address.ts:11`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ± 20 lines around the cited location (`routes/address.ts:11`) and decide whether to keep, downgrade, or remove this finding.

1. Replace `req.body.UserId / userId / ownerId` with `req.user.id` (or equivalent session-derived identity) in every WHERE/filter clause.
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

*Example implementation in `routes/address.ts:11`: it applies **Enforce object-level (ownership) authorization**. The ordered steps above remain authoritative.*

```
// Ownership check before touching a resource.  
const basket = await Basket.findByPk(req.params.id)  
if (!basket || basket.UserId !== req.user.id) return res.status(403).end()
```

Verification: Re-run the `AUTHZ-001` scanner check; it reports no match at `routes/address.ts:11`, and the regression test passes.

M-002 — Move secrets to a managed secret store

Addresses:

- [F-018](#) — Hard-coded seeded account credentials (`datacreator.ts:200`)

Weaknesses addressed: [W-004](#)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `data/datacreator.ts:200`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ± 20 lines around the cited location (`data/datacreator.ts:200`) and decide whether to keep, downgrade, or remove this finding.

1. Replace the password and `totpSecret` values read from static user data in `data/datacreator.ts:190-200` with values resolved from environment variables, generating a random password per account when the variable is unset and printing it once to the startup log.
2. Add a startup assertion that refuses to boot when a seeded account still carries a repository-tracked password and `NODE_ENV` is not a test or training profile.
3. Add a test asserting that `UserModel` rows created by the seeder do not match any credential literal present in the static data file.

*Example implementation in `data/datacreator.ts:200` : it applies **Move secrets to a managed secret store**. The ordered steps above remain authoritative.*

```
// Read secrets from env / vault, not the source.
const hmacKey = process.env.ORDER_HMAC_KEY
if (!hmacKey) throw new Error('ORDER_HMAC_KEY not set')
const sig = createHmac('sha256', hmacKey).update(payload).digest('hex')
```

Verification: Run the seeder, then execute a login request against the seeded admin email using the password from the static data file and assert the response is HTTP 401 rather than a token.

Reference: [CWE-798](#): Use of Hard-coded Credentials

M-003 — Remove server-side evaluation of untrusted input

Addresses:

- [F-021](#) — Input compiled as template source (`userProfile.ts:87`)

Weaknesses addressed: [W-010](#)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/userProfile.ts:87`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ± 20 lines around the cited location (`routes/userProfile.ts:87`) and decide whether to keep, downgrade, or remove this finding.

1. Keep input as data; remove dynamic code or template-source compilation
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

Verification: Re-run the INJ-NODE-008 scanner check; it reports no match at `routes/userProfile.ts:87` , and the regression test passes.

M-004 — Enforce server-side authorization on every endpoint

Addresses:

- **F-040** — Sensitive Routes Registered Without Authentication Middleware (`server.ts:310`)

Weaknesses addressed: [W-002](#)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `server.ts:310`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ± 20 lines around the cited location (`server.ts:310`) and decide whether to keep, downgrade, or remove this finding.

1. Add an explicit auth middleware (`isAuthorized()` / `passport.authenticate()` / `requireAuth`) to the route registration, or mark the route intentionally public via `app.get(...)` for read-only access only.
2. Add a regression test (or CI check) that fails on the vulnerable pattern and passes once the fix is in place.

*Example implementation in `server.ts:310` : it applies **Enforce server-side authorization on every endpoint**. The ordered steps above remain authoritative.*

```
// Add server-side role check on every admin route.
router.use('/admin/*', (req, res, next) => {
  if (req.user?.role !== 'admin') return res.status(403).end()
  next()
})
```

Verification: Re-run the AUTHZ-008 scanner check; it reports no match at `server.ts:310` , and the regression test passes.

M-005 — Manual review: verify Missing Content-Security-Policy at the cited location

Addresses:

- ● F-048 — Missing Content-Security-Policy

Priority: P3 - Next Quarter · **Effort:** Medium

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ± 20 lines around the cited location (the cited location) and decide whether to keep, downgrade, or remove this finding.

1. Serve a Content-Security-Policy header with a nonce-based script-src, object-src 'none', base-uri 'self' and frame-ancestors 'none' for the SPA's index document.
2. Run the policy in report-only mode first, collect violations from the existing bundle, then enforce.
3. Add a CI check asserting the served index response carries a Content-Security-Policy header without 'unsafe-inline' or 'unsafe-eval'.

Verification: `curl -sI http://localhost:3000/ | grep -i content-security-policy` and assert the header is present and contains neither 'unsafe-inline' nor 'unsafe-eval'.

Reference: OWASP ASVS V5.2 - Sanitization and Sandboxing

M-006 — Manual review: verify Unescaped anchor persisted into product description

Addresses:

- ● F-049 — Unescaped anchor persisted into product description (datacreator.ts:406)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** data/datacreator.ts:406

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ± 20 lines around the cited location (data/datacreator.ts:406) and decide whether to keep, downgrade, or remove this finding.

1. Emit `rel="noopener noreferrer"` on the anchor built at data/datacreator.ts:406 and pass `urlForProductTamperingChallenge` through the `html-entities` encoder already imported at line 41 before concatenating it into the href attribute.
2. Add a test asserting the persisted product description matches `rel="noopener noreferrer"` whenever it contains `target="_blank"`.

Verification: Seed the database, query the tampering product description, and assert it contains `rel="noopener noreferrer"` and no unencoded double quote inside the href value.

Reference: OWASP Cheat Sheet: HTML5 Security - Tabnabbing (`rel=noopener`)

M-007 — Manual review: verify Ineffective author email masking

Addresses:

- ● [F-058](#) — Ineffective author email masking in feedback (`datacreator.ts:578`)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `data/datacreator.ts:578`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ± 20 lines around the cited location (`data/datacreator.ts:578`) and decide whether to keep, downgrade, or remove this finding.

1. Change the author fragment at `data/datacreator.ts:578` to retain at most the first character of the local part and to replace the domain entirely, or drop the author fragment and rely on the `UserId` foreign key for attribution.
2. Add a test asserting that no seeded feedback comment contains a substring matching the configured application domain.

Verification: Seed the database and assert that a query over the Feedback comment column returns zero rows matching the application domain suffix.

Reference: [CWE-359](#): Exposure of Private Personal Information to an Unauthorized Actor

M-008 — Manual review: verify Plaintext security answer written to log

Addresses:

- ● [F-059](#) — Plaintext security answer written to log (`datacreator.ts:692`)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `data/datacreator.ts:692`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ± 20 lines around the cited location (`data/datacreator.ts:692`) and decide whether to keep, downgrade, or remove this finding.

1. Remove the answer interpolation from the `logger.error` call at `data/datacreator.ts:692` and log only `SecurityQuestionId` and `UserId`.
2. Add a log assertion test that fails when a seeded security answer plaintext appears in captured log output.

Verification: Run the seeder with a forced `SecurityAnswerModel.create` rejection and assert the captured log output does not contain the answer plaintext.

Reference: [CWE-532](#): Insertion of Sensitive Information into Log File

M-009 — Move cryptographic keys to a managed secret store

Addresses:

- [F-066](#) — Security answers stored under embedded HMAC key (`datacreator.ts:691`)

Weaknesses addressed: [W-004](#)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `data/datacreator.ts:691`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ± 20 lines around the cited location (`data/datacreator.ts:691`) and decide whether to keep, downgrade, or remove this finding.

1. Replace `security.hmac` in the answer setter at `models/securityAnswer.ts:45` with `bcrypt` at a cost factor of at least 12 so that answers persisted by `data/datacreator.ts:691` are salted and iterated.
2. Move the HMAC key at `lib/insecurity.ts:42` out of the repository into an environment variable for its remaining non-answer uses, and fail startup when it is unset.
3. Add a test asserting a seeded security answer digest differs between two runs for the same plaintext answer.

*Example implementation in `data/datacreator.ts:691`: it applies **Move cryptographic keys to a managed secret store**. The ordered steps above remain authoritative.*

```
// Load the RSA private key from an environment variable / KMS – never
// the source tree. Rotate the prior key and revoke outstanding tokens.
const privateKey = process.env.JWT_PRIVATE_KEY
if (!privateKey) throw new Error('JWT_PRIVATE_KEY not set')
const token = jwt.sign(claims, privateKey, { algorithm: 'RS256' })
```

Verification: Seed the database twice with the same static answers and assert the stored `SecurityAnswer` digests differ between the two runs.

Reference: [CWE-321](#): Use of Hard-coded Cryptographic Key

M-010 — Manual review: verify Entitlement token derived from account email

Addresses:

- [F-067](#) — Entitlement token derived from account email (`datacreator.ts:198`)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `data/datacreator.ts:198`

How: The evidence-verifier sample could not confirm or refute the claim from the cited snippet alone. Have a developer familiar with this code path read ± 20 lines around the cited location (`data/datacreator.ts:198`) and decide whether to keep, downgrade, or remove this finding.

1. Generate the `deluxeToken` at `data/datacreator.ts:198` from `crypto.randomBytes` , persist it per user, and change the membership check to compare the presented value against the stored row rather than recomputing it from the email.
2. Add an expiry or revocation column so a granted entitlement can be withdrawn without changing the account email.
3. Add a test asserting two users with known emails receive different, non-derivable deluxe tokens.

Verification: Seed the database twice and assert the `deluxeToken` value for the same email differs between runs.

Reference: [CWE-1391](#): Use of Weak Credential

M-012 — Store session tokens in HttpOnly, Secure cookies

Addresses:

- [F-002](#) — JWT in localStorage (`request.interceptor.ts:13`)

Prevents CWEs:

- [CWE-922](#) - Insecure Storage of Sensitive Information

Priority: P3 - Next Quarter · **Effort:** High · **File:** `frontend/src/app/Services/request.interceptor.ts:13`

How:

1. Issue the session as an `HttpOnly` , `Secure` , `SameSite=Strict` cookie and delete the `localStorage` token read at `request.interceptor.ts:13` together with the header assembly at line 16.
2. Remove the `X-User-Email` header at `request.interceptor.ts:23` and derive the identity server-side from the session instead of from client-supplied state.
3. Add a test asserting that after login no `token` or `email` key exists in `localStorage` .

*Example implementation in `frontend/src/app/Services/request.interceptor.ts:13` : it applies **Store session tokens in HttpOnly, Secure cookies**. The ordered steps above remain authoritative.*

```
// Move the JWT out of localStorage into an httpOnly cookie.
res.cookie('session', token, {
```

```
httpOnly: true, secure: true, sameSite: 'lax', maxAge: 3600_000
})
```

Verification: Log in through the UI, then evaluate `localStorage.getItem('token')` in the browser console and assert it returns null while authenticated requests still succeed.

Reference: [CWE-922](#): Insecure Storage of Sensitive Information

M-014 — Add security audit logging

Addresses:

- [F-004](#) — Missing server-side security audit logging (`login.ts:50`)

Prevents CWEs:

- [CWE-778](#) - Insufficient Logging

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/login.ts:50`

How:

1. Log a structured event for each authentication outcome - login success, login failure, password change, password reset and 2FA disable - carrying timestamp, user id, source IP and result, without the credential values themselves.
2. Ship those events to an append-only sink that the application process cannot rewrite, and alert on failure bursts per account and per source address.
3. Assert in tests that a failed login produces exactly one authentication-failure event.

*Example implementation in `routes/login.ts:50`: it applies **Add security audit logging**. The ordered steps above remain authoritative.*

```
// Log authn / authz outcomes with stable correlation ids.
logger.info({ event: 'auth.login.fail', userId, ip: req.ip, reqId })
logger.info({ event: 'authz.deny', userId, route: req.path, reqId })
```

Verification: Run the new test: one POST `/rest/user/login` with a wrong password must produce exactly one authentication-failure log record containing the account identifier and source address.

Reference: [OWASP Cheat Sheet: Logging Cheat Sheet](#)

M-015 — Apply least-privilege filesystem access

Addresses:

- [F-005](#) — Unmediated data-store access control (`mongodb.ts:10`)

Prevents CWEs:

- [CWE-284](#) - Improper Access Control

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `data/mongodb.ts:10`

How:

1. Stop exporting the raw MarsDB collections from `data/mongodb.ts`; keep them module-private and export a repository object whose methods require an explicit caller identity and inject an ownership or role predicate into every find, update, and remove query.
2. Add a test that calls each repository method with a non-owner identity against a seeded order document and asserts an empty result for reads and an unchanged document for writes.

*Example implementation in `data/mongodb.ts:10`: it applies **Apply least-privilege filesystem access**. The ordered steps above remain authoritative.*

```
// Centralize access decisions in a single middleware.  
function requireRole(role: 'admin' | 'user') {  
  return (req, res, next) =>  
    req.user?.role === role ? next() : res.status(403).end()  
}
```

Verification: Run the new repository test suite and confirm the non-owner access cases fail closed; additionally grep the codebase for `reviewsCollection / ordersCollection` imports and confirm zero hits outside the repository module.

Reference: [CWE-284: Improper Access Control](#)

M-023 — Replace security-question reset in routes/resetPassword.ts with emailed one-time tokens

Addresses:

- [F-013](#) — Weak password recovery mechanism (`resetPassword.ts:41`)

Prevents CWEs:

- [CWE-640](#) - Weak Password Recovery Mechanism

Priority: P3 - Next Quarter · **Effort:** High · **File:** `routes/resetPassword.ts:41`

How:

1. Replace the answer comparison at `routes/resetPassword.ts:41` with a flow that mails a single-use, time-limited, cryptographically random token to the stored address and accepts the new password only together with that token.

2. Persist the token hashed with an expiry and mark it consumed inside the same transaction that updates the password, so a token cannot be replayed.
3. Return an identical response whether or not the address exists, to keep the endpoint from confirming account existence.

Verification: Add an integration test asserting that `POST /rest/user/reset-password` with a correct security answer but no reset token returns 401, and that a consumed token returns 401 on second use.

Reference: [OWASP Cheat Sheet: Forgot Password Cheat Sheet](#)

M-052 — Add server-side token revocation to the session store

Addresses:

- ● [F-042](#) — Missing session token revocation (`insecurity.ts:54`)

Prevents CWEs:

- [CWE-613](#)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `lib/insecurity.ts:54`

How:

1. Add a jti claim in `authorize()` at `lib/insecurity.ts:54` and a `revoke(jti)` operation on the `authenticatedUsers` store, checked by `isAuthorized()` on every request.
2. Call that revocation from logout, password change and 2FA disable so all sessions of the affected user are retired.
3. Shorten the access-token lifetime and move long-lived sessions to a refresh token that can be revoked centrally.

Verification: Add a test that logs in, changes the password, then reuses the original token against `/rest/user/whoami` and asserts 401.

Reference: [OWASP Cheat Sheet: Session Management Cheat Sheet](#)

M-053 — Require a signed nonce challenge before registering a wallet address

Addresses:

- ● [F-043](#) — Wallet identity accepted without proof of possession (`web3Wallet.ts:16`)

Prevents CWEs:

- [CWE-290](#)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/web3Wallet.ts:16`

How:

1. Before `routes/web3Wallet.ts:16`, issue a server-generated single-use nonce and require the caller to return it signed by the claimed address, verifying with `ethers.verifyMessage()` and inserting into `walletsConnected` only when the recovered address equals `req.body.walletAddress`.
2. Add a route test that asserts `POST /rest/web3/walletExploitAddress` with a wallet address but no valid signature returns 401 and leaves `walletsConnected` unchanged.

Verification: `curl -sS -X POST -H 'Content-Type: application/json' -d '{"walletAddress":"0x0000000000000000000000000000000000000000000000000000000000000001"}' http://localhost:3000/rest/web3/walletExploitAddress` returns 401 instead of the current success message.

Reference: [CWE-290: Authentication Bypass by Spoofing](#)

M-054 — Validate redirect targets against an allowlist

Addresses:

- [F-044](#) — Open redirect (`redirect.ts:19`)

Prevents CWEs:

- [CWE-601](#)

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/redirect.ts:19`

How:

1. In `routes/redirect.ts:16`, parse `toUrl` with the WHATWG URL parser and compare the resulting origin (scheme plus host plus port) for exact equality against the entries of `security.redirectAllowlist` instead of relying on `security.isRedirectAllowed`, rejecting any parse failure or non-https scheme with the existing 406 path at lines 21-22.
2. Add a route test that asserts `GET /redirect?to=https://evil.example/?x=https://github.com/juice-shop/juice-shop` returns 406 and emits no Location header, alongside an existing-behaviour test that an allowlisted origin still returns 302.

Verification: `curl -sS -o /dev/null -w '%{ http_code } %{ redirect_url } \n' 'http://localhost:3000/redirect?to=https://evil.example/?x=https://github.com/juice-shop/juice-shop'` returns 406 with an empty `redirect_url`.

Reference: [CWE-601: URL Redirection to Untrusted Site \(Open Redirect\)](#)

M-055 — Add anti-CSRF protection to state-changing requests

Addresses:

- [F-045](#) — Cookie-only authentication on POST `/profile` (`updateUserProfile.ts:17`)

Prevents CWEs:

- [CWE-352](#) - Cross-Site Request Forgery (CSRF)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/updateUserProfile.ts:17`

How:

1. Set the session cookie with `{ httpOnly: true, sameSite: 'strict', secure: true }` at `lib/insecurity.ts:192` and in `routes/updateUserProfile.ts:42`.
2. Add a synchronizer-token or double-submit check to the `/profile` POST handler and reject requests whose token is missing or mismatched before the `user.update` at `routes/updateUserProfile.ts:38`.
3. Replace the blanket `app.use(cors())` at `server.ts:183` with an explicit origin allowlist for the deployed frontend.

*Example implementation in `routes/updateUserProfile.ts:17`: it applies **Add anti-CSRF protection to state-changing requests**. The ordered steps above remain authoritative.*

```
// Use double-submit cookie / SameSite=Strict for state-changing routes.  
app.use(csrf({ cookie: { sameSite: 'strict', httpOnly: true, secure: true } })))
```

Verification: Run an integration test that posts to `/profile` with a valid session cookie but no CSRF token and assert HTTP 403 with the username unchanged.

Reference: OWASP ASVS V4.2 - Operation-level access control and CSRF defences

M-056 — Verify token signatures before trusting claims

Addresses:

- [F-046](#) — Remote script piped to shell (`ci.yml:358`)

Prevents CWEs:

- [CWE-345](#)

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `.github/workflows/ci.yml:358`

How:

1. Replace the curl-to-shell step at `.github/workflows/ci.yml:358` with a SHA-pinned setup action for the Heroku CLI, or `download` a specific pinned release tarball and verify its published `SHA-256` with `'sha256sum -c'` before executing anything.
2. Scope the `HEROKU_API_KEY` secret to the single deploy step rather than leaving it available to earlier steps in the same job.
3. Add a workflow lint rule that rejects any 'run:' line matching a `download` piped into a shell.

Verification: Run `'grep -nE "curl[^\]]|\s(sh|bash)" .github/workflows/ci.yml'`; it must return no matches, and the heroku job must still deploy successfully on a develop push.

Reference: [CWE-345: Insufficient Verification of Data Authenticity](#)

M-057 — Pin the container base image to an immutable digest

Addresses:

- [F-047](#) — Unpinned container base image — `Dockerfile:1` (`Dockerfile:1`)

Prevents CWEs:

- [CWE-1104](#) - Use of Unmaintained Third-Party Components

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `Dockerfile:1`

How:

1. Change `Dockerfile:1` to `'FROM node:24@sha256: <digest> AS installer'` and `Dockerfile:22` to `'FROM gcr.io/distroless/nodejs24-debian13@sha256: <digest>'`, recording the human-readable tag in a trailing comment.
2. Pin the image reference in `docker-compose.test.yml:7` to the digest or to an immutable release tag instead of `:latest`.
3. Let Dependabot or Renovate raise pull requests that bump these digests, so updates remain visible in the `diff`.

*Example implementation in `Dockerfile:1`: it applies **Pin the container base image to an immutable digest**. The ordered steps above remain authoritative.*

```
# Pin and audit dependencies; fail CI on known vulns.
npm audit --omit=dev --audit-level=high
# Upgrade unmaintained packages in package.json, then:
npm install && npm test
```

Verification: Run `'grep -nE "^FROM " Dockerfile'`; every line must contain '@sha256:', and `'docker build .'` must still complete successfully.

Reference: [CWE-1104: Use of Unmaintained Third Party Components](#)

M-058 — Add a script-src and frame-ancestors Content-Security-Policy

Addresses:

- [F-048](#) — Missing Content-Security-Policy

Prevents CWEs:

- [CWE-693](#)

Priority: P3 - Next Quarter · **Effort:** Medium

How:

1. Serve a `Content-Security-Policy` header with a nonce-based script-src, object-src 'none', base-uri 'self' and frame-ancestors 'none' for the SPA's index document.
2. Run the policy in report-only mode first, collect violations from the existing bundle, then enforce.
3. Add a CI check asserting the served index response carries a `Content-Security-Policy` header without 'unsafe-inline' or 'unsafe-eval'.

Verification: `curl -sI http://localhost:3000/ | grep -i content-security-policy` and assert the header is present and contains neither 'unsafe-inline' nor 'unsafe-eval'.

Reference: OWASP ASVS V5.2 - Sanitization and Sandboxing

M-059 — Add rel=noopener noreferrer and escape the href

Addresses:

- [F-049](#) — Unescaped anchor persisted into product description (`datacreator.ts:406`)

Prevents CWEs:

- [CWE-116](#)

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `data/datacreator.ts:406`

How:

1. Emit `rel="noopener noreferrer"` on the anchor built at `data/datacreator.ts:406` and pass `urlForProductTamperingChallenge` through the `html-entities` encoder already imported at line 41 before concatenating it into the href attribute.
2. Add a test asserting the persisted product description matches `rel="noopener noreferrer"` whenever it contains `target="_blank"`.

Verification: Seed the database, query the tampering product description, and assert it contains `rel="noopener noreferrer"` and no unencoded double quote inside the href value.

Reference: OWASP Cheat Sheet: HTML5 Security - Tabnabbing (`rel=noopener`)

M-060 — Add security audit logging

Addresses:

- **F-050** — Mutable image tag without build attestation (`ci.yml:345`)

Prevents CWEs:

- **CWE-778** - Insufficient Logging

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `.github/workflows/ci.yml:345`

How:

1. Add an immutable tag derived from the package version and commit SHA next to the moving 'latest' tag at `.github/workflows/ci.yml:345`, so each published build keeps a permanent, non-overwritable reference.
2. Add actions/attest-build-provenance (or the equivalent buildx attestation output) to the docker job so each push emits a signed SLSA provenance statement binding the image digest to the workflow run and commit.
3. Record the published image digest in the GitHub release notes or a Slack message so the mapping from run to artifact survives outside the registry.

*Example implementation in `.github/workflows/ci.yml:345`: it applies **Add security audit logging**. The ordered steps above remain authoritative.*

```
// Log authn / authz outcomes with stable correlation ids.  
logger.info({ event: 'auth.login.fail', userId, ip: req.ip, reqId })  
logger.info({ event: 'authz.deny', userId, route: req.path, reqId })
```

Verification: After a develop push, run 'gh attestation verify oci://bkimminich/ juice-shop:snapshot --repo juice-shop/juice-shop'; it must report a verified provenance statement naming the triggering workflow run.

Reference: [CWE-778: Insufficient Logging](#)

M-061 — Neutralise the credential-validity oracle

Addresses:

- **F-051** — Credential validity oracle before 2FA (`login.ts:38`)

Prevents CWEs:

- **CWE-204**

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `routes/login.ts:38`

How:

1. Return an indistinguishable response shape for wrong credentials and for the pending-second-factor case at `routes/login.ts:38`, keeping the 2FA continuation state in a short-lived opaque handle rather than in a body that only correct credentials receive.
2. Count a pending-second-factor response as a failed attempt for lockout and alerting purposes so the endpoint cannot be used as a free oracle.
3. Add the outcome to the authentication audit events introduced for ● [F-004](#) — Missing server-side security audit logging.

Verification: Add a test comparing the status code, body length and timing class of a wrong-password login and a correct-password 2FA-pending login and assert the response shapes are identical.

Reference: [CWE-204](#)

M-062 — Stop exposing internal information to clients

Addresses:

- ● [F-052](#) — Chat tool calls streamed to every caller (`chat.ts:228`)

Prevents CWEs:

- [CWE-200](#) - Exposure of Sensitive Information

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/chat.ts:228`

How:

1. Wrap the `res.write` at `routes/chat.ts:228` in an explicit condition requiring the decoded token role to equal `roles.admin`, leaving the challenge predicate at line 225 unchanged.
2. Move the confidential escalation-discount rule out of the system prompt at `routes/chat.ts:105` and enforce it as a server-side policy check on the coupon tool instead.
3. Add an integration test that posts to `/rest/chat` without a token, drives a `searchProducts` call, and asserts the SSE stream contains no `tool_calls` object.

*Example implementation in `routes/chat.ts:228`: it applies **Stop exposing internal information to clients**. The ordered steps above remain authoritative.*

```
// Remove directory-listing middleware; require auth on management endpoints.
// app.use('/ftp', serveIndex(...)) // delete
app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: Run the new integration test: an anonymous chat exchange that triggers a tool must produce an SSE stream with no `tool_calls` key, while an admin token must still receive it.

Reference: [CWE-200](#): Exposure of Sensitive Information to an Unauthorized Actor

M-063 — Stop exposing internal information to clients

Addresses:

- [F-053](#) — Admin configuration endpoints exposed unauthenticated (`server.ts:607`)

Prevents CWEs:

- [CWE-200](#) - Exposure of Sensitive Information

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:607`

How:

1. Register `app.use('/rest/admin', security.isAuthenticated())` plus an administrator role check in the authorization block before the route definitions at `server.ts:606-607`.
2. Strip secrets and internal endpoint URLs, including the `chatBot` `llmApiUrl`, from the configuration object the handler returns.
3. Add an integration test asserting that unauthenticated GET `/rest/admin/application-configuration` returns HTTP 401.

*Example implementation in `server.ts:607`: it applies **Stop exposing internal information to clients**. The ordered steps above remain authoritative.*

```
// Remove directory-listing middleware; require auth on management endpoints.
// app.use('/ftp', serveIndex(...)) // delete
app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: Run the new integration test: unauthenticated GET of both `/rest/admin` routes must return 401, and an administrator token must still return 200.

Reference: [CWE-200](#): Exposure of Sensitive Information to an Unauthorized Actor

M-064 — Return generic error messages to clients

Addresses:

- [F-054](#) — Development error handler returns stack traces (`server.ts:682`)

Prevents CWEs:

- [CWE-209](#)

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:682`

How:

1. Register `errorhandler()` at `server.ts:682` only when `process.env.NODE_ENV` is development, and register a terminal handler that returns a generic message plus a correlation id otherwise.
2. Log the full error with its correlation id server-side so operators retain the detail the response no longer carries.
3. Add an integration test that forces a 500 with `NODE_ENV=production` and asserts the body contains no stack, no absolute path, and no version string.

Verification: Run the new integration test with `NODE_ENV=production`: the 500 response body must contain a correlation id and must not match `/at .*\.ts:\d+\/`.

Reference: [CWE-209](#): Generation of Error Message Containing Sensitive Information

M-065 — Stop exposing internal information to clients

Addresses:

- [F-055](#) — Client-gated tool-call disclosure (`chat-conversation.component.ts:68`)

Prevents CWEs:

- [CWE-200](#) - Exposure of Sensitive Information

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `frontend/src/app/chatbot/chat-conversation/chat-conversation.component.ts:68`

How:

1. Stop emitting `tool_calls` in the `/rest/chat` stream unless the authenticated caller holds the administrator role, so the data never reaches a non-admin browser regardless of the cookie.
2. Keep the cookie read at `chat-conversation.component.ts:68` as a display preference only, and remove any assumption that it represents an authorization decision.
3. Add a test asserting that a non-admin session receives no `deltaToolCalls` chunks even with `show_tool_calls=true` set.

Example implementation in `frontend/src/app/chatbot/chat-conversation/chat-conversation.component.ts:68`: it applies **Stop exposing internal information to clients**. The ordered steps above remain authoritative.

```
// Remove directory-listing middleware; require auth on management endpoints.
// app.use('/ftp', serveIndex(...)) // delete
app.use('/metrics', requireRole('admin'), promBundle())
```

Verification: As an anonymous user set `document.cookie = 'show_tool_calls=true'`, send a chat message, and assert the SSE response contains no `tool_calls` field.

Reference: [CWE-200](#): Exposure of Sensitive Information to an Unauthorized Actor

M-066 — Move secrets to a managed secret store

Addresses:

- [F-056](#) — Hard-coded test credential (`login.component.ts:62`)

Weaknesses addressed: [W-004](#)

Prevents CWEs:

- [CWE-798](#) - Use of Hard-coded Credentials

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `frontend/src/app/login/login.component.ts:62`

How:

1. Delete the `testingUsername` and `testingPassword` fields at `login.component.ts:61-62`; they have no consumer in the component or its template.
2. Move any credential needed by end-to-end tests into the test harness configuration so it is never compiled into the shipped bundle.
3. Rotate or remove the `testing@juice-sh.op` account in every deployed environment.
4. Add a secret-scanning CI step over `frontend/src` that fails the build on credential literals.

*Example implementation in `frontend/src/app/login/login.component.ts:62`: it applies **Move secrets to a managed secret store**. The ordered steps above remain authoritative.*

```
// Read secrets from env / vault, not the source.
const hmacKey = process.env.ORDER_HMAC_KEY
if (!hmacKey) throw new Error('ORDER_HMAC_KEY not set')
const sig = createHmac('sha256', hmacKey).update(payload).digest('hex')
```

Verification: Build the frontend and grep the emitted bundle for `'IamU**** (17 chars)'`; assert zero matches.

Reference: [CWE-798](#): Use of Hard-coded Credentials

M-067 — Set secure session-cookie attributes

Addresses:

- [F-057](#) — Session cookie without HttpOnly (`login.component.ts:104`)

Weaknesses addressed: [W-011](#)

Prevents CWEs:

- [CWE-1004](#)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `frontend/src/app/login/login.component.ts:104`

How:

1. Remove the client-side cookie write at `login.component.ts:104` and the matching write at `oauth.component.ts:50`, and have the login endpoint return the session as a `Set-Cookie` header carrying `HttpOnly`, `Secure` and `SameSite=Strict`.
2. Keep the expiry policy on the server-issued cookie so session lifetime stays enforced where it cannot be edited.
3. Add a test asserting `document.cookie` contains no 'token' entry after a successful login.

Verification: Log in and inspect the `Set-Cookie` response header; assert it contains `HttpOnly`, `Secure` and `SameSite` and that `document.cookie` does not expose the token.

Reference: [CWE-1004](#): Sensitive `Cookie` Without `HttpOnly` Flag

M-068 — Mask the full local part and domain of the author address

Addresses:

- [F-058](#) — Ineffective author email masking in feedback (`datacreator.ts:578`)

Prevents CWEs:

- [CWE-359](#)

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `data/datacreator.ts:578`

How:

1. Change the author fragment at `data/datacreator.ts:578` to retain at most the first character of the local part and to `replace` the domain entirely, or drop the author fragment and rely on the `UserId` foreign key for attribution.
2. Add a test asserting that no seeded feedback comment contains a substring matching the configured application domain.

Verification: Seed the database and assert that a query over the Feedback comment column returns zero rows matching the application domain suffix.

Reference: [CWE-359](#): Exposure of Private Personal Information to an Unauthorized Actor

M-069 — Redact the answer value from the createSecurityAnswer error log

Addresses:

- [F-059](#) — Plaintext security answer written to log (`datacreator.ts:692`)

Prevents CWEs:

- [CWE-532](#)

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `data/datacreator.ts:692`

How:

1. Remove the answer interpolation from the `logger.error` call at `data/datacreator.ts:692` and log only `SecurityQuestionId` and `UserId`.
2. Add a log assertion test that fails when a seeded security answer plaintext appears in captured log output.

Verification: Run the seeder with a forced `SecurityAnswerModel.create` rejection and assert the captured log output does not contain the answer plaintext.

Reference: [CWE-532](#): Insertion of Sensitive Information into Log File

M-070 — Return generic error messages to clients

Addresses:

- [F-060](#) — Raw provider error returned to caller (`web3Wallet.ts:36`)

Prevents CWEs:

- [CWE-209](#)

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/web3Wallet.ts:36`

How:

1. In `routes/web3Wallet.ts:36` and the matching catch blocks in `routes/nftMint.ts`, log the error through the existing logger and respond with a fixed generic message plus a correlation id instead of `utils.getErrorMessage(error)`.
2. Add a test that forces provider construction to throw and asserts the HTTP body contains neither the provider host, the URL, nor the `ALCHEMY_API_KEY` value.

Verification: Start the service with an invalid `ALCHEMY_API_KEY`, call `POST /rest/web3/walletExploitAddress`, and confirm the 500 body is the generic message while the detailed error appears only in the server log.

Reference: [CWE-209: Generation of Error Message Containing Sensitive Information](#)

M-071 — Rate-limit and lock out repeated authentication attempts

Addresses:

- [F-061](#) — Unbounded in-memory session map (`insecurity.ts:74`)

Weaknesses addressed: [W-006](#)

Prevents CWEs:

- [CWE-770](#)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `lib/insecurity.ts:74`

How:

1. Replace the plain object at `lib/insecurity.ts:70` with a bounded LRU or TTL cache whose entry lifetime matches the token expiry, so entries are evicted automatically.
2. Stop caching the full user record and keep only the identifiers the request path needs, reducing per-entry cost.
3. Add a metric for session-map size and alert when it exceeds the expected active-session count.

Verification: Add a test that inserts more entries than the configured cap and asserts the map size stays at the cap while the oldest entry is gone.

Reference: [CWE-770](#)

M-072 — Offload CPU-bound work and bound execution time

Addresses:

- [F-062](#) — Unbounded CI matrix fan-out (`ci.yml:58`)

Weaknesses addressed: [W-009](#)

Prevents CWEs:

- [CWE-400](#) - Uncontrolled Resource Consumption

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `.github/workflows/ci.yml:58`

How:

1. Add a top-level 'concurrency: { group: \${{ github.workflow }}-\${{ github.ref }}, cancel-in-progress: true }' block to `.github/workflows/ci.yml` so a new push cancels the superseded run.
2. Declare an explicit 'timeout-minutes' on every job in `ci.yml`, sized to observed runtimes, with the custom-config-test and e2e-test jobs capped well below the platform default of 360 minutes.
3. Restrict the unguarded e2e-test, custom-config-test, smoke-test and docker-test jobs on fork pull requests to a single operating-system combination, matching the guard already applied at line 62.

Verification: Run 'grep -c "^concurrency:" `.github/workflows/ci.yml`' and expect 1, and 'grep -c " timeout-minutes:" `.github/workflows/ci.yml`' and expect a count equal to the number of jobs; then push twice in quick succession and confirm the first run is cancelled.

Reference: [CWE-400: Uncontrolled Resource Consumption](#)

M-073 — Rate-limit expensive requests and bound input size

Addresses:

- [F-063](#) — Unbounded chat history resend (`chat-conversation.component.ts:131`)

Weaknesses addressed: [W-009](#)

Prevents CWEs:

- [CWE-400](#) - Uncontrolled Resource Consumption

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `frontend/src/app/chatbot/chat-conversation/chat-conversation.component.ts:131`

How:

1. Truncate `apiMessages` at `chat-conversation.component.ts:126` to a bounded recent window, or summarise older turns, so request size stops growing with conversation length.
2. Enforce the authoritative limit server-side with a per-user rate limit and a `max_tokens` cap on `/rest/chat`, since a browser-side cap is not a boundary.
3. Add a test asserting that a conversation of 200 stored messages produces a request whose message array is within the configured window.

Verification: Replay 200 chat turns and assert each outbound `/rest/chat` body carries at most the configured number of messages and that excess requests receive HTTP 429.

Reference: [CWE-400: Uncontrolled Resource Consumption](#)

M-074 — Rate-limit expensive requests and bound input size

Addresses:

- [F-064](#) — Listener guard race opens unbounded providers (`nftMint.ts:16`)

Weaknesses addressed: [W-009](#)

Prevents CWEs:

- [CWE-400](#) - Uncontrolled Resource Consumption

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `routes/nftMint.ts:16`

How:

1. Replace the boolean guard in `routes/nftMint.ts` with a module-level promise that is assigned synchronously on first entry and awaited by every later request, so provider and listener construction happens exactly once, and apply the same change to the equivalent guard in `routes/web3Wallet.ts`.
2. Add a test that fires fifty concurrent requests at the route and asserts the provider constructor is invoked exactly once.

Verification: Run fifty parallel requests against `/rest/web3/nftMintListen` with the provider constructor spied on and assert a single invocation and a single registered NFTMinted listener.

Reference: [CWE-400: Uncontrolled Resource Consumption](#)

M-075 — Enforce object-level (ownership) authorization

Addresses:

- [F-065](#) — Basket item update without ownership check (`server.ts:426`)

Weaknesses addressed: [W-002](#)

Prevents CWEs:

- [CWE-639](#) - `Authorization` Bypass Through User-Controlled Key (IDOR)

Priority: P3 - Next Quarter · **Effort:** Low · **File:** `server.ts:426`

How:

1. Change the lookup at `routes/basketItems.ts:68` to where: `{ id: req.params.id , BasketId: security.authenticatedUsers.from(req)?.bid }` and return HTTP 403 when no row matches.
2. Add an ownership predicate to the finale BasketItem resource so the generated update cannot bypass the pre-check handler.

3. Add an integration test where user A attempts to update user B's basket item id and asserts HTTP 403 with the item unchanged.

Example implementation in `server.ts:426`: it applies **Enforce object-level (ownership) authorization**. The ordered steps above remain authoritative.

```
// Ownership check before touching a resource.  
const basket = await Basket.findByPk(req.params.id)  
if (!basket || basket.UserId !== req.user.id) return res.status(403).end()
```

Verification: Run the new integration test: user A's PUT against user B's basket item must return 403 and user B's stored quantity must be unchanged.

Reference: [CWE-639](#): Authorization Bypass Through User-Controlled Key

M-076 — Move cryptographic keys to a managed secret store

Addresses:

- [F-066](#) — Security answers stored under embedded HMAC key (`datacreator.ts:691`)

Weaknesses addressed: [W-004](#)

Prevents CWEs:

- [CWE-321](#) - Use of Hard-coded Cryptographic Key

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `data/datacreator.ts:691`

How:

1. Replace `security.hmac` in the answer setter at `models/securityAnswer.ts:45` with `bcrypt` at a cost factor of at least 12 so that answers persisted by `data/datacreator.ts:691` are salted and iterated.
2. Move the HMAC key at `lib/insecurity.ts:42` out of the repository into an environment variable for its remaining non-answer uses, and fail startup when it is unset.
3. Add a test asserting a seeded security answer digest differs between two runs for the same plaintext answer.

Example implementation in `data/datacreator.ts:691`: it applies **Move cryptographic keys to a managed secret store**. The ordered steps above remain authoritative.

```
// Load the RSA private key from an environment variable / KMS – never  
// the source tree. Rotate the prior key and revoke outstanding tokens.  
const privateKey = process.env.JWT_PRIVATE_KEY  
if (!privateKey) throw new Error('JWT_PRIVATE_KEY not set')  
const token = jwt.sign(claims, privateKey, { algorithm: 'RS256' })
```

Verification: Seed the database twice with the same static answers and assert the stored SecurityAnswer digests differ between the two runs.

Reference: [CWE-321](#): Use of Hard-coded Cryptographic Key

M-077 — Persist a random per-user deluxe entitlement token

Addresses:

- [F-067](#) — Entitlement token derived from account email (`datacreator.ts:198`)

Prevents CWEs:

- [CWE-1391](#)

Priority: P3 - Next Quarter · **Effort:** Medium · **File:** `data/datacreator.ts:198`

How:

1. Generate the `deluxeToken` at `data/datacreator.ts:198` from `crypto.randomBytes` , persist it per user, and change the membership check to compare the presented value against the stored row rather than recomputing it from the email.
2. Add an expiry or revocation column so a granted entitlement can be withdrawn without changing the account email.
3. Add a test asserting two users with known emails receive different, non-derivable deluxe tokens.

Verification: Seed the database twice and assert the `deluxeToken` value for the same email differs between runs.

Reference: [CWE-1391](#): Use of Weak Credential

P4 — Backlog

No P4 mitigations.

11. Out of Scope

Not Covered by This Method

This is a code-derived threat model at implementation level: it is built from source, configuration and git history, models the system as built rather than as designed, and does not `replace` a design-time review.

- Design intent and the reasoning behind it - no design documents, ADRs or workshop context are read.

- Business processes and user journeys that leave the code.
- Runtime behaviour, deployment topology and production-only configuration.
- External and organizational controls.

Treat this report as review input, not sign-off.

Excluded from This Assessment

The following items are **explicitly excluded** from this threat model. Findings against these areas should be tracked separately.

- Third-party hosted dependencies and SaaS endpoints
- Browser runtime vulnerabilities and end-user device security
- Operating system kernel and container runtime
- Underlying network infrastructure (DNS, BGP, ISP)
- Physical security of hosting facilities

Appendix: Run Statistics

Field	Value
Invocation	<code>/appsec-advisor:create-threat-model --thorough --sarif --pentest-tasks --pentest-format strix --pentest-target http://localhost:3000 --threatdragon --pdf --html --slug juice-shop-thorough-v0.6.0b3 --keep-runtime-files --skip-context</code>
Generated	2026-09-13 09:16 UTC
Mode	full
Assessment depth	thorough
Plugin version	0.6.0-beta.3 (analysis v5)
Orchestrator model	opus
Reasoning models	opus - STRIDE opus, triage opus, merger opus
Repository	<code>/home/mrohr/juice-shop2</code>
Output directory	<code>/home/mrohr/juice-shop2/docs/security</code>
Wall clock (active)	103m 39s
Agent compute (Σ parallel dispatches)	92m 22s

Per-Stage Breakdown

Stage	Description	Agent	Model	Duration	Tool calls	Tokens
1	recon_scanner	appsec-recon-scanner	haiku	2m 56s	20	91,850
1	actor_discoverer	appsec-actor-discoverer	sonnet	3m 42s	18	87,836
1	architecture_analyst	appsec-architecture-analyst	sonnet	8m 47s	34	99,181
1	trust_boundary_analyst	appsec-trust-boundary-analyst	sonnet	5m 09s	15	63,645
1	control_analyst	appsec-control-analyst	sonnet	6m 24s	12	71,314
1	stride_analyzer	appsec-stride-analyzer-v2	opus	32m 41s	235	3,006,729
1	threat_merger	appsec-threat-merger	opus	3m 56s	12	63,197
1	post_stride_synthesizer	appsec-post-stride-synthesizer	opus	1m 07s	7	42,779
1	evidence_verifier	appsec-evidence-verifier	sonnet	9m 15s	19	96,930
1	Abuse Case Verification	appsec-abuse-case-verifier	sonnet	9m 53s	59	121,712
2	parallel	appsec-secarch-renderer	sonnet	8m 28s	48	132,632
3	qa-gate	qa_checks.py	none	-	0	0
Total	-	-	-	92m 22s	479	3,877,805

Per-Phase Duration Breakdown

Phase	Description	Agent (Model)	Duration
Phase 11	Finalization (controller compose)	threat-analyst (sonnet-4-6)	11m 35s

Appendix A — Vektor Taxonomy

This appendix defines the attacker-starting-position labels used in the Top Threats table and throughout [§8 Findings Register](#). Each label answers the question *what does the attacker need before the exploit begins?*

Internet Anon

Attacker position: Unauthenticated attacker from the public internet · **Breach distance:** 1

Preconditions:

- Endpoint is reachable from the internet (no IP allowlist, no VPN)
- No authentication middleware blocks the request

Typical CWEs: [CWE-89](#) · [CWE-79](#) · [CWE-306](#) · [CWE-327](#) · [CWE-611](#) · [CWE-918](#)

Typical OWASP Top 10: A01:2025, A05:2025, A07:2025

Internet User

Attacker position: Any authenticated low-privilege user (valid JWT / session) · **Breach distance:** 2

Preconditions:

- Attacker has signed up or otherwise obtained a valid user session
- Endpoint is behind auth but not behind role/admin checks

Typical CWEs: [CWE-434](#) · [CWE-611](#) · [CWE-918](#) · [CWE-352](#) · [CWE-287](#)

Typical OWASP Top 10: A01:2025, A06:2025, A02:2025

Internet Priv User

Attacker position: Authenticated admin-level user (JWT with admin role or equivalent) · **Breach distance:** 2

Preconditions:

- Attacker holds admin credentials or has elevated privileges
- Endpoint gated on admin role but still exploitable once reached

Typical CWEs: [CWE-862](#) · [CWE-79](#) · [CWE-94](#)

Typical OWASP Top 10: A01:2025

Victim-Required

Attacker position: Attacker needs victim interaction - social engineering, crafted link, or live session · **Breach distance:** 2

Preconditions:

- Victim must click a link, load a page, or have an active session
- Applies to XSS, CSRF, click-jacking, open redirect

Typical CWEs: [CWE-79](#) · [CWE-352](#) · [CWE-601](#) · [CWE-1021](#)

Typical OWASP Top 10: A01:2025, A05:2025

Build-Time

Attacker position: Attacker controls a build input - CI runner, dependency, base image, or external data fetched during build · **Breach distance:** 3

Preconditions:

- Compromise of a dependency, registry, or base image
- OR compromise of a CI runner with write access to artifacts

Typical CWEs: [CWE-506](#) · [CWE-829](#) · [CWE-1039](#) · [CWE-1104](#)

Typical OWASP Top 10: A08:2025

Repo-Read

Attacker position: Attacker gains read access to source repository (leaked clone, forked fork, insider, compromised developer workstation) · **Breach distance:** 3

Preconditions:

- Read access to the source tree at or after commit time
- No runtime exploit needed - the vulnerability is the content of the repo

Typical CWEs: [CWE-798](#) · [CWE-312](#) · [CWE-540](#)

Typical OWASP Top 10: A04:2025, A07:2025

n/a

Attacker position: Architectural / meta-finding - no runtime entry point, the finding describes a design defect aggregating multiple code-level findings

Preconditions:

- Finding is AF-NNN (architectural) rather than F-NNN (code-level)